

**MINISTÉRIO DA DEFESA
EXÉRCITO BRASILEIRO
DEPARTAMENTO DE CIÊNCIA E TECNOLOGIA
INSTITUTO MILITAR DE ENGENHARIA
CURSO DE GRADUAÇÃO EM ENGENHARIA DE COMPUTAÇÃO**

**JOÃO PEDRO BANDEIRA BELCHIOR
CARLOS ALEXANDRE SIQUEIRA DE ALMEIDA**

**UTILIZAÇÃO DE INTELIGÊNCIA ARTIFICIAL
PARA ANÁLISE E PROSPECÇÃO TECNOLÓGICA
ATRAVÉS DA CONSULTA A ARTIGOS E PATENTES**

**RIO DE JANEIRO
2026**

**JOÃO PEDRO BANDEIRA BELCHIOR
CARLOS ALEXANDRE SIQUEIRA DE ALMEIDA**

**UTILIZAÇÃO DE INTELIGÊNCIA ARTIFICIAL
PARA ANÁLISE E PROSPECÇÃO TECNOLÓGICA
ATRAVÉS DA CONSULTA A ARTIGOS E PATENTES**

Projeto de Final de Curso apresentado ao Curso de Graduação em Engenharia de Computação do Instituto Militar de Engenharia, como requisito parcial para a obtenção do título de Bacharel em Engenharia de Computação.

Orientadora: Maria Cláudia Reis Cavalcanti,
D.Sc.

Coorientadores externos:
Cel QEM José Adalberto França Junior – AGI-
TEC
Maj Giselle de Farias Rosa – AGITEC

**RIO DE JANEIRO
2026**

© 2026

INSTITUTO MILITAR DE ENGENHARIA

Praça General Tibúrcio, 80 – Praia Vermelha

Rio de Janeiro – RJ CEP: 22290-270

Este exemplar é de propriedade do Instituto Militar de Engenharia, que poderá incluí-lo em base de dados, armazenar em computador, microfilmar ou adotar qualquer forma de arquivamento.

É permitida a menção, reprodução parcial ou integral e a transmissão entre bibliotecas deste trabalho, sem modificação de seu texto, em qualquer meio que esteja ou venha a ser fixado, para pesquisa acadêmica, comentários e citações, desde que sem finalidade comercial e que seja feita a referência bibliográfica completa.

Os conceitos expressos neste trabalho são de responsabilidade do(s) autor(es) e do(s) orientador(es).

Bandeira Belchior, João Pedro; Siqueira de Almeida, Carlos Alexandre.

Utilização de Inteligência Artificial para Análise e Prospecção Tecnológica Através da Consulta a Artigos e Patentes / João Pedro Bandeira Belchior, Carlos Alexandre Siqueira de Almeida. – Rio de Janeiro, 2026.

67 f.

Orientadora: Maria Cláudia Reis Cavalcanti.

Coorientadores externos: José Adalberto França Junior; Giselle de Farias Rosa.

Projeto de Final de Curso (graduação) – Instituto Militar de Engenharia, Engenharia de Computação, 2026.

1. Inteligência Artificial. 2. Prospecção Tecnológica. 3. Análise de Patentes. 4. Processamento de Linguagem Natural. 5. Exército Brasileiro. 6. Recuperação Aumentada por Geração. 7. Modelos de Linguagem de Grande Escala.

I. Reis Cavalcanti, Maria Cláudia (orient.) II. França Junior, José Adalberto (coorient. externo) III. Rosa, Giselle de Farias (coorient. externa) IV. Título

AGRADECIMENTOS

Agradecemos ao Instituto Militar de Engenharia (IME) pela formação técnica, científica e humana de excelência, que tornou possível conceber e desenvolver este trabalho.

Agradecemos especialmente à nossa orientadora, Prof.^a Maria Cláudia Reis Cavalcanti, D.Sc., pela confiança depositada na equipe, pelas discussões técnicas, pelas orientações metodológicas e pelo rigor acadêmico com que conduziu o projeto do início ao fim.

À Agência de Gestão e Inovação Tecnológica (AGITEC), pela demanda real que deu origem a este trabalho. Agradecemos em particular aos nossos coorientadores externos, Cel QEM José Adalberto França Junior e Maj Giselle de Farias Rosa, pela coordenação do projeto junto à Agência, pela disponibilidade em compartilhar sua experiência com a elaboração de Relatórios de Prospecção Tecnológica e pelas contribuições que aproximaram a solução das necessidades reais do Exército Brasileiro.

Aos nossos familiares, pela paciência, pelo apoio incondicional e pela compreensão nos inúmeros fins de semana dedicados a este trabalho.

Aos colegas de turma, com quem compartilhamos discussões técnicas, código, café e amizade.

Por fim, à comunidade de software livre e de código aberto, em especial aos mantenedores do Ollama, do ChromaDB, do PostgreSQL, do MinIO e do FastAPI, e às equipes responsáveis pelos modelos Qwen e Gemma. A generosidade deles tornou viável construir uma solução executada integralmente em infraestrutura local.

RESUMO

Este trabalho apresenta o projeto e a implementação de um sistema baseado em Inteligência Artificial para automatizar a prospecção tecnológica no âmbito do Exército Brasileiro, desenvolvido em atendimento a uma demanda real da Agência de Gestão e Inovação Tecnológica (AGITEC). A partir de um tema fornecido pelo usuário em uma interface *web* guiada, o sistema conduz um *pipeline* de cinco fases — refinamento do tema, busca exploratória, extração de termos, busca final e geração do relatório — sobre as APIs *Open Patent Services* (OPS), do Escritório Europeu de Patentes, e Scopus, da Elsevier. Modelos de Linguagem de Grande Escala (LLMs), remotos ou executados localmente pela plataforma Ollama e configuráveis por ponto de uso, são direcionados por técnicas de engenharia de *prompt* para propor variações do tema e construir consultas booleanas na sintaxe de cada base. Os termos candidatos são extraídos dos resultados exploratórios por um *pipeline* local de Processamento de Linguagem Natural (PatternRank, BM25F, KeyBERT, fusão por RRF e C-value), sem uso de LLM nessa etapa, e cada decisão relevante é submetida à validação humana. Para evitar consultas excessivamente complexas, propõe-se uma função heurística de complexidade de consulta, empregada em um ciclo de autocorreção por métrica determinística que reinjeta no *prompt* os valores medidos na tentativa anterior. Os documentos da busca final são consolidados em um formato unificado e persistidos em PostgreSQL; a partir deles, geram-se automaticamente os indicadores bibliométricos previstos no padrão REPTEC e as curvas S de patentes e artigos, com a classificação do estágio do ciclo de vida da tecnologia pelos pontos GP/MP/SP, complementados por estimação de completude amostral (Chao1) e de estabilidade de *ranking* (*bootstrap*). O relatório é redigido seção a seção por um LLM com Geração Aumentada por Recuperação (RAG) sobre os documentos indexados no ChromaDB, enquanto números, figuras, seções de texto fixo, citações e bibliografia são produzidos de forma determinística, restringindo o modelo à redação do texto corrido e mitigando alucinações. O resultado é um documento LaTeX no padrão REPTEC, editável e compilável em PDF na própria interface, com os artefatos armazenados no MinIO. O sistema, estruturado em arquitetura hexagonal, foi validado de ponta a ponta com temas reais e consultas efetivamente submetidas às APIs, tanto com modelos de menor porte executados localmente quanto com o modelo hospedado no servidor interno do IME, reduzindo um levantamento que manualmente consome dias de trabalho a um fluxo único, no qual o esforço humano se concentra nos pontos de decisão.

Palavras-chave: inteligência artificial; prospecção tecnológica; análise de patentes; modelos de linguagem de grande escala; geração aumentada por recuperação; curva S; Exército Brasileiro.

Sumário

1	Introdução	3
1.1	Motivação	3
1.2	Caracterização do Problema	4
1.3	Objetivo	5
1.4	Metodologia	6
1.5	Estrutura do Texto	7
2	Fundamentação Teórica	8
2.1	Prospecção Tecnológica	8
2.2	Curva S de Maturidade Tecnológica	11
2.2.1	Distinção entre fase de maturidade (curva S) e <i>Technology Readiness Level</i> (TRL)	13
2.2.2	Formalização matemática e pontos de leitura (GP, MP, SP)	13
2.2.3	Diagnóstico de confiabilidade do ajuste	14
2.2.4	Estimação de completude amostral e estabilidade de <i>ranking</i>	15
2.3	<i>Large Language Models</i> (LLMs)	16
2.3.1	Capacidades e Limitações	19
2.4	Engenharia de <i>Prompt</i>	19
2.4.1	Validação e correção de saída estruturada	20
2.5	<i>Retrieval-Augmented Generation</i> (RAG)	21
2.6	<i>Embeddings</i> e Bancos de Dados Vetoriais	23
2.7	Filtragem de Documentos por Idioma e por Relevância ao Tema	24
2.8	APIs Utilizadas	25
2.8.1	OPS — <i>Open Patent Services</i>	25
2.8.2	Scopus — Elsevier	26
2.9	Ferramentas de Infraestrutura Local	26
2.9.1	Ollama	26
2.9.2	ChromaDB	27
2.9.3	MinIO	28
2.9.4	TeX Live e LanguageTool	28
3	Solução Proposta	28
3.1	Escopo	28
3.2	Especificação de Requisitos	31
3.2.1	Requisitos Funcionais	31
3.2.2	Requisitos Não Funcionais	33
3.3	Especificação da Solução	35
3.4	Configurações e Parâmetros Adotados	41
3.4.1	Cadastro e seleção dos modelos de linguagem	41
3.5	Métrica Proposta: Função de Complexidade de Consulta	43
3.5.1	Autocorreção por métrica determinística	43
3.5.2	Definição da métrica	43
3.5.3	Exemplo de aplicação	44
3.6	Tecnologias Adotadas	45
3.7	Modelagem do Banco de Dados	46

3.8	Estado Atual de Desenvolvimento	49
4	<i>Pipeline</i> de Processamento	52
4.1	Fase 1: Refinamento do Tema	52
4.2	Fase 2: Busca Exploratória	52
4.3	Fase 3: Extração de Termos	53
4.4	Fase 4: Busca Final	57
4.5	Fase 5: Geração do Relatório	57
4.5.1	Consolidação e gráficos	57
4.5.2	Redação das seções	58
4.5.3	Montagem do documento	59
4.5.4	Edição e revisão	60
4.5.5	Exportação	60
5	Conclusão	60
5.1	Síntese das Contribuições	60
5.2	Atendimento aos Objetivos	61
5.3	Limitações	62
5.4	Trabalhos Futuros	63
5.5	Considerações Finais	63
	Referências	64
	Apêndice A PROMPTS DE INTELIGÊNCIA ARTIFICIAL NO PIPELINE DE PROSPECÇÃO	67
A.1	Organização dos <i>prompts</i>	67
A.2	Sessão de exemplo	68
A.3	Fase 1: Refinamento do tema	68
A.4	Fase 2: Busca exploratória	69
A.5	Fase 3: Extração de termos	71
A.6	Fase 4: Busca final	71
A.7	Fase 5: Geração do relatório (exemplo completo)	73

1 Introdução

1.1 Motivação

No contexto do desenvolvimento tecnológico, profissionais e técnicos envolvidos em projetos de pesquisa e inovação frequentemente se deparam com um desafio recorrente: a necessidade de identificar, avaliar e validar tecnologias relevantes para suas áreas de atuação. Esse processo demanda um esforço considerável de busca em bases de dados de artigos científicos e registros de patentes [1], exigindo não apenas a localização das tecnologias pertinentes, mas também a verificação de sua atualidade — isto é, se a produção científica sobre o tema permanece ativa e se a tecnologia já atingiu maturidade [2].

A realização manual de um levantamento de prospecção tecnológica envolve, em geral, múltiplas etapas de trabalho: a definição de termos de busca adequados ao domínio investigado, a consulta a diferentes bases de dados com sintaxes e interfaces distintas, a triagem de centenas ou milhares de resultados, a verificação das instituições e pesquisadores mais relevantes no campo, a análise das tendências de publicação ao longo do tempo e, por fim, a consolidação dessas informações em um documento estruturado que possa subsidiar a tomada de decisão. Cada uma dessas etapas, realizada de forma isolada e manual, consome horas ou dias de trabalho de profissionais qualificados [27].

Esse trabalho consome tempo e recursos que poderiam ser direcionados às etapas mais criativas e estratégicas do projeto. A ausência de ferramentas integradas que automatizem esse fluxo de prospecção tecnológica representa um gargalo significativo no ciclo de desenvolvimento de soluções inovadoras. Adicionalmente, a qualidade do resultado depende fortemente da experiência do pesquisador, da abrangência das bases consultadas e da consistência metodológica adotada ao longo do processo — fatores que introduzem variabilidade e risco de viés nos levantamentos produzidos [24].

Diante desse cenário, surge a motivação para o desenvolvimento de uma ferramenta baseada em Inteligência Artificial capaz de integrar-se a *Application Programming Interfaces* (APIs) de bases de patentes e artigos científicos, realizando buscas direcionadas por meio de parâmetros-chave fornecidos pelo usuário. O sistema propõe um ciclo iterativo de validação, no qual o usuário refina progressivamente os critérios de busca, garantindo que os resultados obtidos sejam relevantes e alinhados às necessidades do projeto em questão. Esse ciclo iterativo, como será detalhado no Capítulo 2, é sustentado em boa parte por técnicas de engenharia de *prompt* — a forma como o sistema instrui os modelos de linguagem em cada etapa é, na prática, um dos pilares estruturais da solução, e não um detalhe de implementação secundário, razão pela qual lhe é dedicada uma seção própria da fundamentação teórica.

Ao final desse processo, a ferramenta consolida as informações coletadas em um relatório estruturado e editável, contendo os dados das patentes e artigos mais relevantes sobre o tema, bem como as curvas S que ilustram a evolução e o estágio de maturidade das tecnologias identificadas. Com isso, o usuário é capaz de, a partir da leitura de um único documento, inferir informações estratégicas essenciais — como o grau de consolidação de uma tecnologia, as tendências de pesquisa mais recentes e as lacunas ainda existentes na literatura — subsidiando de forma eficiente e embasada a tomada de decisão para a incorporação dessas tecnologias em seus projetos.

1.2 Caracterização do Problema

No âmbito do Exército Brasileiro, técnicos e engenheiros envolvidos em projetos de Pesquisa, Desenvolvimento e Inovação (PD&I) dedicam um tempo significativo à busca manual de tecnologias em bases de artigos científicos e registros de patentes relativas aos temas abordados nesses projetos. Além do esforço de localização, é necessário verificar se essas tecnologias ainda estão em desenvolvimento ativo — tarefa igualmente trabalhosa e sujeita a inconsistências quando realizada sem apoio de ferramentas adequadas.

O problema central não se resume à simples busca de documentos. A dificuldade reside em delimitar e consolidar, de forma coerente, informações provenientes de fontes heterogêneas — as diferentes APIs de patentes e artigos científicos possuem estruturas, campos, metadados e síntese de busca distintas — e transformar esses dados brutos em indicadores estratégicos úteis: quais instituições lideram o desenvolvimento de uma tecnologia, em que fase de maturidade ela se encontra, quais são as tendências recentes de publicação e quais lacunas ainda permanecem em aberto na literatura. Produzir esse panorama de forma confiável exige, além de esforço de pesquisa, critério metodológico rigoroso e capacidade de síntese analítica [27].

Agravando esse cenário, as informações coletadas ao longo do processo permanecem dispersas, sem um documento consolidado que permita ao responsável pelo projeto extrair conclusões de forma rápida e confiável, e construir gráficos baseados nas fontes de pesquisa que estão sendo utilizadas. Essas lacunas comprometem a qualidade das decisões e aumentam o risco de adoção de tecnologias obsoletas ou redundantes, impactando diretamente a eficiência dos projetos de defesa.

Especificamente, as organizações que realizam pesquisa de tecnologias emergentes enfrentam os seguintes desafios:

- A demanda em elaborar uma estratégia de busca coerente e sistemática, capaz de recuperar resultados mais fiéis, relevantes e alinhados aos objetivos da prospecção tecnológica.
- A necessidade de consolidar dados provenientes de múltiplas fontes heterogêneas (patentes e artigos científicos), cujas estruturas e campos diferem entre si, exigindo mapeamento e normalização antes de qualquer análise.
- A análise manual desses dados, que consome tempo e recursos humanos qualificados, sujeita a inconsistências e variações decorrentes da subjetividade do avaliador.
- A ausência de uma ferramenta integrada que combine, em um único fluxo automatizado, busca, análise e geração de relatórios.
- As limitações inerentes ao uso de APIs externas de busca científica e tecnológica: algumas exigem assinatura institucional ou chave de acesso, impõem limites ao número de resultados retornados por consulta, restringem os campos disponíveis para filtragem ou limitam a complexidade das consultas booleanas aceitas. Essas restrições precisam ser contornadas por estratégias de paginação, decomposição de consultas e tratamento de erros.
- O uso de Modelos de Linguagem de Grande Escala (LLMs) remotos pode oferecer qualidade textual elevada, mas implica custos recorrentes por *token*, limites de

contexto e dependência de infraestrutura externa [37], o que é problemático em contextos que exigem sigilo e controle de dados sensíveis. Já os modelos locais reduzem esses riscos, mas impõem requisitos computacionais que podem ser incompatíveis com máquinas comuns.

- A geração automática de relatórios por LLMs está sujeita ao fenômeno de alucinação [13], no qual o modelo produz informações plausíveis, porém factualmente incorretas. A adoção de técnicas como *Retrieval-Augmented Generation* (RAG) e a manutenção de etapas de validação humana são essenciais para mitigar esse risco.

No caso específico do Exército Brasileiro, soma-se a esses desafios a exigência de que os relatórios de prospecção sigam o padrão REPTEC (Relatório de Prospecção Tecnológica), definido pela AGITEC (Agência de Gestão e Inovação Tecnológica) [3], com estrutura e conteúdo mínimo próprios, o que implica trabalho adicional de formatação e organização.

Existe, portanto, um compromisso permanente entre qualidade, custo, privacidade e desempenho que precisa ser gerenciado ao longo de todo o projeto. Modelos remotos tendem a ser mais precisos e rápidos, porém dependem de limites externos e apresentam custo recorrente [5]. Modelos locais garantem controle e privacidade, mas podem ser mais lentos e menos precisos, especialmente em tarefas que exigem geração de texto técnico em português formal.

1.3 Objetivo

Objetivo Geral

O objetivo geral do presente projeto é o desenvolvimento de um sistema inteligente baseado em Inteligência Artificial para automatizar o processo de prospecção tecnológica, por meio da consulta integrada a bases de artigos científicos e patentes, da análise dos resultados recuperados e da geração automática de um relatório técnico consolidado no padrão estabelecido pela AGITEC [3].

Objetivos Específicos

Para a consecução do objetivo geral, estabeleceram-se os seguintes objetivos específicos:

- Promover a integração com modelos de linguagem selecionados pelo usuário — remotos, como as famílias Claude, da Anthropic, e Gemini, do Google, ou locais, executados por meio da plataforma Ollama —, configuráveis individualmente para cada ponto de uso do sistema, de modo a viabilizar a geração assistida de estratégias de busca mais coerentes, estruturadas e aderentes ao tema investigado, a partir dos parâmetros fornecidos pelo usuário.
- Integrar o sistema às APIs *Open Patent Services* (OPS) do Escritório Europeu de Patentes (EPO) e Scopus da Elsevier, viabilizando a recuperação automatizada de patentes e artigos científicos a partir de parâmetros fornecidos pelo usuário.

- iii. Implementar módulos de construção de consultas (*query builders*) capazes de transformar termos de busca em consultas estruturadas, compatíveis com a sintaxe de cada API, incluindo o uso de operadores booleanos, filtros temporais e campos específicos.
- iv. Desenvolver um mecanismo de geração e refinamento de termos e consultas de busca por meio de LLMs, combinado a um pipeline de processamento de linguagem natural local para extração e pontuação de termos candidatos a partir dos resultados exploratórios, de modo a ampliar a cobertura temática e garantir a qualidade das consultas submetidas às APIs externas.
- v. Implementar um ciclo iterativo de validação humana que permita ao usuário aprovar, ajustar ou rejeitar os termos e consultas gerados pelo modelo antes da execução das buscas definitivas.
- vi. Consolidar os metadados heterogêneos retornados pelas diferentes APIs em um formato interno unificado, realizando agregações por ano, por depositante ou instituição, e por classificação tecnológica ou área de estudo, utilizando técnicas de inferência estatística, a fim de subsidiar a análise de tendências.
- vii. Calcular automaticamente a fase de maturidade tecnológica de Ernst da área pesquisada com base na taxa de crescimento das publicações e patentes recuperadas, gerando a representação gráfica da curva S correspondente.
- viii. Gerar automaticamente um relatório técnico consolidado no padrão REPTEC da AGITEC, editável, com embasamento factual garantido pelo uso de técnicas de Geração Aumentada por Recuperação (RAG), a partir dos documentos indexados no banco vetorial.
- ix. Persistir de forma estruturada e rastreável os dados gerados ao longo do processo de prospecção, distribuindo-os entre diferentes sistemas de armazenamento conforme sua natureza: um banco de dados relacional (PostgreSQL) para os metadados estruturados de sessões, consultas, documentos de patente e artigo recuperados, e histórico de execução; um banco de dados vetorial (ChromaDB) para os *embeddings* utilizados na recuperação semântica via RAG; e um serviço remoto de armazenamento de objetos (MinIO) para os artefatos gerados, como os gráficos da curva S e o relatório final.
- x. Desenvolver uma interface *web* para acompanhamento e auditoria do processo de prospecção, para edição do relatório gerado e exportação do documento final, e verificação de dados estatísticos acumulados de pesquisas realizadas ao longo do tempo, como custos, tempo de processamento etc.

1.4 Metodologia

O sistema proposto implementa um fluxo iterativo de prospecção tecnológica baseado em Inteligência Artificial, estruturado em torno de duas frentes paralelas e independentes: a busca em bases de patentes e a busca em bases de artigos científicos. Ambas seguem o mesmo ciclo de execução.

O processo se inicia com o fornecimento de parâmetros pelo usuário por meio de uma interface *web* dedicada: a etapa de *intake* é a primeira etapa do mesmo *wizard* que conduz o usuário por todo o fluxo de prospecção. A partir dessas entradas, o modelo de linguagem remoto escolhido, direcionado por técnicas de engenharia de *prompt*, gera um conjunto de variações temáticas relacionadas ao tema informado, que são apresentadas ao usuário para validação [36]. Nessa etapa, o usuário pode aprovar o tema, solicitar maior especificidade ou alterá-lo diretamente antes de prosseguir.

Com o tema validado, o sistema realiza uma busca exploratória inicial nas APIs selecionadas para patentes e artigos, utilizando a linguagem de consulta CQL (*Common Query Language*) no caso do OPS, e a sintaxe própria de campos no caso do Scopus, recuperando um conjunto preliminar de resultados. Os títulos e resumos desses resultados são então processados por um pipeline local de Processamento de Linguagem Natural (PLN) — e não por uma nova chamada ao modelo de linguagem remoto — que extrai e pontua os termos mais relevantes, apresentados ao usuário para seleção. Com os termos selecionados, o modelo de linguagem remoto constrói, a partir deles, opções de *queries* refinadas para a busca definitiva, novamente submetidas à validação do usuário, que deve selecionar uma opção ou retornar aos valores anteriores, podendo solicitar ajustes.

Após a aprovação, o sistema executa a busca final e recupera o conjunto definitivo de patentes e artigos. Os dados coletados alimentam, por um lado, o cálculo automático das curvas S da tecnologia pesquisada (Seção 2.2) e, por outro, a indexação, em um banco de dados vetorial (ChromaDB), do título e do resumo dos documentos recuperados na busca final, por meio da técnica de RAG (*Retrieval-Augmented Generation*) [11, 16], que permite a um modelo de linguagem local gerar conteúdo factualmente embasado nos documentos recuperados, minimizando a ocorrência de alucinações [13]. A aderência ao padrão REPTEC é garantida por um *template* LaTeX e por seções de texto fixo extraídas de um relatório real [4]. Com esses dados, é gerado automaticamente um relatório consolidado em LaTeX, contendo os gráficos da análise, as referências efetivamente citadas e o quadro com as consultas de busca utilizadas, que o usuário pode editar, revisar e compilar em PDF diretamente na interface *web*.

1.5 Estrutura do Texto

O presente trabalho está organizado em mais quatro capítulos. O **Capítulo 2** aborda a fundamentação teórica necessária à compreensão do sistema desenvolvido, tratando de prospecção tecnológica, curva S de maturidade tecnológica (e sua distinção em relação ao conceito de *Technology Readiness Level*), *Large Language Models* (LLMs) e as redes neurais em que se baseiam, engenharia de *prompt* — técnica que perpassa praticamente todas as etapas do *pipeline* e que, por isso, recebe tratamento aprofundado —, *Retrieval-Augmented Generation* (RAG), *embeddings*, bancos de dados vetoriais e as principais APIs e ferramentas utilizadas. O **Capítulo 3** apresenta a solução proposta: escopo, requisitos funcionais e não funcionais, especificação da arquitetura (incluindo a arquitetura hexagonal e os princípios SOLID que a sustentam), configurações e parâmetros adotados, a métrica de complexidade de consulta proposta pelos autores, tecnologias empregadas e estado final de desenvolvimento. O **Capítulo 4** detalha o *pipeline* de processamento em suas cinco fases. O **Capítulo 5** traz a conclusão, com a síntese das contribuições, o atendimento aos objetivos, as limitações, os trabalhos futuros e as considerações finais.

2 Fundamentação Teórica

2.1 Prospecção Tecnológica

A prospecção tecnológica é um conjunto de métodos e ferramentas utilizado para identificar, analisar e monitorar tecnologias emergentes e suas tendências de desenvolvimento [27]. Seu objetivo é fornecer subsídios para a tomada de decisão estratégica em organizações de pesquisa, desenvolvimento e inovação, permitindo antecipar oportunidades e riscos associados à adoção de novas tecnologias. Segundo Mayerhoff [24], o propósito dos estudos de prospecção não é desvendar o futuro, mas sim delinear e testar visões possíveis e desejáveis para que sejam feitas, hoje, escolhas que contribuirão da forma mais positiva possível na construção do futuro.

No contexto brasileiro de defesa, essa atividade é formalizada por meio do padrão REPTEC, definido pela AGITEC, que estabelece estrutura, conteúdo mínimo e formato para os relatórios de prospecção tecnológica utilizados pelo Exército Brasileiro [3]. Um exemplo concreto e público dessa formalização é o Relatório de Prospecção Tecnológica 001/2023, elaborado pela AGITEC sobre a tecnologia *Terrestrial Trunked Radio* (TETRA) [4], que segue a estrutura em oito seções (Finalidade, Referências, Objetivo, Introdução, Metodologia, Resultados, Conclusão e Referências Bibliográficas) e emprega, nas subseções de Resultados, exatamente o tipo de indicador bibliométrico — distribuição temporal de publicações e patentes, *ranking* de instituições e depositantes, classificações CPC mais frequentes e curvas de extrapolação — que o sistema aqui descrito busca automatizar. Tais relatórios devem contemplar, entre outros elementos, análise de patentes, levantamento de artigos científicos, análise de tendências e representação gráfica do ciclo de vida das tecnologias investigadas. O cumprimento desse padrão exige que os dados coletados sejam organizados em seções específicas, como finalidade, referências institucionais, metodologia de busca, informações científicas e tecnológicas, tendências e ciclo de vida, e conclusão embasada nos dados apresentados.

A utilização de artigos científicos e patentes como fontes primárias de informação tecnológica é amplamente reconhecida na literatura [27]. Artigos científicos refletem o estado do conhecimento acadêmico e são indicadores do avanço teórico e experimental em um campo, geralmente associados aos estágios iniciais do ciclo de vida de uma tecnologia, quando a pesquisa básica e aplicada ainda predomina sobre a exploração comercial. Patentes, por sua vez, revelam tendências de inovação industrial e comercial, identificando as organizações mais ativas no desenvolvimento tecnológico, os inventores de destaque e as classificações tecnológicas predominantes, e tendem a acompanhar estágios de maturidade mais avançados, em que o interesse comercial pela proteção de propriedade intelectual se intensifica [4]. A análise combinada dessas duas fontes oferece, portanto, uma visão mais abrangente e multidimensional do panorama tecnológico de um tema — é exatamente essa complementaridade entre “perfil científico” e “perfil tecnológico” que justifica, no sistema desenvolvido, a manutenção de duas frentes de busca paralelas e independentes (OPS para patentes, Scopus para artigos), em vez de uma fonte única.

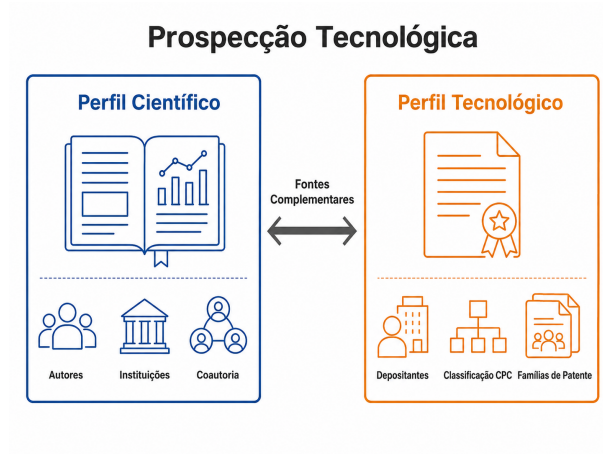
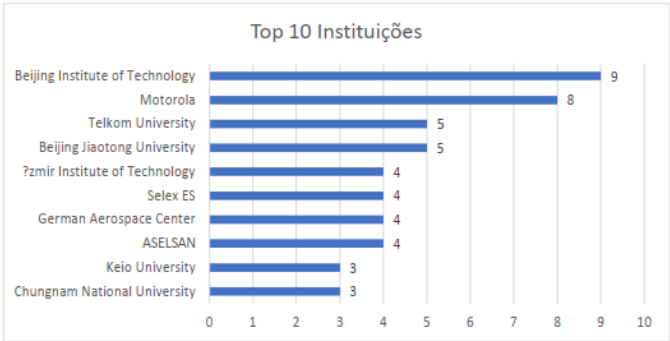


Figura 1: Perfil científico e perfil tecnológico como fontes complementares de prospecção.

da Motorola e do *Beijing Institute of Technology*, na quantidade de artigos, com 8 e 9 publicações, respectivamente, indica seu papel significativo na geração de conhecimento nesse setor.

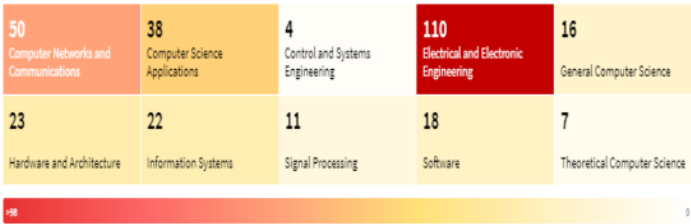
Figura 5: Top 10 maiores instituições em quantidade de publicações científicas.



Fonte: O autor.

O Quadro 2 mostra os assuntos que foram abordados com maior frequência, dentre os quais, consecutivamente: Redes de computadores e comunicações (50 publicações), aplicações em ciência da computação (38 publicações), arquitetura a hardware (23 publicações), sistemas de informação (22 publicações) e software (18 publicações).

Quadro 2: Assuntos relacionados às pesquisas científicas.



Fonte: O autor, com auxílio gráfico da base Lens.org.

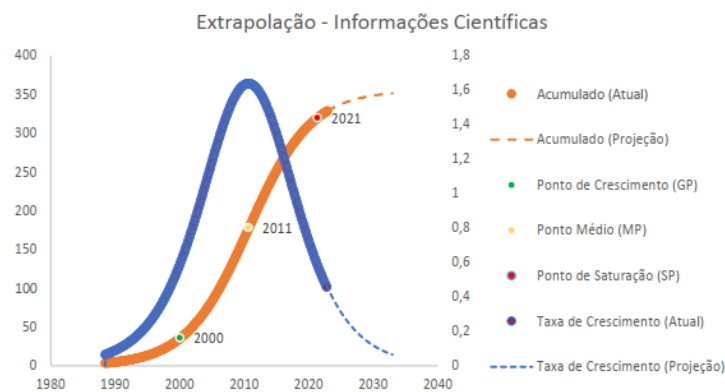
No Brasil há apenas duas publicações indexadas na base de dados Lens.org cujas instituições responsáveis pela publicação são a Agência Nacional de Telecomunicações, Universidade Federal do Paraná, Instituto Federal do Paraná, Universidade Federal do Tocantins e a Universidade de Brasília.

Figura 2: Exemplo de resultados da análise de dados em um relatório de prospecção: gráficos. Fonte: REPTEC 001/2023 — *Terrestrial Trunked Radio* (TETRA) [4].

quantidade de artigos quanto de patentes, indicando maior interesse e adoção da tecnologia, bem como a consolidação das pesquisas e desenvolvimentos iniciais.

Observou-se que os anos 2000 foram marcados pelo estágio de crescimento da tecnologia, cujos pontos de crescimento foram alcançados em 2000 (publicações científicas) e 2006 (depósitos de patentes), respectivamente nas curvas de artigos e patentes relacionados à tecnologia TETRA. Atribui-se esse crescimento à crescente aceitação e implantação prática dessa tecnologia, refletindo um ciclo de expansão em seu desenvolvimento. A figura 9 ilustra a curva-s relativa às informações científicas.

Figura 8: Curva-S relativa às informações científicas



Fonte: O autor.

A partir de meados da década de 2010, a quantidade de artigos relacionados ao TETRA apresentou uma estabilização em seu crescimento, sugerindo que a tecnologia atingiu o estágio de maturidade em termos de pesquisa acadêmica e desenvolvimento teórico, como pode ser visto na Figura 9 (MP = 2011). Esse fenômeno indica que a maioria das pesquisas fundamentais e importantes contribuições científicas já foram desenvolvidas, e novos avanços tentem a ser incrementais.

Ressalta-se que a queda consistente na quantidade anual de publicações científicas pós 2014 foi suficiente para que o modelo indique estágio de saturação em relação ao interesse acadêmico na tecnologia (SP=2021).

Figura 3: Exemplo de resultados da análise de dados em um relatório de prospecção: curva S. Fonte: REPTEC 001/2023 — *Terrestrial Trunked Radio (TETRA)* [4].

2.2 Curva S de Maturidade Tecnológica

A curva S é um modelo gráfico que descreve o ciclo de vida de uma tecnologia ao longo do tempo, relacionando o acúmulo de patentes ou publicações científicas com o grau de maturidade tecnológica [10]. Segundo Foster, toda tecnologia possui limites naturais de desempenho, e à medida em que esses limites são aproximados, o esforço necessário para obter novos avanços cresce exponencialmente — ao passo que uma nova tecnologia concorrente pode emergir a partir de um novo patamar de desempenho, iniciando sua

própria curva S [10]. Esse fenômeno de descontinuidade tecnológica é fundamental para a compreensão da dinâmica de substituição entre tecnologias e para a orientação estratégica de investimentos em PD&I.

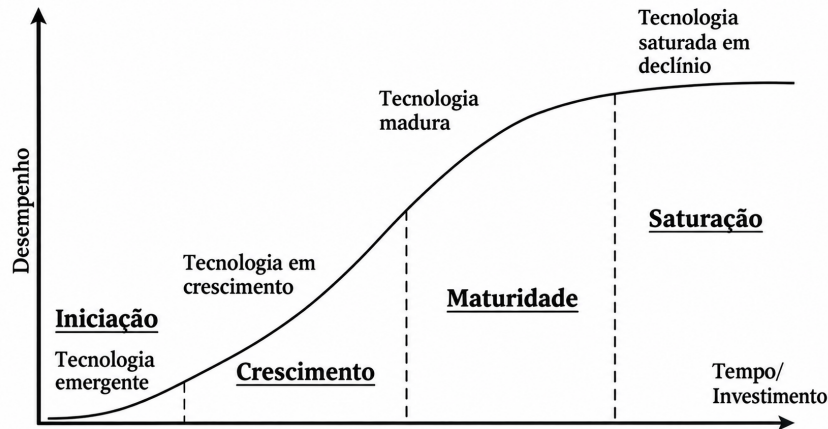


Figura 4: Curva S de maturidade tecnológica e seus estágios. Fonte: adaptado de Madeu (2019) [38].

O fundamento matemático mais difundido para a curva S aplicada à substituição tecnológica é o modelo de Fisher e Pry [9], que descreve a fração de substituição de uma tecnologia antiga por uma nova como uma função logística do tempo. Ernst [14] aplicou esse raciocínio à contagem *acumulada* de depósitos de patentes ao longo do tempo, propondo que o ciclo de vida de uma tecnologia, e não apenas a substituição de uma tecnologia por outra, pode ser descrito por quatro estágios sucessivos:

- (i) **Emergente ou Iniciação:** caracterizada por baixo volume de resultados e crescimento inicial lento, típica de tecnologias recentemente introduzidas que ainda não atraíram atenção ampla da comunidade científica ou industrial.
- (ii) **Crescimento:** marcada por expansão acelerada da produção científica e tecnológica, indicando que a tecnologia está sendo ativamente pesquisada e desenvolvida por um número crescente de atores.
- (iii) **Maturidade:** correspondente à estabilização do crescimento, quando a tecnologia alcança um patamar de desenvolvimento consolidado e a taxa de novas publicações ou patentes se estabiliza.
- (iv) **Declínio ou Saturação:** associada à obsolescência ou à substituição da tecnologia por soluções mais avançadas, refletindo uma redução no interesse e na produção científica e tecnológica [10].

Rogers [31] complementa essa perspectiva ao descrever como as inovações se difundem ao longo do tempo em uma população de adotantes, seguindo igualmente um padrão em curva S, o que reforça a robustez do modelo logístico como ferramenta de leitura de maturidade em domínios distintos (adoção de inovações, substituição tecnológica, difusão bibliométrica). Kucharavy e De Guio [15] revisam justamente essa recorrência do padrão

em curva S em campos tão diversos quanto epidemiologia, demografia e previsão tecnológica, reforçando que a escolha do modelo logístico para leitura de maturidade, adotada neste trabalho, não é uma particularidade do domínio de patentes e publicações científicas, mas uma aplicação de um padrão matemático amplamente validado.

2.2.1 Distinção entre fase de maturidade (curva S) e *Technology Readiness Level* (TRL)

É importante não confundir a fase de maturidade tecnológica descrita nesta seção com o *Technology Readiness Level* (TRL), uma escala amplamente utilizada em engenharia de sistemas e gestão de programas de defesa e espaço. O TRL foi originalmente concebido pela NASA na década de 1970 e formalizado por Mankins [17] em nove níveis, do TRL 1 (princípios básicos observados) ao TRL 9 (sistema real comprovado em ambiente operacional). Trata-se de uma métrica de prontidão de engenharia, avaliada por critérios técnicos objetivos (ou por julgamento de especialistas) para *uma solução ou protótipo específico*, tipicamente no contexto de um único projeto de desenvolvimento.

A fase de maturidade obtida pela curva S, por outro lado, não avalia um protótipo específico, mas o estágio coletivo de uma *área tecnológica como um todo*, inferido estatisticamente a partir do volume acumulado de patentes e publicações científicas ao longo do tempo em toda uma comunidade de pesquisadores e empresas. Enquanto o TRL responde à pergunta “o quão pronta está *esta* solução para ser empregada?”, a curva S responde a “em que estágio de seu ciclo de vida coletivo se encontra *este campo* tecnológico, considerando o esforço de P&D mundial documentado em patentes e artigos?”. São, portanto, instrumentos complementares, não substitutos: uma tecnologia em estágio de “crescimento” na curva S pode conter, simultaneamente, soluções específicas em TRLs bastante distintos, e o sistema desenvolvido neste trabalho não realiza e nem se propõe a realizar avaliação de TRL.

2.2.2 Formalização matemática e pontos de leitura (GP, MP, SP)

O ajuste de curva S implementado no sistema é feito sobre a contagem *cumulativa* de documentos por ano (anos sem publicação são reindexados como zero antes da soma acumulada, e o ano corrente, ainda incompleto, é descartado, já que sua contagem parcial seria lida pelo ajuste como uma queda artificial), por meio da função logística clássica:

$$N(t) = \frac{K}{1 + e^{-r \cdot (t-t_0)}}$$

onde $N(t)$ é a quantidade acumulada de documentos até o ano t ; K é a capacidade máxima ou platô de saturação (assíntota superior, correspondente ao total estimado de documentos quando a tecnologia amadurecer completamente); r é a taxa de crescimento, associada à inclinação da subida da curva; e t_0 é o ponto de inflexão — o ano em que a taxa de publicação é máxima e $N(t) = K/2$. O ajuste desses três parâmetros aos dados observados é realizado por regressão não linear, com estimativa inicial de K como 1,5 vez o valor acumulado mais recente, e a taxa de crescimento instantânea é obtida por diferenciação numérica sobre a curva ajustada.

A partir de K , r e t_0 já ajustados, o sistema inverte algebricamente a função logística para obter três pontos de leitura do ciclo de vida — Ponto de Crescimento (GP),

Ponto Médio (MP) e Ponto de Saturação (SP) — que correspondem, exatamente, aos marcos empregados na prática de prospecção tecnológica da AGITEC, como demonstrado com dados reais de patentes e publicações sobre a tecnologia TETRA em [4], onde as curvas de extrapolação científica e tecnológica trazem GP, MP e SP anotados diretamente sobre o gráfico. A inversão é dada por:

$$t = t_0 - \frac{\ln\left(\frac{1}{f} - 1\right)}{r}, \quad f = \frac{N(t)}{K}$$

A Tabela 1 resume a definição de cada ponto e a fração de K usada como referência padrão, ambas configuráveis por parâmetro:

Tabela 1: Pontos de leitura do ciclo de vida tecnológico via curva S

Ponto	Definição	Fração de K (padrão)
GP	Ano em que a curva acumulada atinge a fração <code>growth_threshold</code> de K ; antes disso, a tecnologia está em fase embrionária	0,10
MP	O próprio t_0 ; ano de inflexão, com taxa de publicação anual máxima	0,50 (sempre igual a t_0)
SP	Ano em que a curva acumulada atinge a fração <code>saturation_threshold</code> de K ; a partir daí, fase de maturidade	0,90

Quando o ano calculado para GP, MP ou SP é posterior ao último ano efetivamente observado nos dados, ele constitui uma **projeção**, isto é, uma extrapolação do ajuste logístico, e não um valor observado, distinção que o sistema mantém explícita na resposta gerada, seguindo a mesma convenção visual adotada em [4] (linha contínua para o acumulado real, linha tracejada para a projeção).

2.2.3 Diagnóstico de confiabilidade do ajuste

Um ajuste de curva logística pode convergir numericamente e, ainda assim, ser pouco confiável, como por exemplo, quando poucos anos de dado levam a um platô “inventado” por sobreajuste (*overfitting*). Por isso, o sistema também calcula um coeficiente de determinação R^2 sobre os resíduos entre a curva ajustada e a série acumulada real,

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}},$$

e marca o ajuste como não confiável quando: (i) $R^2 < 0,90$; (ii) a saturação atual já ultrapassa 85% de K com menos de cinco anos de dado observado (sinal típico de sobreajuste); (iii) a taxa de crescimento r é nula ou negativa (sem sentido físico para uma curva de acumulação); ou (iv) $r > 5,0$, o que produziria uma curva próxima de um degrau, implausível para fenômenos de difusão tecnológica. Esse diagnóstico é o que garante que o GP/MP/SP apresentado ao usuário venha acompanhado de uma indicação clara de quão bem-fundamentada é a leitura, em vez de apresentar um número sem contexto de sua incerteza estatística.

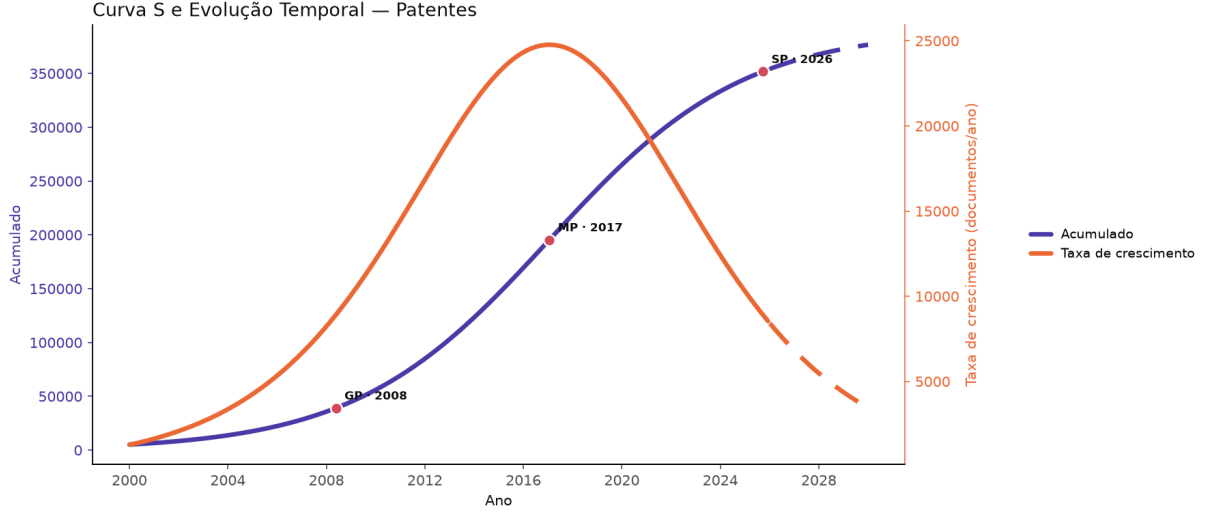


Figura 5: Curva S sintética anotada com os parâmetros K , t_0 e os pontos de leitura GP, MP e SP.

2.2.4 Estimação de completude amostral e estabilidade de *ranking*

Um limite inerente a qualquer indicador construído sobre uma amostra de patentes ou artigos, incluindo os próprios GP/MP/SP, é que ele reflete o que foi *recuperado* pela busca, não necessariamente o universo completo do que existe. Duas técnicas estatísticas complementares, implementadas no sistema como um módulo de inferência estatística sobre a busca final, endereçam esse limite sob ângulos distintos.

O **estimador Chao1** [6] é uma técnica originária da ecologia de populações, empregada para estimar a riqueza total de espécies em uma comunidade a partir de uma amostra incompleta, e que pode ser adaptada ao contexto bibliométrico tratando cada depositante, instituição ou classificação tecnológica distinta como uma “espécie” observada. A formulação clássica é

$$\hat{S}_{Chao1} = S_{obs} + \frac{f_1^2}{2f_2},$$

onde S_{obs} é o número de entidades distintas efetivamente observadas na amostra, f_1 é o número de entidades observadas exatamente uma vez (*singletons*) e f_2 é o número de entidades observadas exatamente duas vezes (*doubletons*). Quanto mais próxima a razão $S_{obs}/\hat{S}_{Chao1}$ estiver de 1, mais “saturada” está a amostra, ou seja, mais próxima

de já ter capturado a diversidade real de entidades existentes no universo pesquisado, e menor a chance de que um aumento no volume de busca revele muitas entidades novas e relevantes ainda não observadas. Como a formulação clássica não é definida quando $f_2 = 0$, o sistema emprega a variante com correção de viés [6],

$$\hat{S}_{Chao1} = S_{obs} + \frac{f_1(f_1 - 1)}{2(f_2 + 1)},$$

e considera a amostra **insuficiente** quando qualquer um de três sinais é observado: saturação $S_{obs}/\hat{S}_{Chao1} < 0,5$; mais de 70% das entidades observadas são *singletons* ($f_1/S_{obs} > 0,7$); ou menos de cinco *doubletons* ($f_2 < 5$), situação em que a própria estimativa se torna instável.

O **método de *bootstrap*** [7] complementa essa leitura verificando a **estabilidade** de um indicador derivado da amostra, como o *ranking* dos dez principais depositantes ou dos dez principais CPCs. A técnica consiste em reamostrar os documentos observados, com reposição, um grande número de vezes, recalculando o indicador de interesse (por exemplo, a composição do *top-10*) a cada reamostragem; a variância da composição e da ordem do *top-10* entre as reamostragens é então usada como medida de confiança — um *ranking* que muda substancialmente entre reamostragens é sinal de que a amostra atual ainda é pequena demais para sustentar aquele agregado específico com confiança. No sistema, a reamostragem é feita 1.000 vezes, de forma vetorizada, sobre a distribuição observada de contagens (reamostragem multinomial com o mesmo tamanho total da amostra original), e a estabilidade de cada entidade do *top-10* original é medida como a fração das reamostragens em que ela permanece entre as dez primeiras.

O sistema combina as duas técnicas em um laço iterativo de enriquecimento: a busca final é repetida, solicitando novas páginas de resultados às APIs, até que a amostra deixe de ser insuficiente segundo o critério de Chao1 ou até que se esgote um tempo máximo configurado; ao final, o resultado agregado é resumido em *rankings top-10* acompanhados da estabilidade de cada item e da relevância semântica de cada documento ao tema pesquisado. A aplicação conjunta dessas duas técnicas é especialmente relevante para as agregações de classificação tecnológica (CPC) e área de estudo — que, por terem cardinalidade tipicamente baixa (poucas categorias distintas possíveis), tendem a atingir saturação (Chao1) e estabilidade de *ranking* (*bootstrap*) com volumes de amostra relativamente pequenos — e para as agregações de depositante/instituição, cuja cardinalidade alta e cauda longa tornam esperável a necessidade de volumes de busca maiores antes que o *ranking* apresentado possa ser considerado estatisticamente estável. O objetivo funcional desse módulo é sinalizar automaticamente ao usuário quando a amostra recuperada ainda é pequena demais para que o indicador correspondente seja interpretado com confiança, complementando, para métricas de *ranking* e diversidade, o mesmo papel que o diagnóstico de R^2 (Seção anterior) já cumpre para o ajuste da curva S.

2.3 Large Language Models (LLMs)

A modelagem de linguagem é uma das principais abordagens para o avanço da inteligência linguística das máquinas. Antes de discutir os modelos de grande escala propriamente ditos, cabe situar brevemente o modelo computacional que os sustenta: as redes neurais artificiais.

Uma rede neural é um modelo composto por unidades de processamento simples (neurônios artificiais) organizadas em camadas, em que cada unidade calcula uma combinação linear ponderada de suas entradas seguida de uma função de ativação não linear; ao empilhar várias dessas camadas e ajustar os pesos por retropropagação do erro (*backpropagation*) sobre grandes volumes de dados, a rede aprende a aproximar funções extremamente complexas, incluindo a distribuição estatística de sequências de palavras em um idioma [26]. É esse mecanismo de ajuste de pesos por exposição a dados, e não regras gramaticais programadas manualmente, que permite aos modelos de linguagem generalizar para textos nunca vistos durante o treinamento.

Em geral, busca-se modelar a probabilidade generativa de sequências de palavras, de modo a prever a ocorrência de *tokens* futuros dado um contexto [37]. Os *Large Language Models* (LLMs) são modelos de linguagem de grande escala baseados na arquitetura *Transformer* [34], capazes de compreender e gerar texto em linguagem natural com alta coerência e relevância contextual. Seu treinamento em extensos *corpora* de texto permite sua aplicação em tarefas diversas, como geração de resumos, extração de informações, construção de *queries* de busca e redação de relatórios técnicos.

A arquitetura *Transformer*, introduzida por Vaswani et al. [34], revolucionou o campo do Processamento de Linguagem Natural ao substituir as redes neurais recorrentes por mecanismos de atenção (*self-attention*), que permitem ao modelo processar todas as posições de uma sequência simultaneamente, em vez de sequencialmente, capturando dependências de longo alcance com maior eficiência. Em termos simplificados, o mecanismo de *self-attention* calcula, para cada palavra da sequência, um conjunto de pesos que indicam o quanto as demais palavras do mesmo texto devem influenciar sua representação. Por exemplo, ao processar a palavra “ela” em uma frase, o mecanismo aprende a atribuir maior peso à palavra que “ela” retoma anteriormente no texto. Esses pesos são calculados a partir de três projeções aprendidas de cada *token* de entrada (usualmente denominadas *query*, *key* e *value*), e o resultado é uma representação de cada palavra que já incorpora o contexto de toda a sentença. Esse mecanismo é fundamental para a compreensão do contexto linguístico e semântico, uma vez que cada posição da sequência passa a carregar informações sobre as demais posições.

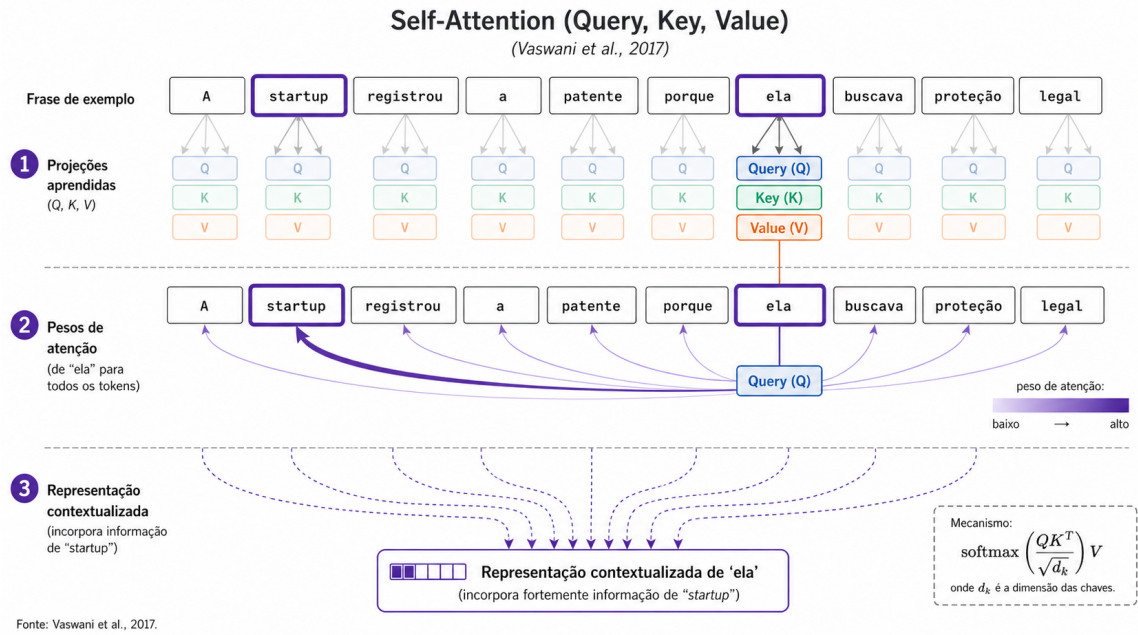


Figura 6: Mecanismo de *self-attention* (*query/key/value*) da arquitetura *Transformer*.

Diversas famílias de LLMs de propósito geral estão disponíveis atualmente, cada uma com diferentes compromissos entre custo, desempenho, tamanho e possibilidade de execução local — entre elas, GPT (OpenAI), Claude (Anthropic), Gemini (Google), Qwen (Alibaba) e Gemma (Google), estas duas últimas disponibilizadas em variantes de menor porte adequadas à execução local. No sistema desenvolvido, o modelo utilizado não é uma decisão fixa de arquitetura, mas um parâmetro de configuração substituível, consistente com a arquitetura hexagonal descrita no Capítulo 3: cada **ponto de uso** de LLM — geração das variações de tema, geração das consultas exploratórias, geração da consulta final, redação do relatório e revisão linguística do relatório — é associado, em tempo de execução, a uma configuração de provedor e modelo armazenada no banco de dados e editável pela interface *web*. As etapas de refinamento de tema e de construção de consultas podem utilizar modelos remotos das famílias **Claude** (Anthropic) e **Gemini** (Google), ou modelos servidos localmente; a redação e a revisão do relatório final via RAG utilizam um modelo acessado por meio de uma API compatível com o padrão OpenAI.

Essa última categoria merece destaque, porque não se trata de um provedor específico, e sim de um *protocolo*: qualquer serviço que implemente o formato de *chat completions* da OpenAI pode ser cadastrado pelo próprio usuário, informando apenas o endereço do serviço, o identificador do modelo e, quando exigida, uma credencial de acesso. O sistema não distingue onde esse serviço está hospedado — um contêiner na própria máquina do usuário, um servidor da rede interna de uma instituição ou qualquer outro destino alcançável para o qual se tenha autorização são, do ponto de vista do código, o mesmo caso. Nos testes deste trabalho, essa capacidade foi exercitada tanto com modelos de menor porte executados localmente pela plataforma Ollama (**Qwen 2.5 (3B-Instruct)** e **Gemma 3 (4B)**) quanto com o modelo de linguagem hospedado no servidor interno do Instituto Militar de Engenharia, alcançado por rede privada virtual — sem nenhuma alteração de código entre os dois casos, apenas cadastro distinto.

2.3.1 Capacidades e Limitações

O aumento da escala dos LLMs tem se mostrado um fator determinante para a ampliação de suas capacidades. Brown et al. [5] demonstraram que modelos de grande escala são capazes de realizar tarefas a partir de poucos exemplos fornecidos no próprio *prompt* (*few-shot learning*), sem necessidade de re-treinamento específico para cada tarefa. Entretanto, os LLMs apresentam limitações documentadas que precisam ser consideradas em projetos de aplicação prática:

Alucinações: os LLMs podem gerar conteúdo que parece plausível, mas é factualmente incorreto [13]. Esse fenômeno ocorre porque o modelo produz texto com base em padrões estatísticos aprendidos durante o treinamento, e não por raciocínio sobre fatos verificáveis. Ji et al. [13] apresentam uma revisão sistemática sobre a ocorrência e mitigação de alucinações em sistemas de geração de linguagem natural, destacando que esse é um dos principais desafios para a adoção de LLMs em aplicações críticas. Em tarefas onde a precisão factual é essencial, como na geração de relatórios técnicos de prospecção, as alucinações representam um risco significativo que deve ser mitigado por mecanismos complementares, como o RAG e a validação humana.

Limitações de contexto: cada LLM possui um limite máximo de *tokens* que podem ser processados em uma única chamada [37]. Isso impõe restrições ao volume de documentos que podem ser incluídos no contexto de uma requisição, exigindo estratégias de segmentação e seleção dos trechos mais relevantes.

Custo e dependência de infraestrutura: LLMs remotos disponibilizados via API apresentam custo recorrente por *token* consumido [5]. Em aplicações com alto volume de requisições ou grandes quantidades de texto, esses custos podem se tornar proibitivos. Além disso, o uso de modelos externos implica dependência de conectividade e de disponibilidade do serviço, além de possíveis restrições ao processamento de dados sensíveis.

Modelos locais: a execução local de LLMs, por meio de plataformas como o Ollama, elimina os custos recorrentes e garante a privacidade dos dados, porém exige capacidade computacional adequada [37]. Modelos locais de menor porte, como o Qwen 2.5 (3B-Instruct) e o Gemma 3 (4B), testados neste projeto, apresentam desempenho aceitável em máquinas comuns, mas com tempo de inferência superior ao de modelos remotos e com possível redução na qualidade da geração de texto, especialmente em tarefas que exigem raciocínio complexo ou geração de português formal de alta qualidade. Na prática do projeto, a limitação mais recorrente observada nesses modelos foi a concordância verbal com sujeito distante (por exemplo, “a concentração de depositantes ... indicam”), o que motivou a etapa de revisão linguística assistida descrita na Seção 4.5.

2.4 Engenharia de *Prompt*

A engenharia de *prompt* (*prompt engineering*) refere-se ao conjunto de técnicas e estratégias utilizadas para formular as instruções enviadas a um LLM de modo a maximizar a qualidade e a adequação das respostas geradas [36]. Dado que o comportamento de um LLM é altamente sensível à formulação das instruções recebidas, a definição cuidadosa dos *prompts* representa uma etapa crítica no desenvolvimento de sistemas baseados em IA generativa e, no sistema aqui descrito, essa etapa não se restringe a um único ponto do fluxo: praticamente toda interação com um LLM remoto (refinamento de tema, ge-

ração de consultas exploratórias, geração das variantes de consulta final) é mediada por um *prompt* de sistema cuidadosamente estruturado, o que faz da engenharia de *prompt* um componente estrutural do projeto, e não um detalhe de implementação. Os *prompts* utilizados em cada fase, e a forma como os dados de uma sessão são inseridos neles, estão reproduzidos no Apêndice A.

White et al. [36] descrevem um catálogo de padrões de *prompt engineering* aplicáveis a diferentes contextos de uso de LLMs, incluindo padrões para definição de papel (*persona pattern*), para estruturação da saída esperada e para restrição do comportamento do modelo. Dentre as técnicas mais relevantes para o presente projeto destacam-se: a definição explícita do papel do modelo por meio de um *system prompt*, que orienta o LLM sobre o contexto em que deve operar e as restrições que deve observar; o encadeamento de raciocínio (*chain-of-thought*), que induz o modelo a explicitar etapas intermediárias antes de produzir a resposta final, melhorando o desempenho em tarefas que exigem raciocínio estruturado [35]; e o uso de exemplos contrastantes (*few-shot learning*), em que pares de exemplo bom/ruim são fornecidos no próprio *prompt* para forçar diversidade e especificidade reais nas respostas geradas, em vez de paráfrases superficiais.

Wei et al. [35] demonstraram empiricamente que a técnica de *chain-of-thought prompting* — na qual alguns exemplos de raciocínio passo a passo são fornecidos no *prompt* — melhora significativamente o desempenho de LLMs de grande escala em tarefas de raciocínio aritmético, de senso comum e simbólico, especialmente em modelos com mais de 100 bilhões de parâmetros. No sistema desenvolvido, essa técnica é aplicada de forma particularmente disciplinada nas instruções de geração de consultas: o *prompt* de sistema explicita uma sequência numerada de passos de raciocínio (identificação de conceitos semânticos, seleção de conceitos centrais versus secundários, montagem de grupos de termos) que o modelo deve seguir antes de produzir a consulta final.

Um limite prático da engenharia de *prompt* isolada é que instruções textuais nem sempre são suficientes para restringir o comportamento do modelo de forma confiável. Por isso, o sistema desenvolvido complementa as técnicas acima com mecanismos de verificação programática externos ao próprio LLM: uma métrica de complexidade de consulta concebida pelos autores, acompanhada de um ciclo de autocorreção, apresentada como parte da solução proposta (Seção 3.5), e uma defesa em camadas para validação da saída estruturada do modelo, descrita a seguir.

2.4.1 Validação e correção de saída estruturada

A função de complexidade de consulta (Seção 3.5) resolve o problema de uma consulta *sintaticamente válida*, mas *estruturalmente arriscada* para as APIs externas. Um problema anterior e distinto, comum a qualquer sistema que dependa de um LLM para produzir dados estruturados (aqui, objetos JSON representando termos, campos e operadores de consulta), é o de a própria saída do modelo não ser sequer um JSON válido, ou não respeitar o formato esperado, já que, tecnicamente, um LLM gera texto, não estruturas de dados, e sua aderência a um formato solicitado depende inteiramente de quão bem o modelo segue a instrução. O sistema desenvolvido resolve esse problema com uma defesa organizada em camadas sucessivas, cada uma cobrindo os casos que a anterior deixa passar:

- (a) **Modo JSON nativo**, quando disponível: para o provedor Gemini, a chamada é

configurada para exigir explicitamente que a resposta seja um JSON sintaticamente válido, o que reduz substancialmente a ocorrência de erros de formatação já na origem. O provedor Anthropic, no momento da implementação, não expõe esse mesmo mecanismo de imposição de formato para o caso de uso do sistema, de modo que sua saída depende apenas de instrução textual explícita no *prompt* (“retorne apenas JSON válido”) e de extração posterior de blocos de código ou interpretação direta da resposta como JSON.

- (b) **Reparo sintático leve**, por expressão regular, que remove vírgulas finais indevidas e insere vírgulas ausentes entre membros de um objeto ou lista antes de uma nova tentativa de interpretação do texto como JSON — erros comuns, sobretudo em modelos locais de menor porte, e de baixo risco de correção incorreta. Se, ainda assim, a resposta não puder ser interpretada, a chamada ao LLM é repetida, em vez de a etapa falhar na primeira resposta malformada.
- (c) **Recuperação tolerante a item individual corrompido**: no fluxo de refinamento de tema, quando a resposta é uma lista de candidatos e apenas um deles está malformado, o sistema extrai individualmente cada objeto da lista (por contagem de profundidade de chaves) e descarta apenas o candidato corrompido, em vez de descartar a chamada inteira por um único item defeituoso.
- (d) **Validação de esquema**: o JSON já sintaticamente válido é então validado contra um esquema de tipos e campos esperados, rejeitando qualquer estrutura que não corresponda ao formato de saída definido pelo sistema.
- (e) **Validação semântica de conteúdo**: funções específicas verificam se cada termo retornado não é uma *stopword* ou palavra genérica isolada, se cada grupo de termos respeita a forma esperada (operador e lista de termos) e se o operador informado pertence ao conjunto permitido (AND ou OR); campos que a LLM eventualmente produza fora da lista de campos habilitados para a API em uso são removidos nesta etapa.
- (f) **Normalização final**: o objeto validado é remodelado de acordo com os campos efetivamente habilitados por configuração, zerando qualquer campo não habilitado antes de prosseguir para a construção da consulta pelos construtores da Seção 4.2.

Assim como a função de complexidade de consulta, essa sequência de camadas de defesa é uma decisão de engenharia do próprio projeto, não uma técnica padronizada por nenhuma das referências bibliográficas citadas neste capítulo, concebida especificamente para lidar com a variabilidade de aderência a formato observada entre os provedores de LLM utilizados, e constitui, junto com o RAG (Seção 2.5) e o ciclo de validação humana (Capítulo 4), a terceira linha de defesa do sistema contra o risco de alucinação e de saída inconsistente apontado por Ji et al. [13].

2.5 Retrieval-Augmented Generation (RAG)

A técnica de *Retrieval-Augmented Generation* (RAG) foi formalmente introduzida por Lewis et al. [16] como um modelo que combina memória paramétrica (o conhecimento codificado nos pesos do modelo durante o treinamento) com memória não paramétrica

(uma fonte de conhecimento externa acessada por um mecanismo de recuperação). Em vez de depender exclusivamente do conhecimento parametrizado internamente no LLM, o RAG enriquece o contexto enviado ao modelo com trechos relevantes recuperados de uma base de documentos real, reduzindo significativamente a ocorrência de alucinações [11,13].

Conceitualmente, o processo RAG pode ser descrito em duas etapas [16]: (i) dado um texto de consulta x , o sistema recupera um conjunto de documentos $z = \{z_1, \dots, z_n\}$ de uma base de conhecimento, utilizando uma função de similaridade semântica; (ii) a partir de x e dos documentos recuperados z , o modelo gera a resposta y . Gao et al. [11] propõem uma taxonomia que classifica as implementações de RAG em três categorias progressivamente mais sofisticadas:

- **RAG Naive (ingênuo)**: o padrão mais simples — indexação de documentos, recuperação dos k trechos mais similares à consulta por similaridade vetorial, e concatenação direta desses trechos como texto puro dentro do *prompt* de usuário enviado ao LLM, sem nenhum processamento intermediário sobre os trechos recuperados nem uso de *function calling/tool use*.
- **RAG Avançado**: adiciona etapas de pré-recuperação (reescrita ou expansão da consulta, roteamento entre diferentes bases) e pós-recuperação (*reranking* dos trechos recuperados, compressão do contexto) em torno do núcleo do RAG Naive.
- **RAG Modular**: reorganiza o processo em módulos independentes e recombináveis (por exemplo, módulos de busca híbrida, memória de conversas anteriores, ou múltiplas rodadas de recuperação intercaladas com geração), permitindo arquiteturas mais flexíveis e adaptadas a tarefas específicas.

O sistema desenvolvido neste trabalho adota uma variante de **RAG Naive, inteiramente local**, acrescida de dois pós-processamentos leves típicos do RAG Avançado. O corpus indexado é composto pelo título e pelo resumo de cada patente e de cada artigo recuperado na busca final da sessão, segmentados em fragmentos, convertidos em *embeddings* por um modelo executado localmente e armazenados em um banco de dados vetorial também local (ChromaDB, detalhado na Seção 2.9.2), isolados por sessão de pesquisa. No momento da geração de cada seção do relatório, os fragmentos mais similares à consulta semântica daquela seção são recuperados e concatenados como **texto puro** dentro do *prompt* de usuário enviado ao modelo de linguagem **local** — deliberadamente sem uso de *function calling/tool use*, tanto por simplicidade de integração quanto porque os modelos locais de menor porte têm suporte mais limitado e menos confiável a chamadas de ferramentas estruturadas do que os modelos remotos de maior escala.

Os dois pós-processamentos aplicados sobre os fragmentos recuperados são: (i) um **corte relativo de relevância**, que descarta todo fragmento cujo escore de similaridade seja inferior a uma fração configurável (por padrão, 0,75) do escore do fragmento mais relevante, eliminando documentos periféricos ao tema que a simples seleção dos k mais similares traria de qualquer forma; e (ii) a remoção dos escores de similaridade do texto enviado ao modelo, que, quando presentes, tendiam a ser reproduzidos literalmente no relatório. Cada fragmento carrega ainda, em seus metadados, a citação no formato autor-data e a referência bibliográfica completa no padrão ABNT do documento de origem, montadas a partir dos metadados estruturados devolvidos pelas APIs; essas são as únicas citações que o modelo está autorizado a utilizar no texto (Seção 4.5).

A concepção inicial do sistema previa uma segunda coleção vetorial, com relatórios de prospecção já elaborados pela AGITEC, indexados como referência de estrutura e estilo redacional. Essa abordagem foi substituída, na implementação, por um mecanismo determinístico: a estrutura do relatório é fixada por um *template* LaTeX que reproduz o padrão REPTEC [4], e as seções cujo texto é essencialmente o mesmo em todo relatório (Finalidade e Metodologia, por exemplo) são inseridas como texto fixo, sem passar pelo modelo. Essa escolha elimina, nessas seções, o risco de alucinação e reduz o volume de contexto de cada chamada, cabendo ao modelo apenas a redação das seções que efetivamente dependem dos dados da busca.

Essa escolha de arquitetura é deliberada e alinhada a dois objetivos que a motivam: manter toda a etapa de geração do relatório final, que é a mais sensível em termos de volume de contexto e, potencialmente, de sigilo dos dados sob análise, inteiramente dentro da infraestrutura local, sem envio de dados a serviços remotos; e produzir um relatório que siga fielmente o padrão REPTEC já estabelecido pela literatura institucional [3, 4], com cada seção fundamentada nos documentos efetivamente recuperados na busca, mitigando o risco de alucinação factual apontado por Ji et al. [13].

2.6 *Embeddings* e Bancos de Dados Vetoriais

Os *embeddings* são representações vetoriais densas de textos em um espaço de alta dimensionalidade, obtidas por modelos de linguagem treinados para capturar a semântica e as relações contextuais entre palavras, frases ou documentos [25]. O trabalho seminal de Mikolov et al. [25] demonstrou que modelos de representação distribuída de palavras são capazes de capturar um grande número de relações sintáticas e semânticas precisas, de modo que textos com significado similar são mapeados para vetores próximos no espaço vetorial. Isso permite realizar buscas por similaridade semântica, ou seja, encontrar documentos relevantes para uma consulta sem depender de correspondência exata de palavras-chave.

Reimers e Gurevych [28] introduziram o modelo Sentence-BERT (SBERT), que estende a arquitetura BERT com redes siamesas e de triplas para derivar *embeddings* de sentenças semanticamente significativos, comparáveis por similaridade de cosseno. O SBERT reduziu drasticamente o custo computacional de tarefas de similaridade semântica, tornando viável a comparação eficiente de grandes coleções de documentos, fundamento direto do subsistema de recuperação em sistemas RAG, e também da etapa de pontuação de termos descrita na Seção 4.3.

No contexto do RAG, os *embeddings* são o mecanismo que permite conectar uma consulta do usuário aos trechos mais relevantes da base de documentos indexados [11]. Para cada consulta, o sistema gera um vetor de representação semântica e compara esse vetor com os vetores dos fragmentos indexados, selecionando os mais próximos por similaridade de cosseno. O banco de dados vetorial adotado no sistema desenvolvido é o **ChromaDB**, cujos detalhes técnicos de configuração são apresentados na Seção 2.9.2, junto das demais ferramentas de infraestrutura local.

2.7 Filtragem de Documentos por Idioma e por Relevância ao Tema

Antes que os documentos recuperados por qualquer uma das APIs alimentem as etapas de extração de termos (Seção 4.3) ou de geração de relatório, dois filtros independentes atuam em nível de documento, reduzindo o conjunto de trabalho aos itens efetivamente úteis.

O **filtro de idioma** descarta documentos cujo título e resumo não sejam identificados com confiança como texto em língua inglesa, idioma predominante e mais uniformemente disponível nas bases de patentes e artigos consultadas, por meio de uma biblioteca de detecção estatística de idioma aplicada sobre o texto concatenado de título e resumo. Esse filtro é necessário porque nem todos os registros de patente possuem resumo em inglês disponível (Seção 2.8.1), e um resumo em outro idioma, se não descartado, distorceria tanto as pontuações de termo (Seção 4.3) quanto o embedding do documento usado no filtro de relevância descrito a seguir.

O **filtro de relevância documento-tema** avalia, para cada documento que passou pelo filtro de idioma, o quão semanticamente próximo ele está do tema de pesquisa informado pelo usuário. O sistema gera um *embedding* do tema e um *embedding* de cada documento com uma estratégia de composição de texto por prioridade de campo (resumo, se disponível e suficientemente longo; caso contrário, título e resumo combinados; caso contrário, apenas o título), e calcula a similaridade de cosseno entre os dois vetores:

$$\text{sim}(\vec{t}, \vec{d}) = \frac{\vec{t} \cdot \vec{d}}{\|\vec{t}\| \|\vec{d}\|} \in [-1, 1],$$

onde $\vec{t}$ é o *embedding* do tema e $\vec{d}$ é o *embedding* do documento. Na prática, como os *embeddings* de sentença aqui empregados (Seção 2.6) tendem a produzir vetores em uma região restrita do espaço, os valores observados de $\text{sim}(\vec{t}, \vec{d})$ raramente são negativos, ainda que a faixa teórica da similaridade de cosseno seja $[-1, 1]$. O documento é marcado como aprovado quando essa similaridade atinge ou ultrapassa um limiar de 0,4; documentos aprovados e reprovados são mantidos em listas ordenadas separadamente por score, de modo que o critério de corte permaneça auditável. É esse filtro que efetivamente decide quais documentos chegam à etapa de extração de termos, funcionando como um portão de qualidade a montante de todo o restante do *pipeline* de análise textual.

Complementarmente às técnicas de *embedding* descritas na Seção 2.6, o pipeline de extração de termos do sistema (Seção 4.3) combina técnicas consagradas de recuperação de informação para gerar, pontuar e ranquear os candidatos a termo-chave extraídos de título e resumo de cada documento: PatternRank, BM25F, KeyBERT, *Reciprocal Rank Fusion* (RRF) e C-value.

O **PatternRank** [33] é um método de geração de candidatos baseado em padrões gramaticais: em vez de fatiar o texto em janelas de palavras de tamanho fixo, extrai as frases nominais maximais que obedecem a um padrão de classes gramaticais — no sistema, zero ou mais adjetivos seguidos de um ou mais substantivos —, de modo que cada candidato já nasce como uma expressão gramaticalmente completa (“*perovskite solar cell*”), e não como um fragmento arbitrário (“*solar cell efficiency of*”).

O **BM25** [30] é a função de ranqueamento probabilístico que se tornou padrão em mecanismos de busca textual, sucessora direta da ponderação TF-IDF (*Term Frequency–*

Inverse Document Frequency) formalizada por Salton e Buckley [32]. Assim como o TF-IDF, favorece termos frequentes em poucos documentos do conjunto; diferentemente dele, satura a contribuição da frequência do termo (parâmetro k_1), de modo que a décima ocorrência de um termo acrescente muito menos que a primeira, e normaliza essa frequência pelo tamanho do documento (parâmetro b), evitando que documentos longos sejam favorecidos apenas por conterem mais palavras. O **BM25F** [29] estende o BM25 a documentos com múltiplos campos ponderados: título e resumo são tratados como campos distintos de um mesmo documento, cada um com seu peso, e as frequências ponderadas são combinadas *antes* da saturação — e não somando dois escores BM25 independentes, o que distorceria a saturação.

O **KeyBERT** [12], por sua vez, é uma técnica de extração de palavras-chave baseada em *embeddings* de sentença (Seção 2.6): tanto o texto de referência quanto cada candidato a termo são convertidos em vetores pelo mesmo modelo de *embedding* [28], e o escore do candidato é dado pela similaridade de cosseno entre os dois vetores — de modo que, diferentemente do BM25F, que mede apenas recorrência estatística, o KeyBERT captura proximidade semântica, inclusive entre um candidato e um sentido que não compartilhe exatamente as mesmas palavras do texto de referência.

Como BM25F e KeyBERT produzem escores em escalas incomparáveis (o primeiro ilimitado, o segundo uma similaridade de cosseno), o sistema não os soma: os dois canais são combinados pela **Reciprocal Rank Fusion** (RRF) [22], que funde *posições* em vez de escores:

$$\text{RRF}(t) = \sum_i \frac{1}{k + \text{posição}_i(t)},$$

onde $\text{posição}_i(t)$ é a posição do candidato t no *ranking* do canal i e $k = 60$ é a constante de suavização recomendada pelos autores do método. Um candidato bem posicionado em ambos os canais supera um candidato excelente em apenas um deles, sem que seja necessário calibrar pesos entre escalas diferentes. Por fim, o **C-value** [23] é uma medida de *termhood* para termos compostos, que penaliza candidatos que ocorrem quase sempre aninhados dentro de um candidato mais longo (por exemplo, “*ultrafiltration membrane*” dentro de “*composite ultrafiltration membrane*”), sinalizando redundância entre termos aninhados. A aplicação dessas técnicas, com os filtros de fronteira lexical e gramatical aplicados antes e depois da pontuação, é detalhada passo a passo, com fórmulas e um exemplo numérico, na Seção 4.3, no capítulo da solução proposta.

2.8 APIs Utilizadas

2.8.1 OPS — *Open Patent Services*

O *Open Patent Services* (OPS) é uma API REST disponibilizada pelo Escritório Europeu de Patentes (EPO) que permite acesso programático à base Espacenet, uma das maiores coleções mundiais de documentos de patentes. A busca é realizada por meio da linguagem CQL (*Common Query Language*), que permite construir consultas booleanas sobre campos específicos como título, resumo, depositantes, inventores, datas de publicação e classificações tecnológicas (CPC e IPC). O serviço suporta até 2.000 resultados por consulta e impõe limite de aproximadamente 100 requisições por segundo. Os dados

extraídos de cada patente incluem número de publicação, título, resumo, lista de depositantes e inventores, classificações CPC e IPC, ano de publicação e status legal; nas buscas de maior volume, o sistema também recupera dados já agregados por essa API, como contagem de patentes por ano, distribuição por depositante e por classificação CPC.

Cabe destacar algumas limitações práticas da API OPS: consultas excessivamente complexas, com muitos operadores booleanos ou termos encadeados, podem ser rejeitadas por exceder limites de tamanho ou complexidade aceitos pelo serviço. Além disso, nem todos os registros de patentes possuem resumos em inglês disponíveis, o que pode resultar em dados incompletos para algumas entradas. O tratamento robusto dessas situações — por meio de estratégias de decomposição de consultas e normalização de dados ausentes — é parte integrante do sistema proposto.

2.8.2 Scopus — Elsevier

A API Scopus, fornecida pela Elsevier, oferece acesso à maior base de dados de literatura científica revisada por pares do mundo, abrangendo periódicos, anais de conferências e livros. A autenticação é realizada por meio de chave de API (*header-based authentication*) e as consultas empregam a sintaxe própria do Scopus, com suporte a operadores booleanos e filtros por campo. A API suporta até 5.000 resultados por busca, com limite de 9 requisições por segundo. Os dados extraídos incluem título, lista de autores e afiliações, resumo, ano de publicação, periódico ou conferência de origem, número de citações, DOI, palavras-chave e áreas de estudo, devolvidos como lista de itens individuais — diferentemente do OPS, o Scopus não devolve estatísticas já agregadas, e a agregação por ano, autor ou área de estudo é realizada pelo próprio sistema sobre a lista recebida.

O acesso à API Scopus requer chave institucional, o que pode restringir sua disponibilidade a usuários sem vínculo com instituições que possuam contrato ativo com a Elsevier. Em cenários de uso intensivo, a imposição de limites de requisições por segundo exige a implementação de mecanismos de controle de taxa (*rate limiting*) no cliente, para evitar erros de bloqueio.

Um problema prático adicional é que nem todo registro retornado pela API Scopus possui resumo disponível — um campo essencial tanto para o filtro de relevância documento-tema (Seção 2.7) quanto para a etapa de extração de termos (Seção 4.3), ambas dependentes de texto além do título. Para mitigar essa lacuna, o sistema complementa os registros do Scopus com resumos recuperados da API pública da **OpenAlex**, consultada por identificador (DOI) do próprio registro sempre que o campo de resumo original vier vazio ou truncado, antes de o documento seguir para as etapas de filtragem e extração já mencionadas.

2.9 Ferramentas de Infraestrutura Local

2.9.1 Ollama

O Ollama é uma plataforma de código aberto que permite a execução local de modelos de linguagem de grande escala, disponibilizando uma API REST compatível com padrões amplamente utilizados — inclusive o formato de *chat completions* da OpenAI. No contexto deste projeto, o Ollama é responsável por disponibilizar os modelos de geração de

texto locais (nos testes, Qwen 2.5 (3B-instruct) e Gemma 3 (4B)), executados inteiramente na infraestrutura local, sem transmissão de dados para servidores externos. Como o sistema se comunica com o Ollama pelo mesmo formato compatível com a OpenAI adotado pelo modelo hospedado no servidor interno do Instituto Militar de Engenharia, um único adaptador atendeu aos dois ambientes ao longo do desenvolvimento, bastando trocar o endereço cadastrado. Essa abordagem garante a privacidade dos temas pesquisados e a ausência de custos recorrentes de API, ao custo de maior tempo de processamento em relação a modelos remotos de maior porte [37]. Os *embeddings* do RAG, por sua vez, não são gerados pelo Ollama, mas pelo mesmo modelo de sentença (Sentence-BERT [28]) já empregado nos filtros de relevância, o que torna a indexação e a recuperação independentes da disponibilidade do modelo de geração de texto.

2.9.2 ChromaDB

O ChromaDB é um banco de dados vetorial de código aberto, projetado para armazenamento e recuperação eficiente de *embeddings* [11]. Diferentemente de um banco de dados relacional convencional, no qual a busca é feita por igualdade ou comparação exata de valores em colunas, um banco de dados vetorial indexa cada documento (ou fragmento de documento) pelo vetor de *embedding* que o representa e responde a consultas por **busca por vizinhos mais próximos aproximada** (*Approximate Nearest Neighbor*, ANN): dado o vetor de uma consulta, o banco retorna, em tempo sublinear no tamanho da coleção, os k vetores armazenados mais próximos segundo uma métrica de distância configurável — no caso do sistema desenvolvido, a similaridade de cosseno, consistente com a métrica já empregada nos demais filtros de similaridade semântica do sistema (Seção 2.7). Internamente, o ChromaDB organiza cada coleção de vetores em uma estrutura de grafo de proximidade hierárquica (*Hierarchical Navigable Small World*, HNSW), que permite localizar os vizinhos mais próximos sem comparar o vetor de consulta contra a totalidade dos vetores indexados um a um — o que tornaria a busca proibitivamente lenta à medida que o volume de documentos indexados crescesse. No sistema desenvolvido, o ChromaDB é executado como um contêiner local, acessado por HTTP, sem depender de nenhum serviço remoto — e não como uma biblioteca embutida gravando diretamente em disco, o que não seria seguro caso o *backend* fosse executado com mais de um processo simultâneo. Os vetores de todas as sessões ficam em uma única coleção, e cada fragmento carrega, em seus metadados, o identificador da sessão de pesquisa de origem, usado como filtro em toda consulta.

No sistema, o ChromaDB é utilizado como repositório persistente dos vetores gerados pelo modelo de *embedding* de sentença, suportando buscas por similaridade de cosseno para a recuperação de contexto no subsistema RAG descrito na Seção 2.5. Sua utilidade para o RAG decorre diretamente da forma como essa técnica opera: antes de gerar cada seção do relatório, o sistema precisa localizar, entre os documentos recuperados na busca final — segmentados em múltiplos fragmentos —, apenas os poucos trechos mais relevantes ao propósito daquela seção específica, já que o volume de contexto que pode ser enviado ao modelo de linguagem local em uma única chamada é limitado. Sem um banco de dados vetorial dedicado, essa busca por similaridade semântica teria de ser recalculada por força bruta a cada seção gerada, comparando o *embedding* da consulta contra o de cada fragmento indexado individualmente; o ChromaDB elimina esse custo ao manter os vetores já indexados e prontos para consulta eficiente, e ao persistir essa indexação

entre chamadas, de modo que os documentos de uma sessão precisam ser processados e vetorizados apenas uma vez, ainda que múltiplas seções do relatório final consultem repetidamente os mesmos vetores. Adicionalmente, por ser executado inteiramente em infraestrutura local, sem comunicação com serviços externos, o ChromaDB é consistente com o objetivo arquitetural, já discutido na Seção 2.5, de manter a etapa de geração do relatório — a mais sensível em termos de volume de dados e de sigilo — inteiramente dentro do ambiente local, sem envio de conteúdo de patentes, artigos ou relatórios institucionais a serviços remotos.

São indexados o título e o resumo de todos os documentos da busca final persistidos na sessão; como os resumos de artigos nem sempre chegam preenchidos, o corpus de uma sessão tende a ser majoritariamente composto por patentes. A indexação é idempotente e versionada: uma sessão já indexada não é reprocessada a cada seção gerada, e uma sessão indexada por uma versão anterior do sistema, sem os metadados de citação, é reindexada automaticamente.

2.9.3 MinIO

O MinIO é um servidor de armazenamento de objetos de código aberto, compatível com a API S3 da Amazon, que pode ser executado em infraestrutura própria (*self-hosted*). No sistema, armazena, fora do banco relacional, todos os artefatos binários produzidos: as imagens dos gráficos, as figuras fixas da seção de Metodologia do relatório, as imagens anexadas pelo usuário, o arquivo `.tex` montado e o PDF compilado. O banco relacional guarda apenas a chave de cada objeto. O MinIO já é utilizado pelo Exército Brasileiro em outro ambiente; o sistema o executa em um contêiner local, com um caminho claro de promoção para a instância institucional.

2.9.4 TeX Live e LanguageTool

A compilação do relatório em PDF é realizada por um serviço dedicado, executado em contêiner próprio, construído sobre uma distribuição mínima do TeX Live acrescida apenas dos pacotes efetivamente utilizados pelo *template* do relatório. Todos os pacotes são instalados durante a construção da imagem, e não durante a compilação, já que o ambiente-alvo é uma intranet sem acesso à internet.

O **LanguageTool** é um verificador ortográfico e gramatical de código aberto, baseado em regras, com suporte ao português do Brasil, executado localmente em contêiner próprio. No sistema, é empregado na revisão do relatório gerado (Seção 4.5), detectando erros de ortografia, acentuação e concordância sem que o texto seja enviado a nenhum serviço externo.

3 Solução Proposta

3.1 Escopo

O sistema proposto implementa uma plataforma modular para automatização da prospecção tecnológica [24, 27], estruturada em torno de um *backend* desenvolvido em Python com o *framework* FastAPI, um banco de dados relacional para persistência das

pesquisas e resultados, um banco de dados vetorial (ChromaDB) para indexação semântica dos documentos recuperados [11], um módulo de IA baseado em LLM (remota ou local, configurável por ponto de uso) responsável pelo refinamento de termos e geração das *queries* booleanas e um módulo de IA responsável pela redação e pela revisão do relatório técnico, servido por qualquer endpoint compatível com a API da OpenAI cadastrado pelo usuário [37], além de serviços auxiliares executados em contêineres locais para armazenamento de objetos (MinIO), compilação LaTeX (TeX Live) e revisão ortográfica e gramatical (LanguageTool).

O principal objetivo é reduzir significativamente o esforço manual e o tempo necessários para a realização de um levantamento completo de prospecção tecnológica [27], automatizando as etapas de busca, análise e geração de relatório, ao mesmo tempo em que mantém o usuário no controle das decisões estratégicas por meio de ciclos de validação iterativa. O sistema não tem como propósito eliminar a participação do especialista humano, mas fornecer um ponto de partida estruturado e factualmente embasado para análises mais aprofundadas — premissa alinhada ao uso de RAG como ferramenta de suporte à geração de texto [16].

A solução é composta pelos seguintes subsistemas:

Interface *web*: é o único ponto de contato do usuário com o sistema — não há uma interface de entrada separada da interface de acompanhamento: o mesmo *wizard* conduz o usuário desde a submissão dos parâmetros iniciais da pesquisa (tema, área tecnológica, período de interesse e demais filtros) até o acompanhamento do progresso do *pipeline*, a edição do relatório gerado e a exportação do documento final.

Módulo de Orquestração (ChatService): conduz o fluxo de negócio das Fases 1 a 4 do *pipeline* (Capítulo 4) — decide quando acionar o LLM remoto para refinar o tema ou gerar variantes de consulta, quando enviar a consulta construída para busca e quando repassar os termos extraídos à rodada seguinte de geração de consulta —, dependendo apenas de contratos abstratos de LLM e de busca (*ports*), nunca de suas implementações concretas, conforme a Arquitetura Hexagonal detalhada na Seção 3.3. Implementa também o ciclo de verificação de complexidade e retentativa (Seção 3.5) e medeia cada ciclo de validação do usuário entre etapas.

Módulo de IA (LLM): adaptadores técnicos que executam as chamadas aos provedores configurados — Claude (Anthropic), Gemini (Google) ou um modelo local compatível com a API da OpenAI — enviando os *prompts* montados pelo Módulo de Orquestração e interpretando e validando a resposta estruturada retornada (Seção 2.4.1), sem decidir por si só quando ou com que conteúdo cada chamada deve ocorrer. Qual provedor e modelo atende a cada ponto de uso é resolvido em tempo de execução, a partir de configurações armazenadas no banco de dados e editáveis na tela de configurações da interface, sem reinicialização do sistema.

Módulo de busca (QueryBuilders + SearchServices): adaptadores técnicos que traduzem a estrutura de campos já decidida pelo Módulo de Orquestração para a sintaxe específica de cada API (CQL para o OPS, sintaxe própria para o Scopus) e executam as requisições HTTP correspondentes, com tratamento de erros, paginação e controle de taxa — também sem decidir quais termos ou operadores compõem a consulta.

Módulo de extração de termos (PLN local): pipeline de processamento de linguagem natural, independente de chamadas a LLM, responsável por extrair e pontuar

os termos mais relevantes dos resultados exploratórios, detalhado na Seção 4.3.

Módulo de Consolidação e Visualização (ReportService + s_curve.py): normaliza e agrega os dados retornados pelas diferentes APIs, calculando indicadores por ano, por depositante/autor, por classificação tecnológica e por área de estudo, ajusta a curva S (Seção 2.2) e calcula os pontos de leitura GP/MP/SP com seu diagnóstico de confiabilidade, gerando as representações gráficas correspondentes. A deduplicação de documentos — necessária porque o mesmo item pode ser recuperado tanto na busca exploratória quanto na busca final, ou por mais de uma sessão de pesquisa — é resolvida por uma chave de deduplicação construída, preferencialmente, a partir de identificadores primários do próprio documento (por exemplo, o número de publicação de uma patente ou o DOI de um artigo) e, na ausência destes, por uma combinação normalizada de título e ano de publicação; essa chave é o que permite que patentes e artigos sejam reaproveitados entre diferentes consultas e sessões, em vez de duplicados a cada nova busca. Cada gráfico gerado é armazenado no MinIO e registrado no banco de dados junto de um resumo numérico (total, ano de pico, principais entidades, GP/MP/SP), que é repassado ao módulo seguinte para que o texto do relatório descreva cada figura com os números reais.

Módulo de Inferência Estatística: aplica o estimador Chao1 e a verificação de estabilidade por *bootstrap* (Seção 2.2.4) sobre a busca final, enriquecendo iterativamente a amostra até sua saturação ou até o limite de tempo configurado.

Módulo de Geração Textual do Relatório (ReportWriterService + ChromaDB + LLM local): indexa o título e o resumo dos documentos recuperados na busca final, recupera os trechos mais relevantes [28] para cada seção e coordena o modelo de linguagem local na redação das seções que dependem dos dados da busca, enquanto as seções de texto fixo e os elementos estruturais do padrão REPTEC da AGITEC [3, 4] são produzidos de forma determinística. Módulos auxiliares, sem uso de LLM, posicionam as figuras no texto, calculam o estágio do ciclo de vida da tecnologia, restringem as citações às obras efetivamente recuperadas e validam o texto gerado, mitigando o risco de alucinações [13] (Seção 4.5).

Módulo de Montagem, Revisão e Exportação do Relatório: monta o arquivo `.tex` a partir de um *template* LaTeX no padrão REPTEC, disponibiliza-o para edição na interface *web*, revisa-o (verificação de LaTeX, LanguageTool e, opcionalmente, revisão por LLM), compila-o em PDF no serviço TeX Live e exporta um pacote `.zip` com o `.tex` e as imagens referenciadas.

Módulo de Persistência (SQLAlchemy + PostgreSQL + MinIO): responsável pelo armazenamento estruturado de todas as entidades do sistema — pesquisas, patentes, artigos, gráficos, seções do relatório, configurações e histórico de execução —, garantindo rastreabilidade completa do processo e permitindo a retomada ou reanálise de pesquisas anteriores sem necessidade de nova consulta às APIs externas; os artefatos binários (imagens, `.tex` e PDF) ficam no MinIO, referenciados por chave no banco relacional.

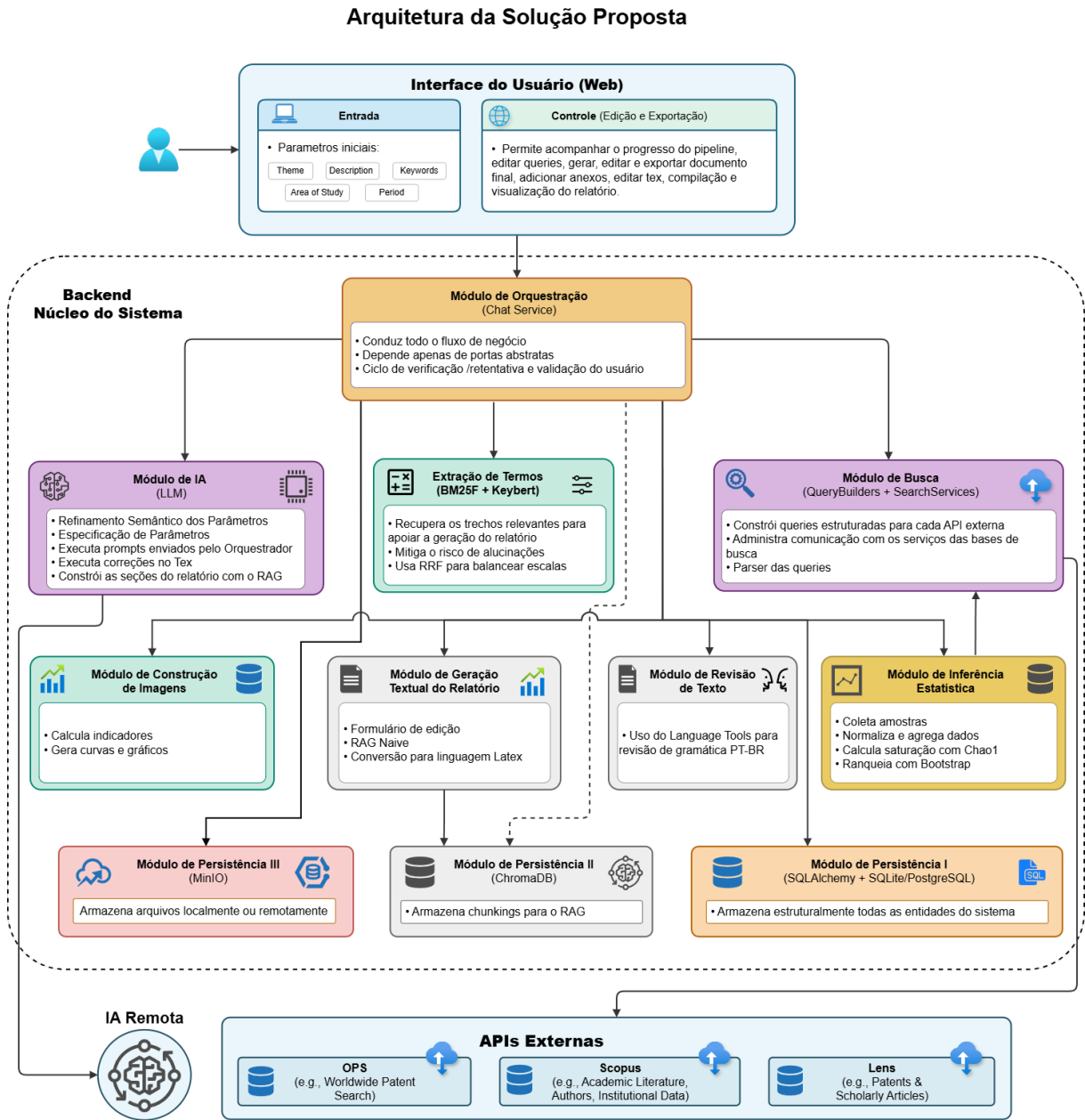


Figura 7: Arquitetura de módulos do sistema de prospecção tecnológica.

3.2 Especificação de Requisitos

3.2.1 Requisitos Funcionais

Nº	Requisito	Descrição
RF01	Entrada de parâmetros	O sistema deve permitir que o usuário informe o tema de pesquisa e parâmetros complementares (período, área tecnológica, palavras-chave e descrição).

Nº	Requisito	Descrição
RF02	Geração de termos de busca	O sistema deve gerar automaticamente, a partir do tema informado, um conjunto expandido de termos de busca relacionados, incluindo sinônimos e variações terminológicas em inglês.
RF03	Validação iterativa pelo usuário	O sistema deve apresentar os termos e consultas gerados ao usuário para validação [36], permitindo aprovação, ajuste ou rejeição antes de prosseguir para a etapa seguinte.
RF04	Consulta a bases de patentes	O sistema deve permitir a busca de patentes a partir dos termos validados, com filtros por campo (título, resumo e classificação tecnológica) e por período, retornando, para cada registro, número de publicação, título, resumo, depositantes, inventores, classificações, ano de publicação e situação legal, além das contagens agregadas por ano, por depositante e por classificação, quando disponíveis.
RF05	Consulta a bases de artigos	O sistema deve permitir a busca de artigos científicos a partir dos termos validados, com filtros por campo (título, resumo, autores, periódico) e por período, retornando, para cada registro, título, autores e afiliações, resumo, ano de publicação, periódico ou conferência de origem, número de citações, DOI, palavras-chave e área de estudo.
RF06	Consolidação de metadados	O sistema deve consolidar os dados retornados pelas diferentes APIs em um formato interno unificado, realizando deduplicação e normalização dos campos.
RF07	Geração de indicadores	O sistema deve calcular indicadores de análise tecnológica: distribuição por ano, por depositante/autor, por classificação CPC e por área de estudo, conforme dados recuperados.

Nº	Requisito	Descrição
RF08	Cálculo e geração de gráficos S	O sistema deve ajustar automaticamente uma curva logística à série acumulada de publicações e patentes recuperadas [9], calcular os pontos de leitura do ciclo de vida (GP, MP, SP) e um diagnóstico de confiabilidade do ajuste, e representar graficamente a curva S e as demais representações gráficas padronizadas pela AGITEC.
RF09	Geração automática de relatório	O sistema deve gerar automaticamente um relatório técnico consolidado no padrão REPTEC da AGITEC [3,4], com as oito seções numeradas do padrão, utilizando RAG [16] para embasamento factual das seções redigidas por LLM, com as figuras apresentadas e interpretadas no texto e com citações restritas aos documentos efetivamente recuperados.
RF10	Edição e exportação	O sistema deve disponibilizar o relatório gerado para edição pelo usuário e permitir sua exportação em LaTeX (incluindo as imagens utilizadas) e em PDF.
RF11	Rastreamento de métricas	O sistema deve registrar o consumo de <i>tokens</i> , o custo estimado por operação e o tempo de execução de cada fase do <i>pipeline</i> .
RF12	Revisão do relatório	O sistema deve revisar o relatório editado, apontando erros de sintaxe LaTeX, de ortografia, de acentuação e de concordância nos trechos gerados por IA ou editados pelo usuário, e permitir que cada sugestão seja aplicada ou ignorada individualmente.
RF13	Configuração em tempo de execução	O sistema deve permitir ao usuário cadastrar endpoints de LLM e escolher qual deles atende cada ponto de uso, bem como definir as APIs de busca ativas e os parâmetros operacionais, sem necessidade de reinicialização.

Tabela 2: Requisitos Funcionais do Projeto

3.2.2 Requisitos Não Funcionais

Nº	Requisito	Descrição
RNF01	Modularidade	A arquitetura deve ser organizada em módulos com responsabilidades bem definidas, permitindo substituição ou atualização de componentes individuais sem impacto no restante do sistema.
RNF02	Extensibilidade	O sistema deve ser projetado para permitir a integração futura de novas APIs de busca (Google Patents, arXiv, PubMed) sem necessidade de reescrita significativa — requisito diretamente sustentado pela arquitetura hexagonal descrita na Seção 3.3.
RNF03	Desempenho aceitável	O sistema deve ser capaz de executar o <i>pipeline</i> completo em tempo satisfatório (média de cinco minutos em máquinas comuns), sem dependência de GPU dedicada, salvo na etapa de geração do relatório, cujo tempo de execução é diretamente influenciado pelo hardware disponível.
RNF04	Robustez	O sistema deve tratar erros de comunicação com APIs externas de forma transparente ao usuário, realizando novas tentativas ou degradando graciosamente em caso de falha. Uma fonte de busca sem credencial configurada deve ser simplesmente omitida, mantendo o sistema operável com as demais; já a ausência de um modelo de linguagem vinculado a um ponto de uso deve produzir erro explícito, nunca texto gerado por um substituto que o usuário possa confundir com resultado real.
RNF05	Rastreabilidade	O sistema deve registrar todas as etapas do processo de prospecção, incluindo os parâmetros fornecidos, os termos gerados, as consultas construídas e as validações realizadas pelo usuário.
RNF06	Controle de custos	O sistema deve estimar e registrar o custo de cada operação que envolva chamadas ao LLM [5], permitindo ao usuário e ao gestor acompanhar o gasto total associado a cada pesquisa.

Nº	Requisito	Descrição
RNF07	Manutenibilidade	O código deve seguir convenções de estilo e documentação que facilitem sua manutenção por equipes distintas da que o desenvolveu originalmente.
RNF08	Interoperabilidade com bases externas	A busca de patentes deve ser realizada por meio da API OPS/Espacenet, com consultas na linguagem CQL, e a busca de artigos, por meio da API Scopus (com complemento de resumos pela OpenAlex), respeitando a sintaxe, os limites de requisição e as estatísticas agregadas disponibilizados por cada serviço.
RNF09	Uso de modelos de linguagem	A geração de termos e consultas de busca e a redação do relatório devem ser realizadas por LLMs [37], remotos ou locais, configuráveis por ponto de uso, de modo que a troca de provedor não exija alteração do código.

Tabela 3: Requisitos Não Funcionais do Projeto

3.3 Especificação da Solução

A arquitetura do sistema é organizada em camadas e módulos com responsabilidades bem definidas, seguindo o padrão de **Arquitetura Hexagonal** (também chamado *Ports and Adapters*), proposto originalmente por Cockburn [8]. A ideia central do padrão é isolar as regras de negócio (o “núcleo” da aplicação) de qualquer dependência concreta de infraestrutura — bibliotecas de terceiros, clientes HTTP de APIs externas, SDKs de provedores de LLM — fazendo com que o núcleo dependa apenas de contratos abstratos (*ports*), implementados por adaptadores substituíveis. A Tabela 4 resume as camadas adotadas e exemplos concretos de cada uma.

Tabela 4: Camadas da arquitetura hexagonal adotada

Camada	Responsabilidade	Exemplos
Domínio	Tipos de dados puros, sem I/O	Estruturas de resultado de busca, de requisição/resposta de LLM, de termo extraído
<i>Ports</i>	Contratos (interfaces) dos quais o núcleo depende, sem implementá-los	Porta de LLM; porta de geração de texto; porta de busca de patentes; porta de busca de artigos; porta de repositório; porta de armazenamento de objetos; porta de banco vetorial
Serviços (núcleo)	Regras de negócio puras, sem dependência de <i>framework</i>	Orquestração do fluxo de prospecção; análise de complexidade de consulta; geração de gráficos; redação, montagem e revisão do relatório; inferência estatística
Adaptadores <i>driven</i>	Implementam as portas, conectando a integrações reais	Clientes das APIs Anthropic, Gemini e de qualquer endpoint compatível com OpenAI; clientes OPS e Scopus; MinIO; ChromaDB
Adaptadores <i>driving</i>	Chamam o núcleo a partir do mundo externo	Rotas HTTP (FastAPI) que expõem cada etapa do fluxo ao <i>frontend</i>

Na prática, isso significa que o serviço central de orquestração do fluxo depende de uma porta de LLM e de portas de busca de patentes/artigos — nunca de uma biblioteca concreta como o SDK da Anthropic ou o cliente HTTP da OPS. A implementação concreta de cada porta é decidida e injetada em um único ponto de composição do sistema, no momento em que a aplicação é inicializada, mantendo o núcleo de regras de negócio livre de dependências de infraestrutura: mudanças em uma API externa — o Scopus alterando seu esquema de resposta, a OPS depreciando um *endpoint*, um novo modelo de LLM sendo lançado — ficam contidas no respectivo adaptador, sem exigir alteração do núcleo.

Vale registrar que essa composição não é feita por meio de um *framework* de injeção de dependência: trata-se de uma função única de composição, executada uma vez na inicialização da aplicação, que monta um dicionário de serviços prontos a partir de duas condições simples — a presença de uma credencial configurada e uma *feature flag* que habilita ou não aquele provedor. Cada fonte de busca (OPS para patentes, Scopus para artigos) só é instanciada e registrada nesse dicionário se estiver habilitada e possuir credencial configurada, o que faz da adição — ou remoção — de uma fonte de busca uma mudança confinada a esse único ponto de composição, nunca ao serviço de orquestração em si, que já está escrito para operar sobre uma lista de adaptadores sem saber quantos ou quais existem.

A escolha do adaptador de LLM, por outro lado, não é feita nesse ponto de composição, e sim em tempo de execução, a cada chamada: o que é injetado nos serviços é um *resolver*, que consulta no banco qual configuração de provedor atende àquele ponto de

uso e constrói o adaptador correspondente, mantendo em memória um cache por ponto de uso, invalidado sempre que uma configuração é editada na interface. É esse desenho que permite trocar o modelo de uma etapa sem reiniciar o sistema (Seção 3.4.1). Diferentemente das fontes de busca, aqui não há degradação silenciosa: se um ponto de uso não tem configuração vinculada, a chamada falha de forma explícita, em vez de produzir texto de um adaptador simulado que o usuário poderia confundir com resultado real — distinção que o requisito de robustez RNF04 incorpora.

Um ponto de atenção, registrado aqui por rigor metodológico, é que essa inversão de dependência é mais forte na camada de integrações externas do que na camada de geração de relatórios: o módulo responsável pelos gráficos é obtido, nas rotas que o expõem, a partir do mesmo ponto de composição central, mas sem uma porta formal que o abstraia — o que é aceitável, já que hoje existe apenas uma implementação de geração de relatório, mas que representa uma aplicação apenas parcial do princípio de inversão de dependência nessa camada específica, distinta da aplicação mais completa observada nas integrações de LLM e de busca.

Essa organização em camadas está diretamente fundamentada nos princípios **SO-LID** de projeto orientado a objetos, formalizados por Martin [18] e posteriormente sistematizados em torno do princípio de inversão de dependência como eixo central de arquiteturas em camadas [19]. Dos cinco princípios, o **Princípio da Responsabilidade Única** (*Single Responsibility Principle*, SRP) se manifesta na granularidade dos serviços do núcleo: o componente que orquestra o fluxo de prospecção não calcula complexidade de consulta, o componente que calcula complexidade de consulta não gera gráficos, e o componente que gera gráficos não sabe nada sobre LLMs — cada um tem um único motivo para mudar. O **Princípio Aberto-Fechado** (*Open-Closed Principle*, OCP) aparece no mecanismo de registro de construtores de consulta por API: uma nova fonte de busca é adicionada registrando um novo construtor em um ponto único de extensão, sem modificar o código dos construtores já existentes. O **Princípio da Substituição de Liskov** (*Liskov Substitution Principle*, LSP) garante que qualquer construtor de consulta, seja para o OPS ou para o Scopus, respeite o mesmo contrato de entrada e saída e possa ser utilizado pelo serviço de orquestração de forma intercambiável, sem que este precise saber qual API está por trás de cada um. O **Princípio da Segregação de Interface** (*Interface Segregation Principle*, ISP) se reflete na existência de portas estreitas e específicas — uma porta apenas para chamadas de LLM, portas distintas para busca de patentes e de artigos — em vez de uma única interface genérica que obrigasse cada adaptador a implementar métodos que não utiliza.

O princípio mais visivelmente ligado à escolha arquitetural do projeto, contudo, é o **Princípio da Inversão de Dependência** (*Dependency Inversion Principle*, DIP), segundo o qual módulos de alto nível não devem depender de módulos de baixo nível — ambos devem depender de abstrações [19]. O serviço central de orquestração depende de uma porta de LLM e de portas de busca de patentes/artigos — abstrações definidas pelo próprio domínio do sistema — e nunca de bibliotecas concretas como o SDK da Anthropic ou o cliente HTTP da OPS; a implementação concreta é decidida e injetada no ponto de composição da aplicação, mantendo o núcleo de negócio livre de dependências de infraestrutura, exatamente na direção descrita na justificativa original da arquitetura hexagonal do projeto: mudanças em APIs externas “ficam contidas nos adaptadores — o núcleo não sabe que aconteceu nada”. O módulo de geração de relatório, como já observado, segue uma versão mais modesta do mesmo princípio: é obtido a partir do

ponto de composição central em vez de instanciado diretamente pelas rotas que o utilizam, invertendo o controle de construção, mas sem uma porta/contrato formal que o abstraia — a diferença é que, ao contrário do serviço de orquestração, não há hoje necessidade real de múltiplos “provedores de relatório”, o que torna essa aplicação parcial do DIP uma escolha aceitável, e não uma inconsistência de projeto. A Tabela 5 sintetiza onde cada princípio é observado.

Tabela 5: Princípios SOLID observados na arquitetura do sistema

Princípio	Onde é observado	Exemplo
SRP	Granularidade dos serviços do núcleo	Orquestração, análise de complexidade e geração de gráficos são serviços distintos
OCP	Registro extensível de construtores de consulta	Nova API de busca = novo construtor registrado, sem alterar os existentes
LSP	Contrato único entre construtores de consulta	Qualquer construtor (OPS, Scopus) é substituível pelo serviço de orquestração
ISP	Portas estreitas e específicas	Porta de LLM separada de portas de busca de patentes/artigos
DIP	Núcleo dependendo de abstrações, não de implementações	Serviço de orquestração → porta de LLM/busca (forte); rota de relatório → classe concreta (parcial)

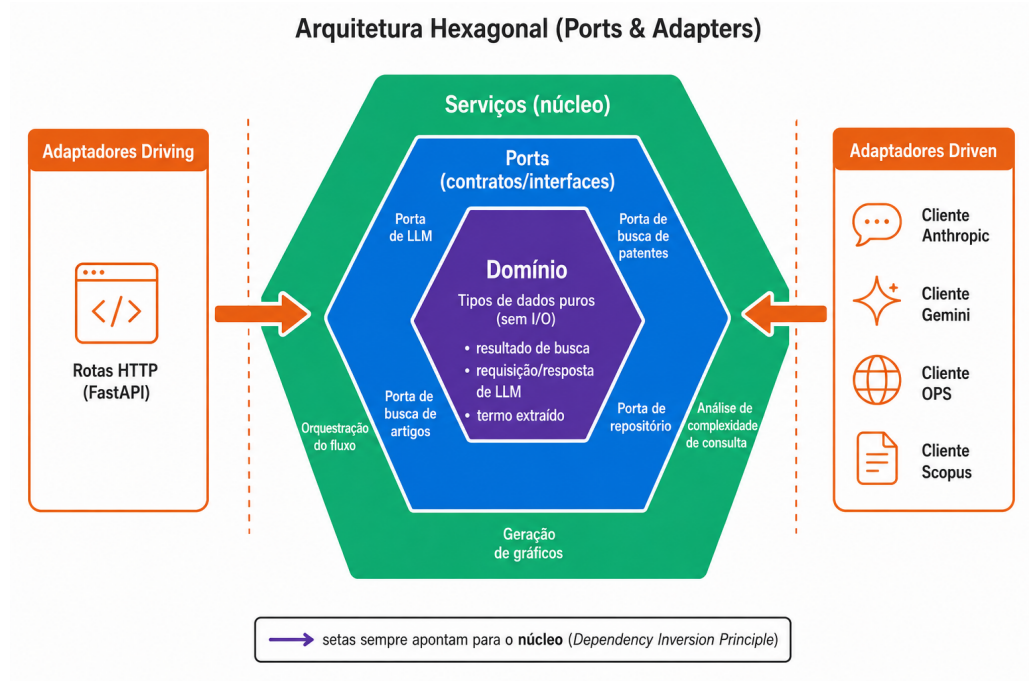


Figura 8: Camadas da arquitetura hexagonal: domínio, *ports*, serviços e adaptadores *driven/driving*.

A Figura 9 apresenta o fluxo geral de operação do sistema. O usuário inicia o processo enviando os parâmetros iniciais da pesquisa por meio da interface *web*. Caso deseje refiná-los, a aplicação aciona o módulo de IA, que gera novas variações do tema [36]; esse ciclo se repete até que o usuário esteja satisfeito com o tema gerado, cujo histórico é armazenado para rastreabilidade.

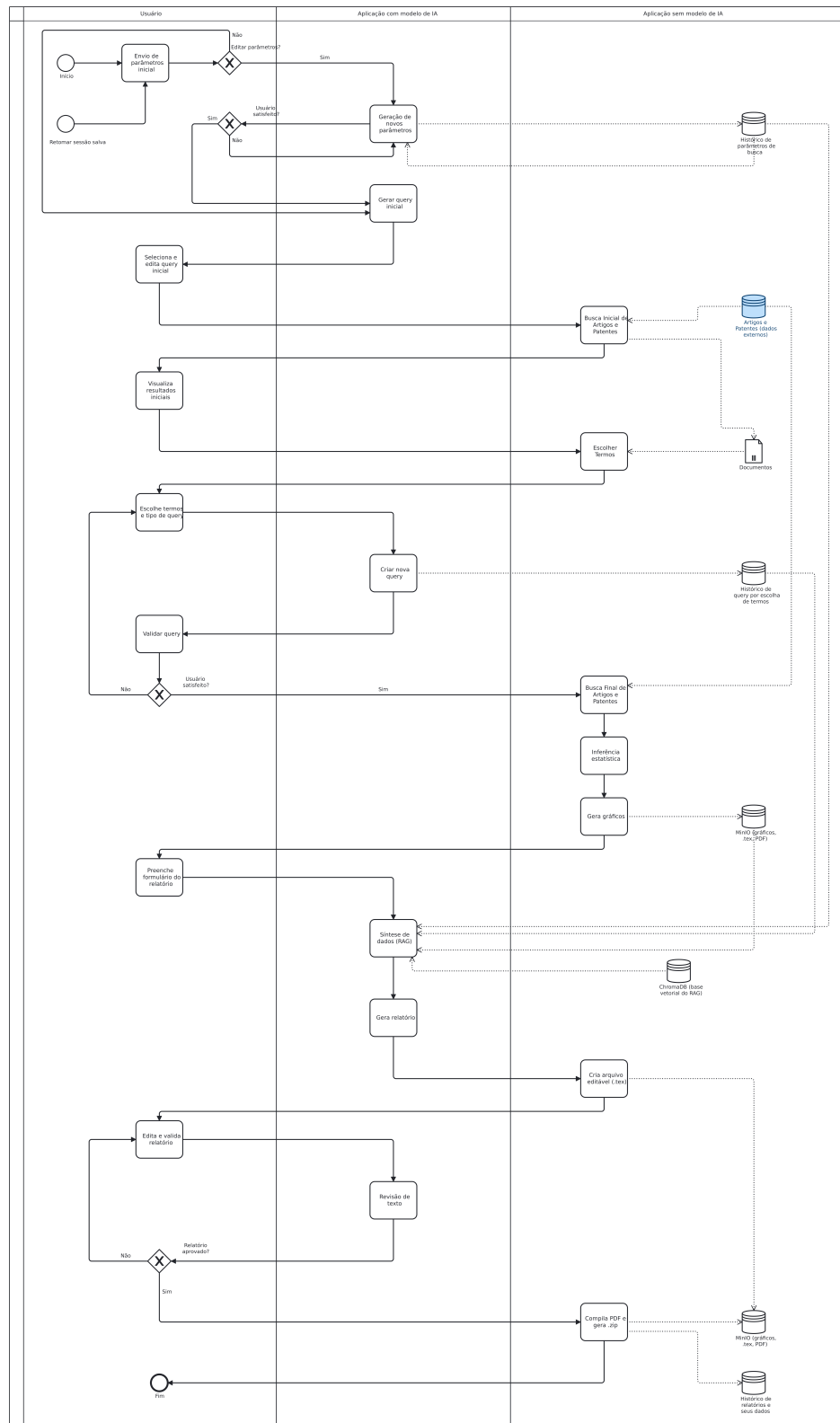


Figura 9: Fluxo geral do sistema de prospecção tecnológica.

Com os parâmetros aprovados, o sistema realiza o *fetch* nas APIs externas e executa uma busca inicial, indexando os documentos recuperados. A partir desses documentos, o pipeline local de PLN sugere um conjunto de termos relevantes ao usuário (Seção 4.3), que pode aprová-los ou editá-los — ciclo que também é registrado em histórico. Com os

termos definidos, o módulo de IA constrói uma nova *query*, que é apresentada ao usuário para validação antes da execução da busca final.

A **busca final** é a execução definitiva da pesquisa. Diferentemente da busca exploratória — limitada a 10 resultados por API e usada apenas como amostra para a sugestão de termos —, ela submete às APIs de patentes e de artigos a consulta booleana escolhida pelo usuário entre as variantes geradas pelo LLM a partir dos termos selecionados (específica, balanceada ou ampla), recupera o conjunto definitivo de documentos, remove duplicatas e registra o total real de resultados informado por cada base. É sobre esse conjunto — e não sobre a amostra exploratória — que são calculados todos os indicadores, gráficos e o próprio relatório (Seção 4.4).

Concluída a busca final, a aplicação gera automaticamente os gráficos de análise [9, 10] — restritos ao conjunto de figuras previsto no padrão REPTEC — e seus resumos numéricos. Os gráficos, juntamente com os dados consolidados e o histórico do processo, alimentam a etapa de síntese de dados, que embasa a geração automática do relatório no padrão REPTEC [3, 4, 16], seção a seção. O relatório é então disponibilizado como arquivo LaTeX editável na própria interface, onde o usuário pode inserir novas imagens, revisar o texto, compilar o PDF e exportar o resultado, cabendo a ele a edição e a validação final antes da entrega.

3.4 Configurações e Parâmetros Adotados

No sistema desenvolvido, a fase de maturidade da curva S é determinada automaticamente ajustando uma função logística à série acumulada de patentes e artigos recuperados (Seção 2.2), e não por um limiar arbitrário sobre a taxa de crescimento ano a ano. A partir dos parâmetros K , r e t_0 desse ajuste, o sistema calcula os pontos de leitura GP, MP e SP por inversão algébrica da função logística, acompanhados de um diagnóstico de confiabilidade baseado em R^2 e em critérios de plausibilidade da taxa de crescimento estimada. Esse método já possui precedente de aplicação real na prática da AGITEC, como demonstrado com dados reais de patentes e publicações sobre a tecnologia TETRA em [4]. A representação gráfica da curva S, gerada automaticamente a partir dos dados de publicação por ano, é incluída no relatório final como elemento de análise estratégica, junto do aviso de confiabilidade do ajuste quando aplicável. O estágio do ciclo de vida (emergente, crescimento, maturidade ou saturação) é determinado por código a partir da posição do último ano observado em relação a GP, MP e SP; o estágio geral da tecnologia segue a curva de patentes, e é informado ao LLM como um fato pronto, em vez de ser inferido por ele.

3.4.1 Cadastro e seleção dos modelos de linguagem

Os modelos de linguagem são configurados por **ponto de uso**, em tabelas próprias do banco de dados: cada um dos cinco pontos de uso (variações de tema, consulta exploratória, consulta final, redação do relatório e revisão do relatório) aponta para uma configuração de provedor, modelo e credencial. A tela de configurações da interface expõe essa estrutura em três blocos: o cadastro de uma nova configuração, a lista das configurações já salvas — editáveis e removíveis, com a credencial exibida de forma mascarada — e um seletor por ponto de uso, que vincula cada etapa do fluxo a uma das configurações

salvas.

Cadastrar uma configuração consiste em escolher o tipo de provedor e preencher apenas os campos que aquele tipo exige; o próprio formulário se adapta, pedindo endereço do serviço para os tipos que o requerem e credencial para os que dependem de autenticação. Essa distinção entre *tipos de provedor* e *instâncias configuradas* é deliberada: os tipos são uma decisão de código, porque suportar um protocolo de comunicação novo exige escrever um adaptador novo; já as instâncias são dado, e vivem no banco. A consequência prática é que apontar o sistema para um serviço de linguagem ainda não utilizado — desde que ele fale o formato de *chat completions* da OpenAI — não demanda alteração de código nem migração de esquema, apenas o preenchimento de um formulário.

Distribuir os pontos de uso entre modelos diferentes não é um recurso acessório: as etapas têm exigências distintas. A construção de consultas booleanas depende de aderência estrita a um formato de saída estruturada, enquanto a revisão linguística depende de domínio do português formal — e raramente o mesmo modelo é o melhor nas duas tarefas. Além disso, é esse mecanismo que permite manter a redação do relatório, etapa que concentra o conteúdo dos documentos recuperados, em um modelo executado dentro da própria rede, enquanto as etapas de busca utilizam modelos remotos.

Ao longo do desenvolvimento, os pontos de uso de redação e revisão foram exercitados com os modelos **Qwen 2.5 (3B-instruct)** e **Gemma 3 (4B)**, disponibilizados localmente pela plataforma **Ollama**, e com o modelo de linguagem hospedado no servidor interno do Instituto Militar de Engenharia, acessado por rede privada virtual — em todos os casos com geração de até 4.096 *tokens* por chamada. Parâmetros operacionais — como o número de fragmentos recuperados por seção, o corte relativo de relevância do RAG e o idioma da revisão ortográfica — ficam em uma tabela de configurações editável pela interface, com valores iniciais sementeados de forma idempotente, sem sobrescrever valores já alterados pelo usuário.

A engenharia de *prompt* (Seção 2.4) é empregada em múltiplas etapas do *pipeline*: na geração de variações do tema, na construção de consultas para as APIs externas e na geração de cada seção do relatório técnico. Os *prompts* são estruturados para instruir explicitamente o modelo a basear suas respostas exclusivamente nos dados recuperados, evitando a introdução de informações não verificadas.

O RAG local (Seção 2.5 do Capítulo 2) opera da seguinte forma: o título e o resumo de cada documento da busca final (patentes e artigos científicos) são divididos em fragmentos (*chunks*) de 1000 caracteres com sobreposição de 200 caracteres e convertidos em *embeddings* pelo mesmo modelo Sentence-BERT [28] usado nos filtros de relevância. Esses vetores são armazenados em uma coleção do ChromaDB, com o identificador da sessão, a citação e a referência ABNT do documento de origem nos metadados. Na etapa de geração de cada seção do relatório, o sistema formula uma *query* semântica a partir do tema e do propósito da seção, recupera os cinco fragmentos mais similares da sessão por similaridade de cosseno, descarta os que ficarem abaixo de 75% do escore do mais relevante e injeta o restante, como texto puro, no *prompt* enviado ao LLM local, junto da lista de citações permitidas — caracterizando a variante de RAG Naive local descrita na Seção 2.5.

3.5 Métrica Proposta: Função de Complexidade de Consulta

3.5.1 Autocorreção por métrica determinística

Como antecipado na Seção 2.4, instruções textuais no *prompt* nem sempre bastam para impedir que o LLM gere consultas booleanas excessivamente complexas. Para lidar com isso, o sistema emprega um padrão que pode ser chamado de **autocorreção por métrica determinística**, não descrito no catálogo original de White et al. [36]: a saída do LLM (uma consulta booleana) é avaliada por um componente de código determinístico — não pela própria LLM se autoavaliando — que calcula um escore objetivo de complexidade estrutural da consulta; caso esse escore ultrapasse um limiar configurável, o resultado exato da avaliação (quantidade de operadores, profundidade de aninhamento, número de termos) é reinjetado no *prompt* da tentativa seguinte, em um sufixo de simplificação que cita de volta os números da tentativa anterior. Após um número máximo de tentativas (tipicamente três), o sistema não falha: retorna a tentativa menos complexa das geradas, com um aviso explícito ao usuário. O escore utilizado nesse ciclo é a métrica proposta descrita a seguir.

3.5.2 Definição da métrica

A métrica de complexidade não é derivada de nenhuma referência bibliográfica externa: trata-se de uma **função heurística concebida pelos próprios autores deste trabalho**, motivada empiricamente pelo comportamento observado das APIs consultadas. Ao longo do desenvolvimento, constatou-se repetidas vezes que consultas booleanas mais “abrangentes”, com muitos termos e muitos operadores OR/AND aninhados, não produziam necessariamente mais resultados, mas sim erros ou zero resultados: a API do OPS retorna erro de servidor quando, por exemplo, vários OR são utilizados, em vez de retornar mais resultados, o que seria o logicamente esperado. Diante da ausência de um limite de complexidade publicado pelas próprias APIs, optou-se por desenvolver uma métrica interna que servisse de *guardrail* preventivo, calculada e verificada *antes* de qualquer consulta ser efetivamente enviada a uma API externa.

A função combina quatro sub-escores independentes, cada um normalizado no intervalo $[0, 100]$ contra uma base de referência escolhida empiricamente, e combinados por uma média ponderada:

$$\begin{aligned} \text{operator_score} &= \min \left(100, \frac{n_{\text{operadores}}}{10} \times 100 \right) \\ \text{nesting_score} &= \min \left(100, \frac{d_{\text{aninhamento}}}{5} \times 100 \right) \\ \text{term_score} &= \min \left(100, \frac{n_{\text{termos}}}{20} \times 100 \right) \\ \text{length_score} &= \min \left(100, \frac{c_{\text{caracteres}}}{1000} \times 100 \right) \end{aligned}$$

$$\text{score} = 0,2 \cdot \text{operator_score} + 0,3 \cdot \text{nesting_score} + 0,3 \cdot \text{term_score} + 0,2 \cdot \text{length_score}$$

onde $n_{\text{operadores}}$ é a contagem total de ocorrências de AND/OR/NOT na consulta (por expressão regular, insensível a maiúsculas/minúsculas); $d_{\text{aninhamento}}$ é a profundidade máxima de parênteses aninhados — com um cuidado deliberado no cálculo: um parêntese de abertura não é contado como um novo nível se o caractere não-espaço imediatamente anterior também for um parêntese de abertura, já que os construtores de consulta (Seção 4.2) embrulham mecanicamente cada cláusula em seus próprios parênteses, o que empilharia parênteses sem representar aninhamento lógico real caso não houvesse essa correção; n_{termos} é a contagem de termos entre aspas; e $c_{\text{caracteres}}$ é o comprimento bruto, em caracteres, da consulta. Os pesos (0,2; 0,3; 0,3; 0,2) e as quatro bases de normalização (10 operadores, 5 níveis de aninhamento, 20 termos, 1000 caracteres) foram calibrados empiricamente pelos autores a partir da observação de consultas que efetivamente falhavam nas APIs consultadas, e não a partir de nenhum estudo ou norma publicada. A partir do escore final, a consulta é classificada em um de quatro níveis, apresentados na Tabela 6.

Tabela 6: Níveis de classificação da função de complexidade de consulta

Nível	Faixa de escore
Simples	escore < 25
Moderado	$25 \leq \text{escore} < 50$
Complexo	$50 \leq \text{escore} < 75$
Muito Complexo	escore ≥ 75

3.5.3 Exemplo de aplicação

Como exemplo de aplicação da fórmula, considere uma consulta hipotética com $n_{\text{operadores}} = 6$, $d_{\text{aninhamento}} = 3$, $n_{\text{termos}} = 9$ e $c_{\text{caracteres}} = 420$. Os sub-escores resultantes são:

$$\text{operator_score} = \min \left(100, \frac{6}{10} \times 100 \right) = 60$$

$$\text{nesting_score} = \min \left(100, \frac{3}{5} \times 100 \right) = 60$$

$$\text{term_score} = \min \left(100, \frac{9}{20} \times 100 \right) = 45$$

$$\text{length_score} = \min \left(100, \frac{420}{1000} \times 100 \right) = 42$$

e o escore final é

$$\text{score} = 0,2(60) + 0,3(60) + 0,3(45) + 0,2(42) = 12 + 18 + 13,5 + 8,4 = 51,9,$$

classificando essa consulta hipotética como **Complexa** (Tabela 6), abaixo, portanto, do limiar padrão de 60 pontos adotado como corte de aceitação, ou seja, ainda

dentro do limite, mas próxima o suficiente dele para que, em uma consulta real com poucos termos ou operadores a mais, o sistema já acionasse o ciclo de retentativa com simplificação assistida.

Além do *escore* em si, a função também emite avisos textuais complementares, como por exemplo, mais de 15 operadores no total, aninhamento acima de 4 níveis, parênteses desbalanceados, mais de 5 termos repetidos, comprimento acima de 2000 caracteres, ou contagem de operadores **OR** mais que o dobro da contagem de **AND**; e recomendações textuais de simplificação (reduzir termos para cerca de 5 a 8, limitar aninhamento, remover duplicatas), usadas tanto na interface do usuário quanto no sufixo de simplificação reinjetado no *prompt* da LLM descrito no início desta seção.

3.6 Tecnologias Adotadas

Para a implementação do sistema, as seguintes tecnologias foram selecionadas:

O *backend* foi desenvolvido em **Python**, com o *framework* **FastAPI**, escolhido por sua alta performance em operações assíncronas e pela geração automática de documentação interativa (Swagger/OpenAPI). A persistência relacional é gerenciada pelo ORM **SQLAlchemy**, com **PostgreSQL** como banco de dados (o **SQLite** é empregado apenas nos testes automatizados). A busca vetorial e a persistência de *embeddings* são realizadas pelo **ChromaDB** [11], e os artefatos binários, pelo **MinIO**. A execução local dos modelos de linguagem é viabilizada pelo **Ollama** [37], e os *embeddings* são gerados com a biblioteca **sentence-transformers** [28]. O pipeline local de extração de termos (Seção 4.3) emprega **spaCy** e **KeyphraseVectorizers** (PatternRank) [33], **BM25F** [29], **KeyBERT** [12], RRF [22] e C-value [23]. As consultas às APIs externas são realizadas por meio de requisições HTTP com a biblioteca **httpx**, com suporte a requisições assíncronas, controle de taxa e novas tentativas em caso de falha de rede. Os gráficos são gerados com **matplotlib** e o ajuste da curva S, com **SciPy**. O relatório é renderizado a partir de um *template* **Jinja2** e compilado por um serviço **TeX Live** dedicado; a revisão ortográfica e gramatical utiliza o **LanguageTool**. Todos os serviços de infraestrutura — PostgreSQL, MinIO, ChromaDB, Ollama, TeX Live e LanguageTool — são orquestrados localmente por **Docker Compose**.

A interface *web* do sistema é desenvolvida com **React**, utilizando o **Vite** como ferramenta de *build* e ambiente de desenvolvimento, o **Zustand** para gerenciamento de estado e o **Tailwind CSS** para a construção da camada visual. Essa escolha busca proporcionar uma aplicação responsiva, modular e de fácil manutenção. Entre as funcionalidades disponíveis na interface, destacam-se o assistente guiado (*wizard*) de quatro etapas para condução de toda a pesquisa, a edição manual dos campos de consulta em cada etapa, o registro de auditoria do uso de IA por sessão, a tela de configurações e, na última etapa, o formulário do relatório e o editor LaTeX com painel de imagens, revisão, compilação em PDF e exportação, conforme detalhado na Seção 3.8.

A persistência de artefatos gerados (gráficos, figuras fixas, anexos, **.tex** e PDF) utiliza o **MinIO**, executado em um contêiner Docker local, no mesmo padrão empregado para o PostgreSQL, e configurável para apontar ou replicar para a instância remota do MinIO do Exército — permitindo que o desenvolvimento local funcione de forma independente, mas com um caminho claro de promoção para o ambiente institucional real.

3.7 Modelagem do Banco de Dados

O modelo de dados relacional do sistema é implementado com o ORM **SQLAlchemy** em sua API assíncrona declarativa, versionado por migrações **Alembic**, com **PostgreSQL** como banco de dados (Seção 3.6). O esquema ativo é organizado em torno de uma entidade central — **research_session** —, que representa uma execução completa do fluxo de prospecção (Figura 9) e da qual derivam, por chave estrangeira, as demais entidades do domínio: os parâmetros de entrada da sessão, as *queries* de busca geradas, o registro de chamadas de IA, as seções e o manifesto do relatório e, indiretamente, por meio das *queries*, os documentos de patente e artigo recuperados e os gráficos gerados. Um segundo grupo, independente das sessões, armazena as configurações editáveis em tempo de execução. A Figura 10 apresenta o diagrama entidade-relacionamento completo do esquema ativo, com as colunas de cada tabela e as chaves estrangeiras que as ligam, e a Tabela 7 resume as entidades e seu papel no domínio.

Tabela 7: Entidades do esquema relacional ativo

Entidade	Papel
<code>research_session</code>	Entidade central (<i>hub</i>); uma linha por execução do fluxo de prospecção, com a etapa corrente do assistente para retomada no ponto certo
<code>session_input</code>	Parâmetros de entrada da sessão; auto-relacionada (tema original vs. variante refinada por IA)
<code>session_probe_query</code>	<i>Queries</i> de busca por fonte (OPS, Scopus); auto-relacionada (<i>query</i> de exploração inicial vs. <i>query</i> final escolhida)
<code>session_ai_call</code>	Registro (<i>log</i>) somente-inserção de cada chamada de IA feita durante o assistente guiado
<code>patent/article</code>	Documentos recuperados, deduplicados globalmente por <code>dedup_key</code> , independentemente de sessão ou <i>query</i>
<code>probe_query_patent</code> <code>probe_query_article</code>	Entidades associativas que resolvem o relacionamento muitos-para-muitos entre <i>queries</i> e documentos
<code>probe_query_term</code>	Termos extraídos por PLN local a partir dos documentos de uma <i>query</i> de exploração
<code>session_chart</code>	Gráfico gerado para uma <i>query</i> final: chave do PNG no MinIO e resumo numérico (JSON) usado na redação do relatório
<code>session_report_section</code>	Uma seção do relatório: contexto recuperado pelo RAG, texto gerado e fontes citáveis (JSON)
<code>session_report</code>	Manifesto do relatório da sessão: chaves do <code>.tex</code> e do PDF no MinIO e dados do formulário
<code>app_settings</code>	Parâmetros operacionais editáveis em tempo de execução (limiares, pesos, quantidades), um por linha
<code>llm_provider_configs</code>	<i>Endpoints</i> de modelo de linguagem cadastrados pelo usuário: tipo de provedor, modelo, credencial e endereço
<code>llm_call_site_bindings</code>	Vínculo entre cada ponto de uso do fluxo e a configuração de LLM que o atende
<code>search_api_selection</code>	API de busca ativa por família (patentes e artigos)

Persistência do relatório. Duas decisões de modelagem merecem registro na parte do esquema dedicada ao relatório. O texto não é guardado como um documento único, e sim em `session_report_section`, com uma linha por seção: assim, uma seção cuja geração falhe não invalida as demais, e apenas ela precisa ser refeita. Já `session_report` guarda o documento montado como artefato da própria sessão — e

não como um arquivo efêmero entregue ao usuário —, registrando o estágio em que ele se encontra e os dados do formulário que o originaram, o que permite remontá-lo posteriormente sem refazer o percurso da pesquisa.

Configuração como dado. As quatro últimas entidades da Tabela 7 não pertencem a nenhuma sessão: guardam parâmetros operacionais (`app_settings`), a fonte de busca ativa de cada família (`search_api_selection`), os *endpoints* de modelo de linguagem cadastrados (`llm_provider_configs`) e o vínculo entre cada ponto de uso e um desses *endpoints* (`llm_call_site_bindings`), conforme descrito na Seção 3.4.1.

Auto-relacionamentos. Duas entidades do esquema empregam um relacionamento reflexivo (unário), no qual uma linha referencia outra linha da mesma tabela por meio de uma chave estrangeira opcional que aponta para a própria chave primária — um padrão clássico de modelagem para representar hierarquias ou histórico de revisão dentro de uma única tabela [20]. Em `session_input`, a linha raiz (`parent_id` nulo) é o tema bruto submetido pelo usuário; se o usuário aciona o refino assistido por IA (Seção 3.3), a variante escolhida para prosseguir é persistida como uma segunda linha, com `parent_id` apontando para a linha raiz. Em `session_probe_query`, o mesmo padrão distingue a *query* de exploração inicial (`tipo` nulo) da *query* final escolhida pelo usuário, gravada como uma nova linha com `parent_id` apontando para a linha de exploração da mesma fonte e `tipo` preenchido com a variante selecionada (específica, balanceada ou ampla). Esse desenho evita duplicar o esquema da tabela de *queries* para cada etapa do fluxo — uma aplicação, na camada de persistência, do mesmo espírito de extensão sem duplicação discutido a propósito do Princípio Aberto-Fechado (Seção 3.3) —, com a restrição de unicidade da tabela ajustada para considerar sessão, fonte de busca e tipo em conjunto (`UNIQUE(session_id, fonte, tipo)`), de modo que cada combinação exista no máximo uma vez. Não há, portanto, um banco ou um conjunto de tabelas literalmente separado para “exploratório” e “final”: as duas etapas compartilham a mesma tabela, distinguidas apenas por qual linha cada uma é e para qual linha ela aponta.

Relacionamentos muitos-para-muitos. Uma mesma patente ou artigo pode ser recuperado por mais de uma *query* — por exemplo, quando a *query* de exploração inicial e a *query* final da mesma fonte retornam parcialmente o mesmo conjunto de documentos — e uma mesma *query* recupera, tipicamente, dezenas de documentos distintos, caracterizando um relacionamento muitos-para-muitos entre `session_probe_query` e `patent/article`. Como o par (*query*, documento) carrega um atributo próprio — o `relevance_score` daquele documento especificamente em relação àquela *query*, que não pertence nem à *query* isoladamente nem ao documento isoladamente —, o relacionamento não é resolvido por uma simples chave estrangeira, mas por uma **entidade associativa** (tabela de junção) dedicada — `probe_query_patent` e `probe_query_article` —, cada uma com restrição de unicidade sobre o par de chaves estrangeiras, o padrão canônico para modelar relacionamentos N:N com atributos próprios do relacionamento [20, 21]. A deduplicação em si acontece em um nível anterior ao relacionamento: `patent` e `article` têm `dedup_key` como coluna única, de modo que o mesmo documento — identificado pela combinação de fonte e identificador de registro na fonte — gera sempre uma única linha na tabela de documentos, não importa quantas *queries* ou sessões diferentes o recuperem; o que se multiplica, nesse caso, são as linhas da tabela associativa, cada uma amarrando aquele documento a uma *query* específica que o encontrou. Por contraste, os termos extraídos por PLN (`probe_query_term`) não são deduplicados nem compartilhados entre *queries*: cada termo só tem sentido no contexto da *query* que o gerou, o que

dispensa uma entidade associativa e justifica um relacionamento um-para-muitos simples entre `session_probe_query` e `probe_query_term`.

A Tabela 8 sintetiza os tipos de relacionamento empregados no esquema e onde cada um ocorre.

Tabela 8: Tipos de relacionamento no esquema relacional ativo

Tipo	Onde ocorre	Observação
Um-para-muitos	<code>research_session</code> → <code>session_input</code> , <code>session_probe_query</code> , <code>session_ai_call</code> , <code>session_report_section</code> ; <code>session_probe_query</code> → <code>session_chart</code>	Exclusão em cascata (<i>cascade delete-orphan</i>) a partir da sessão
Um-para-um	<code>research_session</code> → <code>session_report</code>	Um único relatório montado por sessão, sobrescrito a cada nova montagem
Auto- relacionamento (unário, opcional)	<code>session_input.parent_id</code> ; <code>session_probe_query.parent_id</code>	Distingue estágios do fluxo (raiz/exploração vs. variante/final) dentro da mesma tabela
Muitos-para- muitos com atributo (entidade associativa)	<code>session_probe_query</code> ↔ <code>patent/article</code> , via <code>probe_query_patent</code> <code>probe_query_article</code>	<code>relevance_score</code> é atributo do par <i>query</i> -documento, não de nenhum dos dois isoladamente
Deduplicação global (unicidade)	<code>patent.dedup_key</code> ; <code>article.dedup_key</code>	Garante uma única linha por documento, ainda que recuperado por múltiplas <i>queries</i> ou sessões

3.8 Estado Atual de Desenvolvimento

No estágio atual do projeto, o protótipo desenvolvido contempla, de ponta a ponta, as seguintes funcionalidades implementadas e testadas:

Arquitetura e estrutura do *backend*: a API REST com FastAPI está estruturada segundo o padrão de arquitetura hexagonal descrito na Seção 3.3, com os *endpoints* de interação com IA e busca (`/api/v1/chat/*`), de gerenciamento de sessões (`/api/v1/research-session/*` e `/api/v1/session-input/*`), de geração de gráficos e do relatório (`/api/v1/report/*`), de inferência estatística (`/api/v1/inference/*`) e de

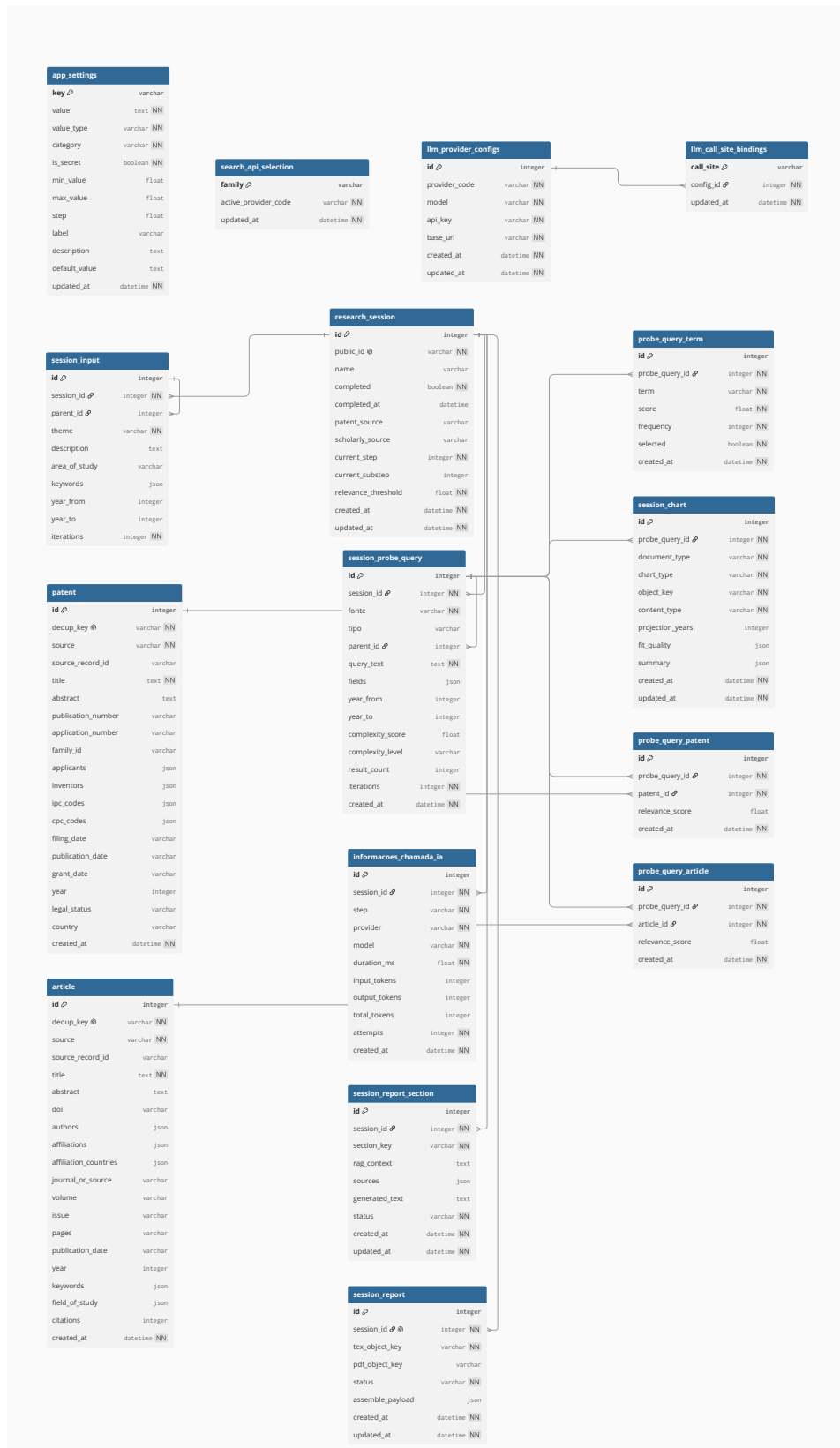


Figura 10: Diagrama entidade-relacionamento do esquema relacional ativo. Fonte: elaborado pelos autores com a ferramenta dbdiagram.io.

configuração (`/api/v1/config/*`) implementados e documentados via Swagger. O modelo de dados ativo segue o esquema relacional detalhado na Seção 3.7, implementado com SQLAlchemy assíncrono e gerenciado por migrações versionadas (Alembic), com PostgreSQL como banco de dados. Os módulos puros do núcleo (complexidade de consulta, curva S, estatística, posicionamento de figuras, citações, validação e revisão de texto) são cobertos por testes unitários automatizados.

Integração com OPS e Scopus: os construtores de consulta e serviços de busca para OPS e Scopus foram implementados e testados com consultas reais, validando a capacidade de recuperação de patentes e artigos a partir de termos fornecidos, incluindo o mecanismo de verificação automática da complexidade estrutural de cada consulta antes do envio, com nova tentativa e simplificação assistida por LLM em caso de consulta excessivamente complexa. O tratamento de erros, paginação, controle de taxa e novas tentativas em falhas de rede estão implementados para ambas as APIs, e a busca final distingue o total real de documentos informado pela base da amostra efetivamente analisada.

Extração de termos: implementada como um pipeline local de Processamento de Linguagem Natural (PatternRank, BM25F, KeyBERT, RRF e C-value), sem uso de LLM nesta etapa específica (Seção 4.3), conectado à interface de seleção de termos apresentada ao usuário.

Geração de gráficos e curva S: implementada e conectada a rotas dedicadas, que geram as figuras previstas no padrão REPTEC — volume anual, principais depositantes e instituições, quadros das classificações CPC e das áreas de estudo mais frequentes e curvas S de patentes e de artigos com os pontos GP/MP/SP e o diagnóstico de confiabilidade do ajuste (Seção 2.2) —, a partir dos documentos já persistidos da busca final da sessão. Nomes de depositantes ou instituições com mais de 45 caracteres são abreviados automaticamente, para não distorcer os gráficos.

Estimação de completude amostral e estabilidade de *ranking* (Chao1/*bootstrap*): implementada como uma rota de inferência estatística sobre a busca final (Seção 2.2.4), que enriquece iterativamente a amostra até sua saturação ou até o limite de tempo configurado.

RAG local e geração do relatório: implementados e conectados a rotas de produção. A indexação dos documentos da busca final no ChromaDB, a recuperação por similaridade com corte relativo de relevância e a redação das seções por LLM local são executadas seção a seção; as seções de texto fixo, as figuras, as citações, o quadro de busca, as assinaturas e a bibliografia são produzidos de forma determinística, e o `.tex` resultante segue o *template* REPTEC (Seção 4.5). A compilação em PDF é realizada pelo serviço TeX Live dedicado.

Persistência de artefatos (MinIO): implementada — os gráficos, as figuras fixas da Metodologia, as imagens anexadas pelo usuário, o `.tex` e o PDF são armazenados no MinIO, com as respectivas chaves registradas no PostgreSQL.

Interface *web*: implementada como um assistente guiado de quatro etapas (submissão de parâmetros, exploração inicial, exploração final e geração de relatório), incluindo edição de campos e da consulta completa, regeneração de consultas, gerenciamento e retomada de sessões, registro de auditoria de uso de IA e tela de configurações. A etapa de geração de relatório reúne o formulário do relatório (número, destinatário, objetivo, local,

referências administrativas, bibliografia e assinaturas, todos validados), a geração seção a seção e um editor LaTeX com painel de imagens e anexos, revisão do texto, compilação em PDF e exportação em .zip.

Validação do fluxo completo: o sistema foi exercitado de ponta a ponta, da submissão dos parâmetros iniciais à exportação do relatório compilado, com temas reais de prospecção e consultas efetivamente submetidas às APIs externas. A etapa de redação foi validada tanto com os modelos de menor porte executados localmente pela plataforma Ollama quanto com o modelo de linguagem hospedado no servidor interno do Instituto Militar de Engenharia, acessado por rede privada virtual — alternância que, conforme a Seção 3.4.1, não exigiu alteração de código, apenas o cadastro do *endpoint* correspondente na tela de configurações.

4 Pipeline de Processamento

O *pipeline* de prospecção tecnológica é composto por cinco fases executadas sequencialmente, cada uma com responsabilidades bem definidas, operando sobre a infraestrutura e as tecnologias apresentadas na Seção 3.6. A seguir, detalha-se o funcionamento de cada fase.

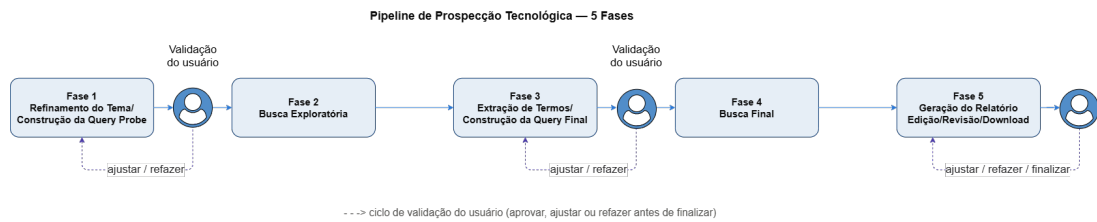


Figura 11: Visão geral das cinco fases do *pipeline* de prospecção tecnológica.

4.1 Fase 1: Refinamento do Tema

A primeira fase recebe o tema bruto fornecido pelo usuário e o processa por meio do modelo de LLM remota, que analisa o texto de entrada e gera um conjunto expandido de variações temáticas relacionadas ao tema, abrangendo sinônimos, termos técnicos correlatos e variações terminológicas em inglês — idioma predominante nas bases de patentes e artigos científicos. O resultado é apresentado ao usuário para validação, que pode aprovar a variação gerada, solicitar maior especificidade ou alterá-la diretamente.

O *prompt* utilizado nesta fase segue as diretrizes de *prompt engineering* descritas na Seção 2.4, em particular o padrão de definição de papel (*persona pattern*) e o uso de exemplos contrastantes bom/ruim [36], instruindo explicitamente o modelo a gerar variações técnicas e organizadas em ordem decrescente de relevância para o tema informado. A temperatura de inferência é mantida em 0,3 para privilegiar respostas consistentes [37].

4.2 Fase 2: Busca Exploratória

Com o tema validado, o sistema constrói consultas específicas para cada API — CQL para o OPS e a sintaxe própria para o Scopus — e realiza buscas exploratórias em

paralelo nas bases de artigos e patentes, limitadas a 10 resultados iniciais por API. O objetivo desta fase é recuperar uma amostra representativa de documentos que permita, na fase seguinte, identificar os termos mais relevantes e recorrentes no domínio pesquisado [27]. A duração típica é de 10 a 20 segundos.

Antes de cada consulta ser efetivamente enviada às APIs externas, sua complexidade estrutural é avaliada por um componente determinístico que conta operadores booleanos, profundidade de aninhamento de parênteses e número de termos, comparando o escore resultante contra um limiar configurável; caso o limiar seja excedido, o sistema solicita uma nova tentativa ao LLM, reinjetando no *prompt* o resultado exato da avaliação anterior (padrão de autocorreção por métrica determinística descrito na Seção 3.5), até um número máximo de tentativas — após o qual a tentativa menos complexa gerada é utilizada, com aviso ao usuário. Esse mecanismo existe porque consultas mais “abrangentes”, com muitos termos e operadores aninhados, tendem, na prática, a quebrar o analisador sintático das APIs remotas ou retornar erro/zero resultados, e não necessariamente mais resultados.

Os construtores de consulta (*query builders*) são responsáveis por transformar os termos validados em consultas booleanas estruturadas utilizando dicionários próprios de cada API. Por exemplo, o construtor para o OPS gera consultas CQL com operadores AND, OR e NOT, com filtros por campo (*ti* para título, *ab* para resumo, *pa* para depositante) e por ano de publicação. Já o construtor para o Scopus converte os termos em uma sintaxe compatível com a API da Elsevier, utilizando campos como TITLE, ABS e KEY, também com suporte a operadores lógicos e filtros temporais. Todos os construtores compartilham o mesmo contrato de entrada e saída, o que os torna substituíveis entre si pelo serviço de orquestração, conforme discutido na Seção 3.3 à luz do Princípio da Substituição de Liskov.

Os documentos recuperados nesta fase, antes de seguirem para a extração de termos (Seção 4.3), passam ainda pelo filtro de idioma e pelo filtro de relevância documento-tema descritos na Seção 2.7 — é esse último filtro, e não um critério definido diretamente pela API de busca, que determina o subconjunto efetivo de documentos que alimentará a extração de termos.

4.3 Fase 3: Extração de Termos

Os resultados da busca exploratória de cada tipo de base são processados por um pipeline local de PLN — e não por uma nova chamada ao LLM —, responsável por extrair, pontuar e ranquear os termos mais relevantes a partir de título e resumo de cada documento recuperado. Diferentemente da função de complexidade de consulta (Seção 3.5), que é uma métrica original criada pelos autores, o pipeline de extração de termos combina métodos consagrados na literatura de recuperação de informação (PatternRank, BM25F, KeyBERT, RRF e C-value, Seção 2.7), organizados por uma sequência de filtros — estes sim, decisões de projeto específicas deste trabalho — detalhada passo a passo a seguir.

Passo 1. Limpeza e delimitação do texto. Título e resumo de cada documento são convertidos para minúsculas, com remoção de URLs e substituição de hífens por espaços, e mantidos alinhados por documento (o BM25F exige saber qual título pertence a qual resumo). Vírgulas e pontos e vírgulas são convertidos

em pontos, e todo texto recebe pontuação final: sem essas fronteiras explícitas, o padrão gramatical do passo seguinte “vaza” do fim de um título para o início do próximo, ou entre itens de uma lista, produzindo candidatos que não existem em nenhum dos textos originais.

Passo 2. Geração de candidatos (PatternRank). Sobre o conjunto de textos, são extraídas as frases nominais maximais que obedecem ao padrão gramatical *adjetivo* substantivo+* [33], com até cinco palavras. Cada candidato já é uma expressão gramaticalmente completa, e não uma janela arbitrária de palavras.

Passo 3. Filtro de termos originais. São removidos os unigramas idênticos aos termos já informados pelo usuário no tema e na descrição; expressões de duas ou mais palavras são mantidas, ainda que contenham um termo original, por acrescentarem especificidade.

Passo 4. Filtro de qualidade lexical. Palavras de uma lista de cerca de 55 palavras de fronteira lexical (*a, an, the, of, is, are, this, that, high, low, new, such, about*) são removidas repetidamente das extremidades do candidato (“*the good permeability*” → “*good permeability*” → “*permeability*”), em vez de o candidato inteiro ser descartado; o candidato é descartado se qualquer palavra restante pertencer a uma lista de termos estruturais típicos de patentes (*wherein, comprising, said, first, second, configured, plurality, substantially, device*) ou de texto acadêmico (*proposed, novel, improved, significant, method, technique, framework, model, system, based, compared*). Candidatos que convergem para a mesma forma após o corte são deduplicados.

Passo 5. Canal léxico (BM25F). Para cada candidato t e documento d , a frequência do candidato em cada campo é normalizada pelo tamanho do campo e ponderada pelo peso do campo — 3 para o título e 1 para o resumo, premissa de que termos presentes no título são, em média, mais representativos do conteúdo do documento [29]:

$$\tilde{f}_{t,d} = w_{\text{tít}} \frac{f_{t,d}^{\text{tít}}}{B_d^{\text{tít}}} + w_{\text{res}} \frac{f_{t,d}^{\text{res}}}{B_d^{\text{res}}}, \quad B_d^c = (1 - b) + b \frac{|d^c|}{|d|},$$

e o escore agregado sobre os N documentos, dos quais n_t contêm o candidato, é

$$\text{BM25F}(t) = \sum_d \frac{\tilde{f}_{t,d}}{k_1 + \tilde{f}_{t,d}} \cdot \ln \left(1 + \frac{N - n_t + 0,5}{n_t + 0,5} \right),$$

com $k_1 = 1,2$ e $b = 0,75$, valores usuais na literatura [30].

Passo 6. Canal semântico (KeyBERT). Cada candidato e o texto do conjunto de documentos são convertidos em *embeddings* de sentença, e o escore do candidato é a similaridade de cosseno entre os dois vetores [12, 28].

Passo 7. Primeira fusão (saliência). Os dois canais são combinados por RRF [22], com $k = 60$: $\text{saliência}(t) = \frac{1}{60 + \text{pos}_{\text{BM25F}}(t)} + \frac{1}{60 + \text{pos}_{\text{KeyBERT}}(t)}$.

Passo 8. Qualidade estrutural. Cada candidato recebe um escore de qualidade estrutural: $-0,4$ para unigramas (palavras isoladas tendem a ser pouco informativas), 0 para bigramas e $+0,25$ para trigramas ou maiores (tendem a capturar

conceitos técnicos completos, como “*convolutional neural network*”), acrescido de $-0,8$ quando a sequência de classes gramaticais do candidato corresponde a um padrão catalogado como pouco informativo — por exemplo, dois verbos consecutivos ou a sequência advérbio-verbo-substantivo.

Passo 9. Segunda fusão (escore final). Saliência e qualidade estrutural são novamente combinadas por RRF, produzindo o escore final do candidato. Como ambas as fusões operam sobre posições, e não sobre escores, nenhum peso precisa ser calibrado entre escalas diferentes.

Passo 10. Redundância entre termos aninhados (C-value). Para cada candidato a , com $|a|$ palavras e frequência $f(a)$, sendo T_a o conjunto de candidatos mais longos que o contêm de forma contígua, calcula-se [23]

$$\text{C-value}(a) = \begin{cases} \log_2 |a| \cdot f(a), & \text{se } T_a = \emptyset, \\ \log_2 |a| \cdot \left(f(a) - \frac{1}{|T_a|} \sum_{b \in T_a} f(b) \right), & \text{caso contrário,} \end{cases}$$

(com $|a|$ limitado inferiormente a 2, para que unigramas não recebam $\log_2 1 = 0$). Candidatos com C-value não positivo — que ocorrem apenas como parte de expressões mais longas, sem uso independente — são descartados. Em seguida, entre os remanescentes ranqueados pelo escore final, um candidato contido de forma contígua em outro já mantido, de posição superior, é descartado: entre “*ultrafiltration membrane*” e “*composite ultrafiltration membrane*”, apenas o de maior escore permanece. Esse passo substitui um filtro anterior de sobreposição de palavras e, antes dele, um mecanismo de Relevância Marginal Máxima (*Maximal Marginal Relevance*, MMR) descartado por apresentar custo cúbico no número de candidatos.

Passo 11. Limiar e corte final. São mantidos os candidatos com escore final igual ou superior a $0,024$ — limiar calibrado para a escala do RRF, cujos valores ficam, com $k = 60$, aproximadamente entre $0,018$ e $0,033$ —, completados, se necessário, pelos melhores candidatos abaixo do limiar até um mínimo de 10 termos, e limitados a no máximo 60 termos, já ordenados pelo escore final.

Uma consequência de projeto desse encadeamento é a ausência de pesos a calibrar entre os canais léxico e semântico: como as duas fusões operam sobre posições, e não sobre escores, não existe um fator de ponderação arbitrado entre BM25F e KeyBERT. Os parâmetros que permanecem — os pesos de campo do BM25F (3 para o título e 1 para o resumo), a constante k do RRF e o limiar de corte — são fixos no código, não expostos à configuração do usuário.

O pipeline foi avaliado sobre conjuntos de documentos de dez domínios tecnológicos distintos, de dessalinização por membrana a biossensores vestíveis. Nesse conjunto, o processamento completo levou $10,4$ s no total — o KeyBERT é chamado uma única vez por requisição, e o conjunto de candidatos brutos se mantém em torno de 50 por lote — e os termos retornados receberam nota média de qualidade $8,1$, atribuída pelos autores. A geração de candidatos por padrão gramatical extrai corretamente expressões de quatro e cinco palavras, sem truncamento, o que preserva a especificidade dos termos oferecidos

ao usuário. Como limitações, a fusão por posições penaliza unigramas frequentes com força moderada, admitindo de dois a quatro unigramas pouco informativos por caso em domínios com palavras de alta frequência, e o padrão gramatical adotado não cobre gerúndios modificadores (como em “*privacy preserving*”). O limiar de 0,024 foi calibrado sobre esse conjunto de teste e deve ser revisto à medida que houver histórico real de seleção de termos pelos usuários.

Exemplo numérico de cálculo. Para ilustrar a aplicação conjunta dos Passos 7 a 11, considere dois candidatos extraídos de um conjunto de documentos sobre veículos autônomos: o trigramma “*autonomous vehicle navigation*”, 2.^o colocado no canal BM25F e 1.^o no KeyBERT, e o unigramma “*vehicle*”, 1.^o no BM25F (por ser muito frequente) e apenas 9.^o no KeyBERT. Aplicando o Passo 7:

$$\text{saliência}(\text{autonomous vehicle navigation}) = \frac{1}{60+2} + \frac{1}{60+1} = 0,01613 + 0,01639 = 0,03252$$

$$\text{saliência}(\text{vehicle}) = \frac{1}{60+1} + \frac{1}{60+9} = 0,01639 + 0,01449 = 0,03088$$

O trigramma, portanto, ocupa a 1.^a posição de saliência e o unigramma, a 2.^a: estar bem posicionado nos dois canais supera ser o primeiro em apenas um. No Passo 8, o trigramma recebe +0,25 de qualidade estrutural e o unigramma, −0,4; supondo que, no *ranking* de qualidade estrutural do lote, o trigramma ocupe a 3.^a posição e o unigramma a 40.^a, o Passo 9 resulta em

$$\text{final}(\text{autonomous vehicle navigation}) = \frac{1}{60+1} + \frac{1}{60+3} = 0,01639 + 0,01587 = 0,03226$$

$$\text{final}(\text{vehicle}) = \frac{1}{60+2} + \frac{1}{60+40} = 0,01613 + 0,01000 = 0,02613$$

Ambos superam o limiar de 0,024 (Passo 11), mas com uma margem bem maior para o trigramma, mais específico. Se o lote contivesse também “*vehicle navigation*”, com escore final inferior ao de “*autonomous vehicle navigation*”, ele seria descartado no Passo 10, por estar contido de forma contígua em um termo de posição superior.

A lista ranqueada resultante é apresentada ao usuário com indicação de relevância, junto da escolha, por fonte, do nível de abrangência desejado para a consulta final (específica, balanceada ou ampla) — sendo a diferença entre esses três níveis, na prática, o limiar mínimo de escore utilizado para selecionar quais dos termos marcados pelo usuário são oferecidos ao LLM como contexto na fase seguinte. Como o escore final do RRF vive em uma faixa estreita, ele é antes normalizado para o intervalo [0, 1] (normalização mín-máx sobre os termos selecionados), e os limiares são aplicados sobre esse valor: 0,4 para a variante específica, 0,3 para a balanceada e 0,2 para a ampla. Quanto mais específica a variante, maior o limiar e menor o conjunto de termos considerado. Esta fase é crítica para a qualidade dos resultados finais, pois as consultas construídas a partir de termos extraídos dos próprios documentos recuperados tendem a ser mais específicas e representativas do universo em questão — patentes ou artigos — do que aquelas baseadas apenas nos termos inicialmente fornecidos pelo usuário [27].

4.4 Fase 4: Busca Final

Com os termos selecionados pelo usuário, o modelo de LLM remoto gera a variante de consulta final escolhida, seguindo o mesmo mecanismo de verificação de complexidade e retentativa descrito na Fase 2; o sistema reconstrói as consultas utilizando os construtores de consulta, combinando os termos em estruturas booleanas otimizadas, e executa a busca final em ambas as APIs. Os resultados são consolidados com remoção de duplicatas, gerando o conjunto definitivo de patentes e artigos que fundamentará a análise.

Nesta fase, aplica-se particular atenção à construção de consultas que não excedam os limites de complexidade aceitos pelas APIs. Consultas com número excessivo de termos encadeados por operadores **OR** podem ser rejeitadas ou truncadas, o que exige uma estratégia de particionamento e combinação de resultados implementada no serviço de busca.

Três decisões de robustez da busca final têm impacto direto sobre a fidelidade dos indicadores do relatório. Primeiro, o sistema distingue o **total real** de documentos informado pela base para a consulta — ou, quando a base não o informa, a soma das contagens por ano — da **amostra** efetivamente baixada e analisada: o total real é o que aparece no quadro de busca do relatório, e a amostra, o que fundamenta as análises. Segundo, na contagem de artigos por ano, base da curva S científica, falhas de rede são repetidas automaticamente e, persistindo, o ano afetado é **omitido** da série, e nunca registrado como zero — um zero falso criaria um “vale” artificial que distorceria o ajuste logístico. Terceiro, nomes de depositantes em escrita não latina (chinês, japonês, coreano) são substituídos pela forma latinizada fornecida pela própria API OPS, já que o compilador LaTeX descarta esses caracteres e o depositante apareceria em branco nos gráficos e no texto.

4.5 Fase 5: Geração do Relatório

A última fase transforma os documentos da busca final em um relatório no padrão REPTEC da AGITEC [3, 4], editável e compilável em PDF. Ela é organizada em cinco etapas: consolidação e gráficos, redação das seções, montagem do documento, revisão e exportação. Um exemplo completo do *prompt* enviado ao modelo na redação de uma seção encontra-se no Apêndice A.

4.5.1 Consolidação e gráficos

O módulo de consolidação agrega os documentos coletados por ano, por depositante e por classificação CPC para as patentes, e por ano, por instituição e por área de estudo para os artigos, ajusta as curvas S de patentes e de artigos e calcula os pontos GP/MP/SP com seu diagnóstico de confiabilidade (Seção 2.2). Somente as figuras previstas no padrão REPTEC são geradas: histórico anual de patentes e de artigos, principais depositantes e instituições, quadros com as dez classificações CPC e as dez áreas de estudo mais frequentes e as duas curvas S. As séries consideram apenas anos completos, as classificações IPC são tratadas como CPC e descritas pelos títulos oficiais da classificação, e nomes com mais de 45 caracteres são abreviados. O título de cada figura não é desenhado na imagem, mas escrito na legenda LaTeX, seguida da indicação “Fonte: O autor.”, como no relatório de referência. Cada gráfico é armazenado no MinIO junto de seu resumo numérico, e é

esse resumo — e não a imagem — que o modelo de linguagem recebe para descrever a figura.

4.5.2 Redação das seções

Nem toda seção do relatório passa pelo modelo de linguagem. A estrutura, extraída de um REPTEC real [4], separa o que depende dos dados da busca do que é essencialmente igual em todo relatório, como resume a Tabela 9.

Tabela 9: Seções do relatório e forma de produção

Seção	Como é produzida
Capa e sumário	<i>Template</i> LaTeX
1 Finalidade	Texto fixo, interpolando o tema e o destinatário informados
2 Referências	Documentos administrativos informados pelo usuário (DIEx, ofícios)
3 Objetivo	Texto informado pelo usuário
4 Introdução	LLM, com contexto do RAG
5 Metodologia (5.1 a 5.4)	Texto fixo com as figuras de referência sobre curva S [15, 38], descrição genérica do apoio computacional e interpolação das palavras-chave, do período e das bases consultadas
6 Resultados (6.1 Científicas, 6.2 Tecnológicas, 6.3 Tendências e Ciclo de Vida)	LLM, uma chamada por subseção, com contexto do RAG e resumos numéricos dos gráficos; a 6.2 inclui o quadro com as consultas de busca e o total real de documentos
7 Conclusão	LLM, com estrutura orientada (estágio do ciclo de vida, destaques e recomendação)
8 Referências bibliográficas	Bibliografia fixa da Metodologia, obras efetivamente citadas no texto e obras adicionadas pelo usuário
Local, data e assinaturas	Formulário (nome, posto/graduação e função de quem elabora, revisa e aprova)

Cada seção redigida por LLM passa por duas requisições separadas: a primeira indexa os documentos da sessão, se ainda não indexados, e recupera e persiste o contexto da seção (Seção 2.5); a segunda monta o *prompt* com esse contexto, chama o modelo e persiste o texto. Essa divisão permite à interface mostrar o andamento seção a seção e mantém cada chamada ao modelo restrita ao contexto de uma única seção, o que é relevante para modelos locais de janela de contexto pequena.

Há ainda uma terceira razão, e é a mais determinante na prática: como o texto

de cada seção é persistido assim que fica pronto, uma falha na geração de uma seção — seja por indisponibilidade do modelo, seja por reprovação na validação descrita adiante — não descarta o que já foi produzido. O usuário regeira apenas a seção afetada, em vez de repetir a redação do relatório inteiro, o que seria especialmente custoso com modelos de menor porte, cujo tempo de inferência por seção é da ordem de dezenas de segundos.

O *prompt* de sistema impõe texto impessoal e proíbe expressões que revelam o funcionamento interno do sistema (escores de relevância, “N/A”, “no contexto fornecido”). Em torno do modelo, módulos determinísticos garantem o que não pode depender dele:

- **Figuras no contexto.** O modelo é instruído a seguir o padrão do relatório de referência — apresentar a figura, exibi-la e, em seguida, interpretar seus números — escrevendo marcadores para a referência e para a posição de cada figura. Um pós-processador converte os marcadores em comandos LaTeX; se o modelo cita uma figura sem posicioná-la, ela é inserida logo após o parágrafo que a cita, e figuras não citadas vão para o fim da seção, na ordem do relatório de referência. O mesmo pós-processador corrige “Figura” e “Quadro” conforme o tipo do elemento, ajustando a concordância do artigo (“a Figura”, “o Quadro”).
- **Ciclo de vida calculado.** O estágio da tecnologia (emergente, crescimento, maturidade ou saturação) é calculado a partir de GP, MP e SP e entregue ao modelo como fato, e não inferido por ele.
- **Citações rastreáveis.** Só são aceitas citações no formato “(SOBRENOME et al., ano)” que correspondam a documentos recuperados pelo RAG; citações inventadas são removidas do texto, autores homônimos recebem sufixo de letra (2020a, 2020b), e as referências ABNT das obras efetivamente citadas são acrescentadas à seção de referências.
- **Validação do texto.** Números são convertidos para o padrão brasileiro (17,3%; 1.800) e o texto é inspecionado em busca de vestígios do *pipeline*; se algum for encontrado, a seção é regenerada uma vez com instrução de correção e, persistindo o problema, a geração falha com mensagem explícita, em vez de produzir um texto defeituoso.

4.5.3 Montagem do documento

A montagem reúne as seções persistidas e as imagens já armazenadas no MinIO — sem disparar nenhuma nova geração — e renderiza o *template* LaTeX, que reproduz a formatação do relatório de referência: cabeçalho com o número do relatório e o tema, rodapé “Página x de y ”, títulos de seção em negrito e caixa alta, texto justificado com recuo de parágrafo, espaçamento de 1,5, figuras e quadros numerados com legenda e fonte, e bloco de assinaturas no formato “NOME – POSTO”, com a função na linha seguinte. O `.tex` resultante é armazenado no MinIO junto de uma cópia de referência, usada depois pela revisão para identificar o que o usuário editou. A compilação em PDF é uma ação separada, sempre disparada pelo usuário, executada pelo serviço TeX Live dedicado; uma falha de compilação não apaga o `.tex` já montado.

4.5.4 Edição e revisão

O `.tex` é aberto em um editor na própria interface, com um painel lateral de imagens — as geradas pelo sistema, as figuras fixas e as anexadas pelo usuário — a partir do qual cada imagem pode ser inserida no documento, aberta, baixada ou, no caso dos anexos, excluída. A revisão do documento combina três verificações:

- (a) **Verificação de LaTeX:** chaves e ambientes desbalanceados, referências a rótulos ou imagens inexistentes, caracteres especiais sem escape e erros do último registro de compilação.
- (b) **LanguageTool:** ortografia, acentuação, concordância e gramática, aplicados apenas às seções geradas por LLM e às linhas editadas pelo usuário — identificadas pela comparação com a cópia de referência da montagem —, de modo que o texto fixo, já revisado, não é reprocessado. Os comandos LaTeX são enviados como marcação, e não como texto, para não gerarem falsos positivos.
- (c) **Revisão por LLM (opcional):** uma segunda opinião, com ponto de uso de LLM próprio — o que permite empregar um modelo melhor em português que o da redação —, restrita a concordância, acentuação e ortografia. Uma sugestão só é aceita se o trecho original existir literalmente no texto e se a correção não alterar números, citações ou comandos LaTeX.

As sugestões são apresentadas por seção, e o usuário aplica ou ignora cada uma individualmente.

4.5.5 Exportação

Além do PDF, o sistema exporta um pacote `.zip` com o `.tex` e todas as imagens nele referenciadas, nomeado segundo o número e o ano do relatório, o que permite ao usuário continuar a edição em qualquer ambiente LaTeX, como o Overleaf.

5 Conclusão

5.1 Síntese das Contribuições

O sistema desenvolvido cumpriu o objetivo de automatizar o percurso completo de uma prospecção tecnológica, da submissão do tema à exportação do relatório no padrão REPTEC. Um levantamento que, conduzido manualmente, consome dias de trabalho qualificado — definição de termos de busca, consulta a bases com sintaxes distintas, triagem de resultados, apuração de instituições e depositantes, análise temporal e consolidação em documento estruturado [24, 27] — passou a ser executado como um fluxo único, com o esforço humano concentrado nos pontos de decisão: validação do tema, seleção dos termos, escolha da consulta final e revisão do texto produzido. Essa redução libera tempo dos técnicos e engenheiros envolvidos em projetos de PD&I do Exército Brasileiro para atividades de maior valor estratégico.

Os relatórios gerados seguem a estrutura em oito seções do padrão REPTEC, com as figuras previstas apresentadas e interpretadas no texto, o quadro das consultas efetivamente submetidas, o estágio do ciclo de vida calculado a partir do ajuste da curva S e as referências restritas aos documentos recuperados na busca. A viabilidade metodológica do formalismo GP/MP/SP adotado para leitura da curva S já estava demonstrada na prática institucional da AGITEC com dados reais de patentes e publicações, como evidenciado em [4], e sua automação mostrou-se aderente a esse precedente.

A separação entre o que é produzido pelo modelo de linguagem e o que é produzido deterministicamente foi decisiva para esse resultado. Os números das figuras, o estágio do ciclo de vida, as seções de texto fixo, o posicionamento das figuras e a bibliografia não dependem do modelo, o que restringe a atuação dele à redação do texto corrido — justamente onde o embasamento é garantido pelo contexto recuperado via RAG e verificado pelo filtro de citações. É importante ressaltar que o sistema não substitui o julgamento especializado do analista humano, mas fornece um documento estruturado e rastreável que torna esse julgamento mais eficiente e informado — premissa que reflete o papel da IA como ferramenta de apoio à decisão, e não como substituto do especialista [13, 16].

Além do sistema em si, destacam-se como contribuições deste trabalho: a função heurística de complexidade de consulta e o padrão de autocorreção por métrica determinística (Seção 3.5), que reinjeta no *prompt* os valores medidos na tentativa anterior em vez de depender da autoavaliação do próprio modelo; o *pipeline* local de extração e pontuação de termos (Seção 4.3), que dispensa o uso de LLM nessa etapa; o emprego da estimação de completude amostral e de estabilidade de *ranking* (Seção 2.2.4) para qualificar a confiabilidade dos indicadores apresentados; e a configuração dos modelos de linguagem por ponto de uso (Seção 3.4.1), que permite alternar entre provedores remotos, modelos locais e o servidor interno do IME sem alteração de código.

5.2 Atendimento aos Objetivos

O objetivo geral — automatizar a prospecção tecnológica por meio da consulta integrada a bases de artigos e patentes, da análise dos resultados e da geração de um relatório no padrão da AGITEC — foi atingido, conforme o estado de desenvolvimento descrito na Seção 3.8. Quanto aos objetivos específicos:

- i. A integração com modelos de linguagem remotos e locais, configuráveis por ponto de uso, foi implementada e exercitada com modelos locais via Ollama e com o modelo do servidor interno do IME.
- ii. A integração com as APIs OPS e Scopus foi implementada e validada com consultas reais, incluindo paginação, controle de taxa e novas tentativas em falhas de rede.
- iii. Os construtores de consulta traduzem os termos selecionados para a sintaxe de cada base, com operadores booleanos, filtros temporais e campos específicos, e têm a complexidade estrutural verificada antes do envio.
- iv. A geração de consultas por LLM foi combinada ao *pipeline* local de PLN para extração e pontuação de termos a partir dos resultados exploratórios.

- v. O ciclo de validação humana está presente em todas as decisões do fluxo: validação do tema, seleção dos termos e escolha da consulta final.
- vi. Os metadados das diferentes APIs são consolidados em formato unificado e agregados por ano, depositante ou instituição e classificação, complementados pela inferência estatística por Chao1 e *bootstrap*.
- vii. A fase de maturidade tecnológica é calculada automaticamente a partir do ajuste da curva S, com os pontos GP/MP/SP e o diagnóstico de confiabilidade do ajuste.
- viii. O relatório é gerado no padrão REPTEC, em LaTeX editável, com as seções redigidas por RAG sobre os documentos indexados no ChromaDB.
- ix. A persistência foi distribuída entre PostgreSQL (metadados), ChromaDB (*embeddings*) e MinIO (gráficos, *.tex* e PDF).
- x. A interface *web* foi implementada como assistente guiado, com auditoria do uso de IA, retomada de sessões, editor do relatório, compilação em PDF e exportação.

5.3 Limitações

O sistema apresenta algumas limitações que devem ser consideradas. A qualidade do texto produzido é diretamente proporcional à capacidade do modelo vinculado ao ponto de uso de redação [37]: com os modelos de menor porte executados localmente (Qwen 2.5 (3B) e Gemma 3 (4B)), adequados a máquinas comuns mas de capacidade inferior à de modelos maiores [5], observaram-se falhas recorrentes de português formal — sobretudo na concordância verbal com sujeito distante — e menor profundidade analítica. Modelos de maior porte, como o hospedado no servidor interno do Instituto Militar de Engenharia, atenuam substancialmente esse efeito, mas o deslocam para outra dependência: a disponibilidade da infraestrutura institucional e do acesso à rede interna. Em qualquer dos casos, a etapa de revisão (Seção 4.5) detecta boa parte dos erros de superfície, mas não substitui a leitura do especialista. O corpus do RAG é composto apenas por título e resumo dos documentos da busca final, e, como os resumos de artigos nem sempre são devolvidos pelas APIs, tende a ser majoritariamente de patentes, o que pode empobrecer a fundamentação das seções sobre a produção científica. O limiar de corte da extração de termos foi calibrado sobre um conjunto de teste, e não sobre histórico real de uso. Por fim, o sistema depende de chave de acesso institucional para a API Scopus, o que pode limitar sua disponibilidade em contextos sem convênio ativo com a Elsevier.

Adicionalmente, a heterogeneidade dos dados retornados pelas diferentes APIs impõe desafios de normalização que, apesar de tratados pelo módulo de consolidação, podem resultar em inconsistências pontuais quando as APIs alteram seus formatos de resposta. O fenômeno de alucinação dos LLMs [13], embora mitigado pela adoção do RAG [16], pela restrição das citações aos documentos recuperados e pela produção determinística das seções fixas, dos números dos gráficos e do estágio do ciclo de vida, não é completamente eliminado, o que reforça a necessidade de revisão humana dos relatórios gerados antes de sua utilização em decisões estratégicas.

5.4 Trabalhos Futuros

As limitações identificadas apontam os principais desdobramentos para a continuidade do trabalho:

- **Avaliação com usuários reais:** conduzir, junto à AGITEC, uma avaliação comparativa entre relatórios elaborados manualmente e relatórios gerados pelo sistema sobre os mesmos temas, medindo o tempo de elaboração e a qualidade percebida pelos analistas.
- **Ampliação do corpus do RAG:** incorporar o texto completo dos documentos, quando disponível, e fontes complementares de resumos de artigos, de modo a equilibrar a fundamentação entre patentes e produção científica.
- **Novas bases de busca:** integrar outras APIs de patentes e artigos, reduzindo a dependência da chave institucional da Scopus.
- **Calibração contínua:** recalibrar o limiar da extração de termos e os parâmetros da função de complexidade a partir do histórico real de uso registrado pelo próprio sistema.
- **Qualidade textual:** avaliar modelos de maior porte ou ajustados ao português formal e ampliar a etapa automática de revisão do texto.
- **Implantação institucional:** implantar o sistema na infraestrutura do Exército Brasileiro, com o armazenamento de artefatos em instância remota do MinIO e controle de acesso por usuário.

5.5 Considerações Finais

Este trabalho mostrou que é viável automatizar, de ponta a ponta, a elaboração de relatórios de prospecção tecnológica no padrão exigido pelo Exército Brasileiro, combinando modelos de linguagem, recuperação de informação e processamento determinístico. O ponto central da solução está na divisão de responsabilidades: ao modelo de linguagem cabe apenas a redação, embasada nos documentos recuperados; aos componentes determinísticos cabem os números, as figuras e as referências; e ao analista humano cabem as decisões que orientam a pesquisa e a revisão do resultado. Essa divisão torna o relatório rastreável e verificável, características indispensáveis a um documento que subsidia decisões estratégicas de PD&I. Espera-se que o sistema, desenvolvido a partir de uma demanda real da AGITEC, contribua para reduzir o esforço dedicado a levantamentos manuais e para tornar a prospecção tecnológica uma prática mais frequente e sistemática no âmbito do Exército Brasileiro.

Referências

- [1] RODRÍGUEZ-SALVADOR, Marisela; RIO-BELVER, Rosa María; GARECHANA-ANACABE, Gaizka. Scientometric and patentometric analyses to determine the knowledge landscape in innovative technologies: the case of 3D bioprinting. *PLOS ONE*, v. 12, n. 6, e0180375, 2017. DOI: 10.1371/journal.pone.0180375.
- [2] LEZAMA-NICOLÁS, René; RODRÍGUEZ-SALVADOR, Marisela; RÍO-BELVER, Rosa; BILDOSOLA, Iñaki. A bibliometric method for assessing technological maturity: the case of additive manufacturing. *Scientometrics*, v. 117, n. 3, p. 1425–1452, 2018. DOI: 10.1007/s11192-018-2941-1.
- [3] AGITEC — Agência de Gestão e Inovação Tecnológica do Exército Brasileiro. *Metodologia de Avaliação e Análise Tecnológica AVANTEC V2*. Manual Técnico. Departamento de Ciência e Tecnologia. Rio de Janeiro: Exército Brasileiro, 2024.
- [4] GUIMARÃES, Ricardo Wagner Amorim; BALTHAZAR NETO, Antonio Victorino Pereira; GALANTE, Erick Braga Ferrão. *REPTEC 001/2023 — Relatório de Prospeção Tecnológica: Terrestrial Trunked Radio (TETRA)*. Rio de Janeiro: Agência de Gestão e Inovação Tecnológica (AGITEC), Exército Brasileiro, 2023.
- [5] BROWN, T. B. et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, v. 33, p. 1877–1901, 2020. Disponível em: <https://arxiv.org/abs/2005.14165>. Acesso em: 05 maio 2026.
- [6] CHAO, Anne. Nonparametric estimation of the number of classes in a population. *Scandinavian Journal of Statistics*, v. 11, n. 4, p. 265–270, 1984.
- [7] EFRON, Bradley. Bootstrap methods: another look at the jackknife. *The Annals of Statistics*, v. 7, n. 1, p. 1–26, 1979. DOI: 10.1214/aos/1176344552.
- [8] COCKBURN, Alistair. *Hexagonal Architecture* (Ports and Adapters). HaT Technical Report 2005.02, 4 set. 2005. Disponível em: <https://alistair.cockburn.us/hexagonal-architecture/>. Acesso em: 10 ago. 2026.
- [9] FISHER, J. C.; PRY, R. H. A simple substitution model of technological change. *Technological Forecasting and Social Change*, v. 3, p. 75–88, 1971. DOI: 10.1016/S0040-1625(71)80005-7.
- [10] FOSTER, R. N. *Innovation: The Attacker’s Advantage*. New York: Summit Books, 1986. 316 p.
- [11] GAO, Y. et al. Retrieval-augmented generation for large language models: a survey. *arXiv*. arXiv:2312.10997, mar. 2024. Disponível em: <https://arxiv.org/abs/2312.10997>. Acesso em: 05 maio 2026.
- [12] GROOTENDORST, Maarten. KeyBERT: Minimal keyword extraction with BERT. *Zenodo*, 2020. DOI: 10.5281/zenodo.4461265.
- [13] JI, Z. et al. Survey of hallucination in natural language generation. *ACM Computing Surveys*, v. 55, n. 12, art. 248, 2023. DOI: 10.1145/3571730.

- [14] ERNST, H. The use of patent data for technological forecasting: the diffusion of CNC-technology in the machine tool industry. *Small Business Economics*, v. 9, n. 4, p. 361–381, 1997.
- [15] KUCHARAVY, D.; DE GUIO, R. Application of S-shaped curves. *Procedia Engineering*, v. 9, p. 559–572, 2011. DOI: 10.1016/j.proeng.2011.03.142.
- [16] LEWIS, P. et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, v. 33, p. 9459–9474, 2020. Disponível em: <https://arxiv.org/abs/2005.11401>. Acesso em: 05 maio 2026.
- [17] MANKINS, John C. *Technology Readiness Levels: A White Paper*. Washington, DC: Advanced Concepts Office, Office of Space Access and Technology, NASA, 6 abr. 1995.
- [18] MARTIN, Robert C. *Agile Software Development: Principles, Patterns, and Practices*. Upper Saddle River: Prentice Hall, 2002.
- [19] MARTIN, Robert C. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston: Pearson, 2017.
- [20] CHEN, Peter Pin-Shan. The entity-relationship model — toward a unified view of data. *ACM Transactions on Database Systems*, v. 1, n. 1, p. 9–36, 1976. DOI: 10.1145/320434.320440.
- [21] CODD, Edgar F. A relational model of data for large shared data banks. *Communications of the ACM*, v. 13, n. 6, p. 377–387, 1970. DOI: 10.1145/362384.362685.
- [22] CORMACK, G. V.; CLARKE, C. L. A.; BUETTCHER, S. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In: INTERNATIONAL ACM SIGIR CONFERENCE ON RESEARCH AND DEVELOPMENT IN INFORMATION RETRIEVAL, 32., 2009, Boston. *Proceedings [...]*. New York: ACM, 2009. p. 758–759. DOI: 10.1145/1571941.1572114.
- [23] FRANTZI, K.; ANANIADOU, S.; MIMA, H. Automatic recognition of multi-word terms: the C-value/NC-value method. *International Journal on Digital Libraries*, v. 3, n. 2, p. 115–130, 2000. DOI: 10.1007/s007999900023.
- [24] MAYERHOFF, Z. D. V. L. Uma análise sobre os estudos de prospecção tecnológica. *Cadernos de Prospecção*, v. 1, n. 1, p. 7–9, 2008.
- [25] MIKOLOV, T. et al. Distributed representations of words and phrases and their compositionality. *Advances in Neural Information Processing Systems*, v. 26, p. 3111–3119, 2013. Disponível em: <https://arxiv.org/abs/1310.4546>. Acesso em: 05 maio 2026.
- [26] NIELSEN, Michael A. *Neural Networks and Deep Learning*. Determination Press, 2015. Disponível em: <http://neuralnetworksanddeeplearning.com/>. Acesso em: 10 ago. 2026.
- [27] PORTER, A. L. et al. Technology futures analysis: toward integration of the field and new methods. *Technological Forecasting and Social Change*, v. 71, n. 3, p. 287–303, mar. 2004. DOI: 10.1016/j.techfore.2003.11.004.

- [28] REIMERS, N.; GUREVYCH, I. Sentence-BERT: sentence embeddings using siamese BERT-networks. In: *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019, Hong Kong. *Proceedings...* Stroudsburg: Association for Computational Linguistics, 2019. p. 3982–3992. Disponível em: <https://arxiv.org/abs/1908.10084>. Acesso em: 05 maio 2026.
- [29] ROBERTSON, S.; ZARAGOZA, H.; TAYLOR, M. Simple BM25 extension to multiple weighted fields. In: *ACM INTERNATIONAL CONFERENCE ON INFORMATION AND KNOWLEDGE MANAGEMENT (CIKM)*, 13., 2004, Washington. *Proceedings [...]*. New York: ACM, 2004. p. 42–49. DOI: 10.1145/1031171.1031181.
- [30] ROBERTSON, S.; ZARAGOZA, H. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, v. 3, n. 4, p. 333–389, 2009. DOI: 10.1561/15000000019.
- [31] ROGERS, E. M. *Diffusion of Innovations*. 5. ed. New York: Free Press, 2003. 551 p.
- [32] SALTON, G.; BUCKLEY, C. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, v. 24, n. 5, p. 513–523, 1988. DOI: 10.1016/0306-4573(88)90021-0.
- [33] SCHOPF, T.; KLIMEK, S.; MATTHES, F. PatternRank: leveraging pretrained language models and part of speech for unsupervised keyphrase extraction. In: *INTERNATIONAL CONFERENCE ON KNOWLEDGE DISCOVERY AND INFORMATION RETRIEVAL (KDIR)*, 14., 2022, Valletta. *Proceedings [...]*. Setúbal: SciTePress, 2022. p. 243–248. DOI: 10.5220/0011546600003335.
- [34] VASWANI, A. et al. Attention is all you need. *Advances in Neural Information Processing Systems*, v. 30, 2017. Disponível em: <https://arxiv.org/abs/1706.03762>. Acesso em: 05 maio 2026.
- [35] WEI, J. et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, v. 35, p. 24824–24837, 2022. Disponível em: <https://arxiv.org/abs/2201.11903>. Acesso em: 05 maio 2026.
- [36] WHITE, J. et al. A prompt pattern catalog to enhance prompt engineering with ChatGPT. *arXiv*. arXiv:2302.11382, fev. 2023. Disponível em: <https://arxiv.org/abs/2302.11382>. Acesso em: 05 maio 2026.
- [37] ZHAO, W. X. et al. A survey of large language models. *arXiv*. arXiv:2303.18223, mar. 2023. Disponível em: <https://arxiv.org/abs/2303.18223>. Acesso em: 05 maio 2026.
- [38] MADEU, F. C. B. *Prospecção tecnológica utilizando a análise multicritério e técnicas bibliométricas: estudo de caso para o setor de Defesa*. 144 p. 2019. Dissertação (Mestrado em Engenharia de Defesa) – Instituto Militar de Engenharia, Rio de Janeiro, 2019.

APÊNDICE A – PROMPTS DE INTELIGÊNCIA ARTIFICIAL NO PIPELINE DE PROSPECÇÃO

A.1 Organização dos *prompts*

Toda chamada a um LLM no sistema combina duas partes. A primeira é um *prompt* de sistema fixo, armazenado em arquivo no repositório (diretório `config/prompts/`) e idêntico para todas as sessões: ele define o papel do modelo, as regras obrigatórias e o formato exato da saída. A segunda é o conjunto de dados da sessão, inserido pelo código no momento da chamada: o tema e os parâmetros informados pelo usuário, os termos extraídos dos documentos, as métricas de complexidade da tentativa anterior ou os trechos recuperados pelo RAG, conforme a fase. Essa separação é o que permite que um mesmo conjunto de instruções, revisado e versionado junto com o código, seja reutilizado em qualquer prospecção. A Tabela 10 resume as chamadas a LLM ao longo das cinco fases do *pipeline*; o modelo usado em cada uma é definido pelo ponto de uso correspondente (Seção 3.4.1).

Tabela 10: Chamadas a LLM ao longo do *pipeline*

Fase	Ponto de uso	Prompt fixo	Dados da sessão inseridos	Saída
1 – Refinamento do tema	theme_candidates	refine_topic_system_prompt.txt (e specify_topic_system_prompt.txt)	Tema, descrição, área de estudo, palavras-chave e campos a preservar	JSON com quatro variações do tema
2 – Busca exploratória	probe_query	probe_system_prompt.txt e dica de campos da API	Tema escolhido, descrição, área e palavras-chave; na retentativa, as métricas de complexidade	JSON com os campos de busca
3 – Extração de termos	–	Não usa LLM	–	–
4 – Busca final	final_query	final_system_prompt.md	Variante, tipo de documento, termos extraídos com pontuação e códigos IPC/CPC observados; na retentativa, as métricas de complexidade	JSON com os campos de busca
5 – Redação do relatório	report_writing	REPORT_SYSTEM_PROMPT e <i>prompt</i> da seção	Trechos recuperados pelo RAG com a citação, números das figuras, estágio do ciclo de vida e tema	Texto corrido da seção
5 – Revisão do texto	report_review	AI_REVIEW_SYSTEM_PROMPT	Parágrafo gerado a revisar	JSON com as correções

Nas listagens deste apêndice, os valores que vêm da sessão aparecem **destacados em azul**; todo o restante é texto fixo. Os *prompts* das Fases 1, 2 e 4 são redigidos em inglês, idioma das consultas submetidas às bases OPS e Scopus, e foram mantidos no original. Os da Fase 5 são redigidos em português, idioma do relatório. Os trechos omitidos dos *prompts* fixos estão indicados por [...].

A.2 Sessão de exemplo

Para mostrar como os dados de uma sessão entram nos *prompts*, adota-se uma sessão ilustrativa, cujos valores são fictícios, mas estão no formato exato produzido pelo sistema. As listagens com dados da sessão foram geradas pelas próprias funções do código que montam os *prompts*, alimentadas com os valores abaixo.

- **Parâmetros informados pelo usuário:** tema “Drones para vigilância de fronteiras”, área de estudo “Engenharia” e palavras-chave “UAV” e “sensoriamento”, sem descrição.
- **Tema escolhido na Fase 1:** “Autonomous UAV-based surveillance systems for border monitoring”, uma das quatro variações propostas pelo modelo, com a descrição gerada por ele.
- **Resultado da Fase 3:** oito termos extraídos dos documentos exploratórios, entre eles *unmanned aerial vehicle*, *border surveillance* e *drone detection*, e quatro códigos IPC observados nas patentes recuperadas.
- **Resultado da busca final:** 1.842 artigos (2005–2025) e 967 patentes (2008–2025), com as curvas S de artigos em estágio de maturidade e de patentes em estágio de crescimento.

A.3 Fase 1: Refinamento do tema

O *prompt* fixo instrui o modelo a transformar o tema informado em quatro variações específicas e distintas entre si (Listagem 1). O código monta a mensagem com os campos preenchidos no formulário e acrescenta a lista dos campos que o modelo deve devolver, indicando os valores a preservar (Listagem 2). Os campos não informados pelo usuário, como a descrição neste exemplo, simplesmente não aparecem na mensagem. Mesmo com a instrução de preservação, o código sobrescreve a área de estudo e as palavras-chave da resposta com os valores originais.

Listagem 1: Trecho do *prompt* fixo da Fase 1 (`refine_topic_system_prompt.txt`)

```
You are a specialist in technology foresight and thematic refinement.

Your task is to receive a user-defined theme and refine it into 4 more specific,
distinct, and technically meaningful variations.

Your output MUST be a valid JSON object following the exact structure defined below.
[...]
2. FIELD PRESERVATION
- If "area_of_study" is provided → COPY EXACTLY in all candidates
- If "keywords" are provided → COPY EXACTLY in all candidates
```

```

- NEVER modify, translate, or expand provided keywords
[...]
5. DIVERSITY REQUIREMENT (CRITICAL)
Each generated theme MUST be substantially different from the others.

Each variation must introduce a distinct angle, such as:
- different application domain
- different technical approach or methodology
- different operational context
- different scale (tactical vs strategic)
- different constraints or environments
[...]
BAD EXAMPLE:
- "AI for drones"
- "Artificial intelligence in drones"
- "Machine learning for UAVs"

GOOD EXAMPLE:
- "Autonomous swarm coordination for military drones in contested environments"
- "Computer vision-based target recognition in low-visibility drone operations"
- "Energy-efficient flight optimization for long-endurance UAV missions"
- "Secure communication protocols for anti-jamming drone systems"
[...]
OUTPUT STRUCTURE

{
  "candidates": [
    {
      "theme": "...",
      "description": "...",
      (include "area_of_study" ONLY if provided in input),
      (include "keywords" ONLY if provided in input)
    },
    [...]
  ]
}

```

Listagem 2: Mensagem montada com os dados da sessão na Fase 1

```

Tema: Drones para vigilância de fronteiras
Área de Estudo: Engenharia
Palavras-chave: UAV, sensoriamento

Campos fornecidos pelo usuário (retorne APENAS estes):
- theme: SIM (sempre refine em 4 variações)
- description: SIM (gere sempre uma descrição nova para cada variação, mesmo que nenhuma tenha sido informada)
- area_of_study: SIM (PRESERVE exatamente: 'Engenharia')
- keywords: SIM (PRESERVE exatamente: ['UAV', 'sensoriamento'])

```

A.4 Fase 2: Busca exploratória

A Fase 2 é executada uma vez para cada base (OPS e Scopus). O *prompt* fixo (Listagem 3) define as regras de construção da consulta exploratória: separação entre conceitos centrais e secundários, tradução para o inglês e limite de complexidade. A

mensagem leva o tema escolhido na Fase 1 (Listagem 4), e o código acrescenta ao *prompt* de sistema uma dica com os campos relevantes para a base consultada (Listagem 5).

Listagem 3: Trecho do *prompt* fixo da Fase 2 (*probe_system_prompt.txt*), com o exemplo JSON compactado

```
You are a specialist in technology foresight and in building structured search queries
for patent and scholarly databases.

Your task is to receive a natural language description and return a JSON with search
fields for a RESTRICTED probe search.
[...]
## MANDATORY RULES

1. Translate all terms to English.
2. Apply light semantic expansion only.
3. Do not invent CPC/IPC codes.
4. Balance precision and recall.
5. Reason briefly first (see REASONING STEP below), then return the final answer as
valid JSON wrapped in a ““json code fence. Nothing may follow the closing fence.
[...]
## CONCEPT PRIORITY (CRITICAL)

Concepts must be divided into:

1. CORE CONCEPTS (MANDATORY)
  * Represent the main domain
  * 1 or 2 concepts [...]
  * All CORE concepts MUST be combined using AND

2. SECONDARY CONCEPTS (FLEXIBLE)
  * Context, methods, or applications
  * MUST NOT all be required simultaneously
  * Used to enrich recall, not restrict it
[...]
## QUERY COMPLEXITY CONSTRAINT (CRITICAL)

The generated query must NOT exceed a complexity score of 0.6 (on a 0-1 scale).
[...]
## OUTPUT EXAMPLE
[...]
““json
{
  "TITLE": {
    "group_operator": "AND",
    "groups": [
      { "operator": "OR", "terms": ["predictive maintenance", "condition-based maintenance"]
    },
      { "operator": "OR", "terms": ["machine learning", "artificial intelligence"] }
    ]
  },
  "ABSTRACT": { [...] },
  "IPC": [], "CPC": [], "APPLICANT": [], "INVENTOR": []
}
““
```

Theme: Autonomous UAV-based surveillance systems for border monitoring
 Description: Unmanned aerial vehicles equipped with electro-optical and thermal sensors for persistent detection and tracking of illegal crossings along land borders.
 Area of Study: Engenharia
 Keywords: UAV, sensoriamento

Listagem 5: Dica de campos acrescentada ao *prompt* de sistema para a base OPS

RELEVANT FIELDS FOR THIS SEARCH

This search targets PATENT documents. Populate TITLE, ABSTRACT, and IPC (technology classification) based on the theme. Leave other fields (CPC, APPLICANT, INVENTOR, AUTHORS, KEYWORDS, etc) empty unless explicitly mentioned in the input.

Se a consulta gerada ultrapassa o limite da função de complexidade (Seção 3.5), ou contém termos fora do inglês, o sistema repete a chamada acrescentando ao *prompt* de sistema o sufixo da Listagem 6, com os valores medidos na tentativa anterior. É esse o mecanismo de autocorreção por métrica determinística descrito no Capítulo 3.

Listagem 6: Sufixo de retentativa com as métricas da tentativa anterior

[CRITICAL RETRY #2] Previous query was TOO COMPLEX (score: 71.4/100, max: 60).

Complexity breakdown:

- OR operators: 9
- AND operators: 4
- Nesting depth: 3
- Total terms: 16

MUST simplify by:

- Use ONLY 1-2 most important concepts (reduce terms)
- Minimize OR operators within groups
- Remove nesting: avoid complex AND/OR combinations
- Use broader, more general search terms
- Maximum 2 groups per field, 2-3 terms per group

Keep it simple and focused!

[LANGUAGE ERROR] The previous query contained non-English terms: "vigilância de fronteira". MANDATORY RULE 1 requires ALL terms to be in English - translate every concept to its standard English technical term. Never copy a term directly from the user's input if it is not already in English.

A.5 Fase 3: Extração de termos

A Fase 3 não faz chamadas a LLM. Os termos candidatos são extraídos e pontuados pelo *pipeline* local de PLN descrito na Seção 4.3, e o resultado, depois da seleção feita pelo usuário, alimenta o *prompt* da Fase 4.

A.6 Fase 4: Busca final

O *prompt* fixo da Fase 4 (Listagem 7) orienta a construção da consulta definitiva a partir dos termos extraídos, com regras distintas para cada variante (específica, equilibrada e genérica). A mensagem enviada é a mesma da Fase 2 (Listagem 4); os dados

novos desta fase entram como um bloco de contexto acrescentado ao *prompt* de sistema (Listagem 8): o tipo de documento, a variante, os termos extraídos acima do limiar da variante, com a pontuação normalizada e a frequência, e os códigos de classificação observados nas patentes da busca exploratória, únicos que o modelo pode usar. O sufixo de retentativa da Listagem 6 também se aplica a esta fase.

Listagem 7: Trecho do *prompt* fixo da Fase 4 (*final_system_prompt.md*)

You are a specialist in technology foresight and in building structured search queries for patent and scholarly databases.

Your task is to receive a search topic AND a curated list of extracted terms discovered during a probe search, and return a JSON with search fields for a COMPREHENSIVE FINAL SEARCH.

This search must maximize recall while maintaining precision. Prioritize extracted terms over the original topic keywords - they were discovered from real documents and carry higher domain specificity.

MANDATORY RULES

1. Translate all terms to English.
2. Prioritize extracted terms over original keywords when building groups.
3. Do not invent CPC/IPC codes - if a "PROBE-DISCOVERED CLASSIFICATION CODES" list is provided below, you may ONLY use codes from that list; if none is provided, leave IPC/CPC empty.

[...]

SEARCH VARIANT GUIDANCE

The search variant (SPECIFIC, BALANCED, or GENERIC) will be specified in the context below.

[...]

HOW TO USE EXTRACTED TERMS

Extracted terms are already semantically ranked. Use them to populate your groups:

1. Identify 2-4 main semantic concepts from the topic and the extracted terms
2. Create ONE group per concept
3. Fill each group with extracted terms that belong to that concept
4. Add synonyms or closely related expressions if they strengthen the group
5. DO NOT mix different concepts in the same group
6. DO NOT split multi-word technical expressions (e.g. "machine learning")

[...]

Variant	TITLE groups	ABSTRACT groups	Terms per group
SPECIFIC	2-3	3-4	3-4
BALANCED	2	2-3	4-6
GENERIC	1-2	1-2	6-10

[...]

Listagem 8: Bloco de contexto da Fase 4 para a variante equilibrada na base OPS

DOCUMENT TYPE

PATENT

SEARCH VARIANT: BALANCED

Build a BALANCED query with good recall and precision. Use mid-range scoring extracted terms. Allow broader OR groups (4-6 terms). Limit AND combinations to 1-2.

EXTRACTED TERMS (score > 0.3)

- unmanned aerial vehicle (score: 1.000, freq: 42)
- border surveillance (score: 0.866, freq: 27)
- aerial surveillance system (score: 0.711, freq: 21)
- drone detection (score: 0.599, freq: 18)
- object tracking (score: 0.415, freq: 15)

PROBE-DISCOVERED CLASSIFICATION CODES

- B64C 39/02
- G06V 20/17
- H04N 7/18
- G08B 13/196

A.7 Fase 5: Geração do relatório (exemplo completo)

Na Fase 5, cada seção redigida por IA resulta de uma chamada com o *prompt* de sistema fixo da Listagem 9, comum a todas as seções, e de um *prompt* específico da seção, montado pelo código com os dados da sessão. A Listagem 10 reproduz na íntegra o *prompt* montado para a subseção 6.3 do REPTEC, “Tendências e Ciclo de Vida da Tecnologia”, a que mais concentra dados da sessão. A Tabela 11 indica a origem de cada bloco destacado.

Listagem 9: *Prompt* de sistema da redação do relatório (REPORT_SYSTEM_PROMPT), na íntegra

Você é um especialista em redação de relatórios de prospecção tecnológica no estilo REPTEC/AGITEC do Exército Brasileiro.

REGRAS OBRIGATÓRIAS:

1. Português formal, técnico e IMPESSOAL ("observa-se", "recomenda-se" - nunca "observamos", "recomendamos").
2. Use APENAS os fatos e documentos fornecidos no pedido. NÃO invente números, datas, normas, regulamentos, mercados, empresas ou autores.
3. Citações: somente no formato (SOBRENOME et al., ano), exatamente como indicado em "citar como" de cada documento. Documento marcado "não citar" não pode ser citado. Nunca escreva "Fonte:", "N/A" ou percentuais de relevância/similaridade.
4. Nunca mencione o material de apoio ("contexto fornecido", "dados fornecidos", "documentos recuperados"). Escreva como se o conhecimento fosse seu.
5. Se um dado não existir, simplesmente não trate daquele ponto - nunca escreva "[Informação não disponível]" nem frases sobre ausência de dados.
6. Mantenha o foco no TEMA do relatório. Documentos sobre assuntos periféricos servem só como exemplo pontual, nunca como assunto central.
7. Números em formato brasileiro: vírgula decimal (17,3%) e ponto de milhar (1.800).
8. Não inclua título/cabeçalho da seção (ex.: "## Seção") - comece direto pelo primeiro parágrafo.
9. Parágrafos corridos, sem listas com marcadores e sem Markdown.

Listagem 10: *Prompt* montado para a subseção 6.3 do relatório, na íntegra

DOCUMENTOS DE APOIO (use só o que for pertinente ao tema):

Documentos recuperados sobre o tema (Resultados - Tendências e Ciclo de Vida da Tecnologia)

Documento 1 - citar como (SILVA et al., 2021)

Persistent border surveillance with fixed-wing UAVs and thermal cameras

This work presents a surveillance architecture based on fixed-wing unmanned aerial vehicles carrying thermal imaging payloads for night-time detection of border crossings.

Documento 2 - citar como (NGUYEN et al., 2023)

Multi-UAV path planning for wide-area border monitoring

A cooperative path planning method is proposed for fleets of drones covering long land borders, reducing revisit time by 34% in simulated scenarios.

Documento 3 - sem autoria identificada (não citar)

Aerial surveillance apparatus with automatic target tracking

An apparatus comprising an unmanned aircraft, a gimbal-mounted camera and an onboard processing unit that tracks moving targets and transmits alerts to a ground station.

FIGURAS E QUADROS DESTA SEÇÃO (os números abaixo são os mesmos desenhados em cada imagem):

- id: article_s_curve | Figura: Curva S relativa às informações científicas.
Dados: 1.842 documentos acumulados entre 2005 e 2025; GP = 2014, MP = 2024, SP = 2033; saturação atual estimada em 61,3% do nível de saturação (3.004)
- id: patent_s_curve | Figura: Curva S relativa às informações tecnológicas.
Dados: 967 documentos acumulados entre 2008 e 2025; GP = 2017, MP = 2026, SP = 2034; saturação atual estimada em 38,9% do nível de saturação (2.486)

COMO APRESENTAR CADA FIGURA/QUADRO (padrão do REPTEC - obrigatório para TODOS os itens da lista acima):

1. Um parágrafo que APRESENTA a figura e cita pelo marcador [[REF:id]], precedido da palavra Figura ou Quadro. Ex.: "A Figura [[REF:article_s_curve]] mostra a distribuição das publicações científicas ao longo do tempo."
2. Na linha seguinte, sozinho, o marcador [[FIG:id]] - é onde a imagem entra.
3. Logo depois, um parágrafo que INTERPRETA os dados da figura: cite os números dela (totais, pico e ano do pico, primeiros colocados e suas quantidades), descreva a tendência (crescimento, estabilização, queda) e o que isso indica sobre o tema.

Exemplo de estrutura:

"No total, foram identificadas 321 publicações científicas entre 1992 e 2022. A Figura [[REF:article_yearly_volume]] mostra a distribuição dessas publicações ao longo do tempo.

[[FIG:article_yearly_volume]]

Entre 1993 e 2013, observa-se crescimento gradual, com o pico de 25 publicações em 2013; a partir de então, nota-se queda sustentada, chegando a 2 publicações em 2022, o que sugere redução do interesse acadêmico pela tecnologia."

Regras: use cada figura exatamente uma vez; nunca coloque duas figuras seguidas sem texto entre elas; nunca termine a seção com figuras; não invente ids; todo número sobre uma figura deve vir dos "Dados" dela.

ESTÁGIO DO CICLO DE VIDA (calculado a partir das curvas S - use exatamente estes estágios e anos, sem reinterpretar):

- Publicações científicas (artigos): estágio de MATURIDADE (GP = 2014, MP = 2024, SP =

2033).

- Depósitos de patentes: estágio de CRESCIMENTO (GP = 2017, MP = 2026, SP = 2034). GP = ponto de crescimento (10% da saturação), MP = ponto médio (50%), SP = ponto de saturação (90%).

Escreva a subseção 6.3 TENDÊNCIAS E CICLO DE VIDA DA TECNOLOGIA do relatório de prospecção sobre: [Autonomous UAV-based surveillance systems for border monitoring](#)

No estilo do REPTEC, em 3 a 5 parágrafos: descreva a evolução das publicações e dos depósitos ao longo do tempo; discuta SEPARADAMENTE a curva S dos artigos e a curva S das patentes, informando o estágio de cada uma e os anos GP/MP/SP acima (ex.: "MP = 2011"); interprete o que isso indica sobre o interesse acadêmico e comercial pela tecnologia.

Tabela 11: Origem dos dados da sessão no *prompt* da subseção 6.3

Bloco	Origem
Documentos de apoio	Busca por similaridade no ChromaDB entre o tema mais a descrição da seção e o título e resumo dos documentos da busca final, com corte relativo de relevância (Seção 2.5). A citação "(SOBRENOME et al., ano)" é calculada a partir dos autores ou inventores; documentos sem autoria identificada são marcados para não serem citados.
Dados das figuras	Resumo numérico de cada gráfico, persistido na tabela <code>session_chart</code> no momento em que o gráfico é gerado. São os mesmos números desenhados na imagem.
Estágio do ciclo de vida	Ajuste das curvas S (Seção 2.2): o estágio resulta da comparação do último ano completo com os anos GP, MP e SP de cada curva.
Tema	Tema escolhido pelo usuário na Fase 1.

O texto devolvido pelo modelo passa por verificações determinísticas: citações que não correspondem a nenhum documento recuperado são removidas, números são convertidos para o formato brasileiro e marcas proibidas, como menções ao material de apoio ou uso da primeira pessoa, são detectadas. Se alguma marca proibida for encontrada, a seção é regenerada uma vez com o acréscimo da Listagem 11, que lista os problemas identificados.

Listagem 11: Acréscimo ao *prompt* da seção na regeneração

ATENÇÃO: a versão anterior deste texto foi rejeitada pelos seguintes problemas:

- citação no formato "(Fonte: ...)"
- referência ao material de apoio ("contexto fornecido", "dados apresentados"...)
- primeira pessoa do plural (use a forma impessoal: "recomenda-se")

Reescreva o texto inteiro sem nenhum desses problemas, seguindo as regras obrigatórias

.

Por fim, na etapa de revisão, cada parágrafo gerado por IA pode ser enviado a um modelo revisor, com o *prompt* de sistema da Listagem 12 e a mensagem "Texto a revisar:"

seguida do parágrafo. As correções sugeridas são apresentadas ao usuário, que decide se as aplica.

Listagem 12: *Prompt* de sistema da revisão do texto (AI_REVIEW_SYSTEM_PROMPT), na íntegra

```
Você é revisor de língua portuguesa (norma culta do Brasil) de relatórios técnicos.
Corrija SOMENTE erros de: concordância verbal, concordância nominal (número e gênero),
acentuação e ortografia.
NÃO reescreva estilo, NÃO troque palavras corretas por sinônimos, NÃO altere números,
siglas, nomes próprios, citações "(AUTOR, ano)" nem comandos LaTeX (\textit{...}, \ref
{...}, \%, \&).
Responda APENAS com JSON no formato:
{"correcoes": [{"original": "<trecho EXATO copiado do texto, curto, contendo o erro>",
"corrigido": "<o mesmo trecho corrigido>", "motivo": "concordância verbal|concordâ
ncia nominal|acentuação|ortografia"}]}
Se não houver erros, responda {"correcoes": []}.
```